

Hardware Platforms and Programming of Embedded Systems

Lecture 6 - Interfaces and protocols 2

Advanced interfaces - USB, Ethernet, Wireless

Dr.-Ing. Alexander Knaub

2024-11-21

Institute of telecommunications (INÜ)

Contents

General Concepts

Universal Serial Bus (USB)

Controller Area Network (CAN)

Ethernet

Wireless Interfaces

- Wireless LAN

- Bluetooth

- Zigbee

- LoRaWAN

General Concepts

Advanced Communication Interfaces

Common to advanced communication interfaces covered in this lecture:

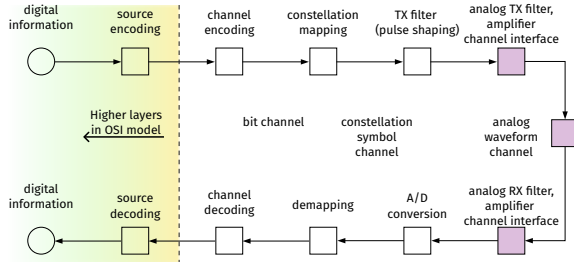
- Layer 1 in the OSI model (PHY, transmission of raw bits over a physical medium) is implemented in hardware – on a separate chip or integrated into the MCU/SoC.
- Multiple peripherals can communicate at high data rate with the host, sometimes also with each other.
- The interfaces offer high configurability, which makes them flexible but also complex.
- Application software relies on driver functions to access the interface, and to automatically maintain the connection.

Image source: Pictures of things, CC BY-SA 4.0 via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:Wlan-router-tp-link-backside.jpg>



Physical layer in high-rate communications

- Transmitter (TX) encodes bits to symbols, converts them to an analog signal. Receiver (RX) does the reverse process.
- Higher data rate requires (generally) higher signal bandwidth. Channel bandwidth is limited by regulations, and/or cost, and/or path loss profiles.
- On the channel and in TX/RX, the signal is attenuated and distorted by noise and interference.
- RX compensates the channel effects and technology impairments → more complex than TX



The fundamentals of physical layer design are covered by lectures of Institute of Telecommunications (INUE).

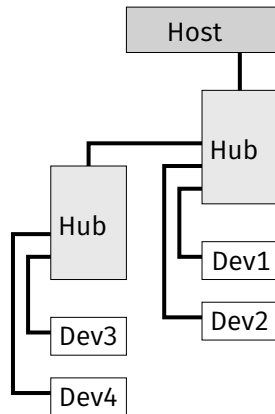
Image: Adopted from M.Sc. lecture "Wireless Communications (Comms. 3)" by INUE, Prof. ten Brink

Universal Serial Bus (USB)

Universal Serial Bus (USB)

Wired interface for connecting peripherals to a host.

- Logical and physical topology: *Star*
 - On the physical layer only two devices are connected directly, optionally one or both of them is a hub.
 - The host controls the bus.
 - Devices can only communicate with the host.
- Eliminates the need for separate interfaces for each peripheral (a variety of different connectors somewhat defeats this purpose).
- Plug and play capability and power supply simplifies the use of peripherals.



Note: The USB communication on various layers of the OSI model is specified on 650 pages in [1]. Easier to read and sufficiently exact explanations can be found on [2] and [3].

USB generations

Generations and maximum data rates:

- USB 1.1: 1.5 Mbit/s (Low Speed, LS), 12 Mbit/s (Full Speed, FS)
- USB 2.0: 480 Mbit/s (High Speed, HS)
- USB 3.0: 5 Gbit/s (Super Speed, SS)
- USB 3.1: 10 Gbit/s (Super Speed+)
- USB 3.2: 20 Gbit/s (Super Speed+)

USB connector types:

USB 1.1 – 2.0



A



B



Mini-A



Mini-B



Micro-A



Micro-B

USB 3.0



A



B



Micro-B



USB 3.1 and newer: Type-C

Image source: Milos.bmx, CC BY-SA 3.0 via Wikimedia Commons

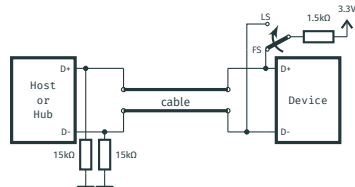
https://commons.wikimedia.org/wiki/File:USB_2.0_and_3.0_connectors.svg

Image source: Andreas Pietzowski, CC BY-SA 3.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:USB_Type-C.svg

USB 1.1 electrical interface

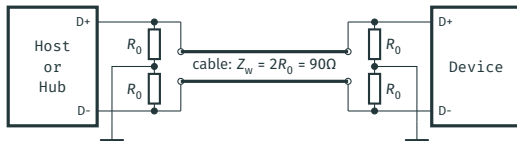
- Pull-down on both lines at host, pull-up on one of the lines at device
- Host senses the single-ended voltage on the data lines to detect the presence and speed of a device (LS vs. FS)



- Three line states are used for signaling, defined by the voltage on the wires:
 - Single-ended zero "SE0": both data lines are pulled low
 - "J": Differential, as in idle state (D+ high in FS, D- high in LS)
 - "K": Differential, opposite to "J"
- Different sequences of line states are used to signal special events, e.g.
 - "Reset" (SE0 for at least 10 ms)
 - "Sync" at packet start (KJKJKJJK)
 - Packet end (2x SE0, 1x J)
 - Transition from FS to HS mode in USB 2.0 (special protocol, called "chirp")

USB 2.0 electrical interface

- During speed negotiation, line termination is as in FS mode
- After switching to HS mode, termination changes:
 - Pull-ups and pull-downs are disabled, line is terminated with $45\ \Omega$ to ground on both wires. This matches the characteristic impedance of the cable ($Z_w = 90\ \Omega$).
 - Data is transferred by driving a current of 17.78 mA into the D+ or D- line. The voltage difference between the lines is then $\pm 400\text{ mV}$.



- USB 3.0 and newer use two signal pairs with more complex signal processing and coding.

USB Data Link Layer (1)

- Data is transferred in *packets* of 1, 3, or more bytes, with a maximum of 1024 bytes per packet, depending on the packet type.
- The packet type is encoded in the first byte of the packet (Packet identifier *PID*).
- 1-byte packets are so-called *handshake* packets and contain **no checksum**,
3-byte packets contain a **CRC-5** checksum, the longer ones a **CRC-16** checksum.

Handshake packets

PID
1 byte

Data packets

PID	Data	CRC-16
1 byte	0-1024 byte	2 byte

Token packets

PID	Address	Endpoint	CRC-5
8 bit	7 bit	4 bit	5 bit

Start of Frame packets

PID	Frame No.	CRC-5
8 bit	11 bit	5 bit

Interface between PHY and Link Layer

- Access to the data lines, termination, proper serialization, etc. is implemented in hardware
- If the data link layer is implemented on the same chip, the standardized parallel interface "**USB 2.0 Transceiver Macrocell Interface**" (UTMI+) is used.
- If the PHY is a separate chip, the high number of signals used in UTMI+ is not practical.
- Compromise: **UTMI+ Low Pin Interface (ULPI)** with 12 pins, providing serial access to UTMI+ signals.
- USB 3.0 and newer use different interfaces, e.g. **PHY Interface For the PCI Express and USB 3.0 Architectures (PIPE)**.

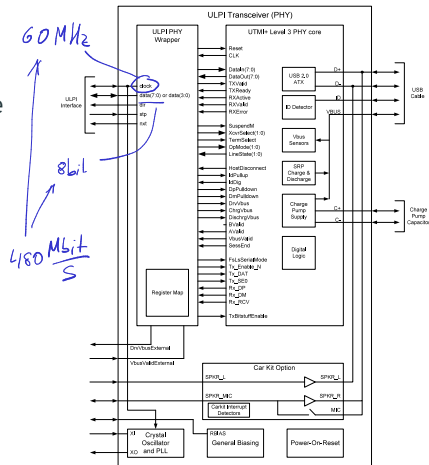


Image source: ULPI specification [4] (also contains UTMI+ specification)

USB Network & Transport Layers

- A device has a unique *address* (7 bit wide) assigned by the host during *enumeration*.
- Packets are grouped into *transactions*, always initiated by the host.
- The first packet of a transaction is a *token* packet, which contains the address of the target device, the endpoint number within the device, and the requested direction of transfer - "host → device" (*OUT*) or "device → host" (*IN*)
- The second packet is a *data* packet, followed by the third *handshake* packet (ACK/NAK).
- Depending on transfer type, retransmissions can be required if errors are detected.

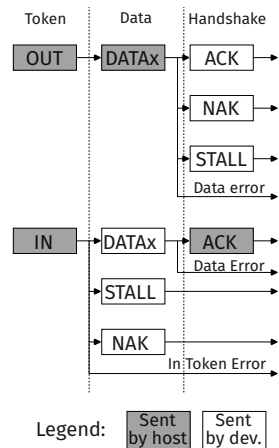


Image: Transaction of a *Bulk* transfer, adapted from [3]

USB Enumeration

- When a device is detected, the hub performs speed detection - LS, FS or HS - and reports this to host.
- Host retrieves so-called *descriptors* from the device - predefined data structures containing information about the device, e.g.
 - device descriptor (vendor and product ID, device class, ...)
 - configuration descriptor (offered interfaces, power consumption, ...)
 - ...
- During the process, host assigns an address. If a driver software is found by OS, the device is *configured* and then ready for use.

USB Usage

A device can be a *function* or a *host*.

- A function is usually a microcontroller or an FPGA/ASIC.
 - Lower layer functionality can be accessed via driver functions or IP cores.
 - The actual payload is generated/consumed by the application.
- A host is usually a CPU-based device, running an operating system providing the USB host stack.
 - The kernel takes care of the USB host functionality, especially enumeration and flow control, implementing of host controller interfaces: OHCI, UHCI, EHCI, xHCI.
 - The application can either access the "payload" functionality via an existing device class driver, or implement the endpoint handling itself.
 - Some low-level preparation steps upon device detection can be done by UDEV on Linux.
 - In case of custom devices, the host application needs to implement the protocol. The library libusb can be used for this task.

Device classes

- Session- and presentation-layer interface specifications for different types of devices.
- Allow the host application to access the device functions without knowing the details of the underlying hardware, and without the need for a custom driver
⇒ Plug and play capability.
 - Human Interface Device (HID)
 - Mass Storage Device (MSD)
 - Audio Device
 - Printer
 - ...

Stopped here on 21.11.2024

Controller Area Network (CAN)

Controller Area Network (CAN)

- Developed by Bosch for automotive applications.
- Used also in complex embedded systems like industrial automation, medical equipment, etc.
- Physical topology: *Bus*:
All devices are connected to the same two wires.
- Logical topology: *Bus*:
Every device can communicate with every other one
(depending on application, *star* topology can be used on higher layers).

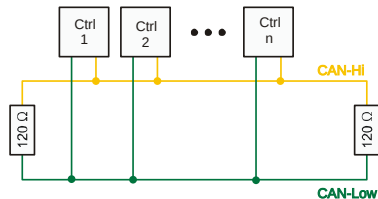


Image: High-speed CAN bus with termination resistors Image source: Adopted from Stefan-Xp, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:CAN-Bus_Elektrische_Zweidrahtleitung.svg

CAN Physical Layer

- Two wires: CAN-High (CAN-H) and CAN-Low (CAN-L)
Often twisted pair to reduce interference.
- Differential signalling with two bus states:
Dominant (logical zero, CAN-H > CAN-L) and *Recessive* (logical one CAN-H $\approx$ CAN-L)
- For high-speed CAN variants, the bus is terminated with 120 Ω resistors at both ends.

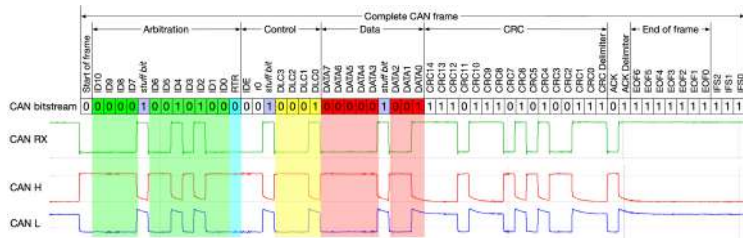


Image source: Dr Ken Tindell, CC BY-SA 4.0 via Wikimedia Commons

<https://commons.wikimedia.org/wiki/File:CAN-bus-frame-with-stuff-bit-and-correct-CRC.png>

CAN - higher layers example

CANopen - One definition of protocols at network and application layers, building on CAN as physical and data link layers.



- Specified and maintained by the organization CiA (CAN in Automation).
- Defines a network of *nodes*, based mainly on a client-server architecture.
- Specifies a set of standard interfaces "object dictionaries" for different application areas, e.g. motor control (CiA 402), sensors (CiA 404), batteries (CiA 418/9), lifts (CiA 417), etc.
- Often used in complex embedded devices, reusable software libraries are available.

Image source: CAN in Automation, public domain via Wikimedia Commons
https://de.wikipedia.org/wiki/Datei:Canopen_logo.svg

Ethernet

Ethernet

- Ethernet specifies the physical and data link layer of a local area network (LAN).
- Logical topology: *Bus* - Every connected computer can communicate with every other one.
- Physical topology (today): *Star* - A switch or hub device is needed to forward the traffic between the computers.

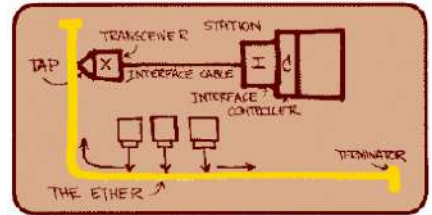


Image: Attributedly Robert Metcalfe's original sketch of the Ethernet concept on a napkin (1973)

<https://hackaday.com/2020/10/19/ethernet-at-40-from-a-napkin-sketch-to-multi-gigabit-links/>

Ethernet - Physical Layer (1)

- The electrical interface is standardized in IEEE 802.3.
 - The medium is a cable with 4 twisted wire pairs, with a characteristic impedance of $100\ \Omega$.
 - Usually shielded to reduce cross-talk and sensitivity to electromagnetic interference (EMI) - individual pairs and/or all wires together.
 - 10 and 100 Mbit/s standards use only two pairs, 1 Gbit/s uses all four wire pairs in a cable and in a "8 position 8 contact" connector (8P8C) - commonly known as "RJ45".

10BaseTx
100BaseTx
1GBaseTx

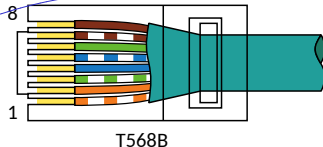


Image source: Miha Ulanov, public domain via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:RJ-45_TIA-568B_Left.svg

Ethernet - Physical Layer (2)

A transformer is used to couple the signal to the wire pairs.

- Galvanic isolation between sender and receiver (no distortions caused by ground offset)
- Rejection of common mode noise
- Special encoding patterns ensure that the signal has no DC component.

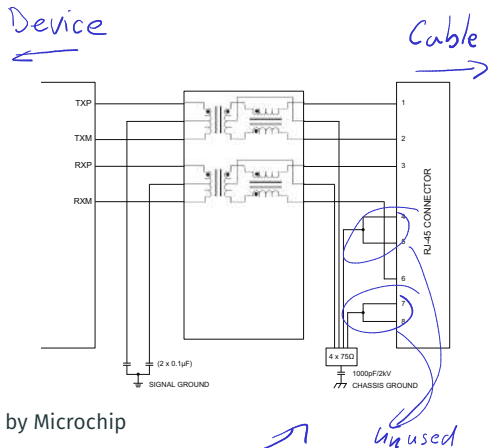


Image source: Every 100BASE-TX PHY datasheet, e.g. KSZ8091RNA by Microchip

→ 2 wire pairs used

Ethernet - Data Link Layer

On Ethernet, the data link layer is often called *Media Access Control* (MAC) layer.

- PHY and MAC Layer are usually implemented by different chips - the latter sometimes integrated into SoCs. This allows using the same link layer implementation for different PHYs.
- A standardized interface between PHY and MAC is *Media Independent Interface* (MII), or in a version with less pins *Reduced Media Independent Interface* (RMII).
- Every device has a unique 48 bit MAC address, assigned by the manufacturer.
- Data is transferred in *frames* of 64 to 1518 bytes.
- A frame contains a CRC-32 checksum, which is checked by the receiver. If an error is detected, the frame is discarded, causing a packet error. Higher layers (e.g. TCP) are responsible for retransmission.

Similar
to UTM+
& ULPI in USB

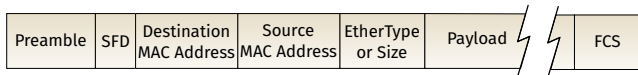


Image source: Mikm, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Ethernet_frame.svg

Internet Protocol - IP (1)

- Ethernet specifies the physical and data link layer of a local area network (LAN).
- Network of networks: The Internet.
- The Internet Protocol (IP) defines addresses and packet formats to specify the *network layer*:
How packets are routed *between networks*.
- IP does not require Ethernet as a physical layer.
Alternatives: Wireless LANs or Wide-Area Networks (WAN).

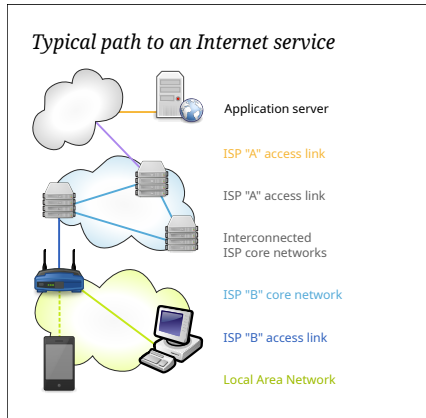


Image source: Ferran, CC BY-SA 4.0 via Wikimedia Commons

[https://commons.wikimedia.org/wiki/File:Internet_access_diagram_\(english\).svg](https://commons.wikimedia.org/wiki/File:Internet_access_diagram_(english).svg)

Internet Protocol - IP (2)

- An IP address is part of packet header and contains a network part and a host part.
- A *router* forwards packets between networks based on the network part of the destination address.
- Addresses are network-specific, and thus easier to handle than MAC addresses.
⇒ IP is used for addressing also within one LAN.
- Special mechanisms for easier address handling:
 - Dynamic Host Configuration Protocol (DHCP):
IP addresses are automatically assigned by a central server,
no need for manual configuration
 - Domain Name System (DNS):
IP addresses are *resolved* from human-readable names by a server;
resolution can be delegated to other servers.

Wireless Interfaces

General Concepts

- The analog TX signal is modulated from *baseband* to a carrier frequency → frequency division multiplexing (FDM) is possible - and necessary to avoid interferences.
- Higher data rate requires higher bandwidth, but: bandwidth is expensive (either real fees per MHz of licensed spectrum, or limited bandwidth and cost of interference in unlicensed spectrum).
- Optimal spectrum utilization is a continuous research effort (spectrum efficiency: bit/s achievable per Hz of bandwidth).

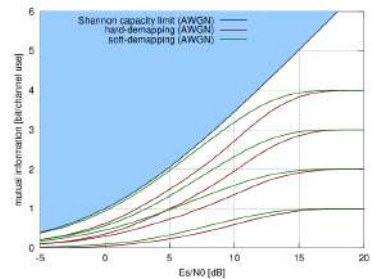


Image: Achievable spectral efficiency depending on modulation and signal-to-noise ratio (SNR)
Source: Webdemo "Physical Layer Performance Measures" [5]

Wireless LAN

- Standardized in IEEE 802.11 and amendments, basically a wireless extension of Ethernet.
- Operation in unlicensed spectrum (not unregulated!).
 - 2.4 ... 2.5 GHz - 802.11b/g (Wi-Fi0...Wi-Fi3), 802.11n (Wi-Fi4)
 - 5.1 ... 5.9 GHz - 802.11a/n, 802.11ac (Wi-Fi5)
 - 57 ... 71 GHz - 802.11ad/ay (WiGig)
 - 5.9 ... 7.1 GHz - 802.11ax (Wi-Fi6) and 802.11be (Wi-Fi7, draft)
 - Regional restrictions apply, e.g. requirement for dynamic frequency selection (DFS) or transmit power control (TPC) in 5 GHz band in some countries.
- Wi-Fi: Trademark and certification program of the Wi-Fi Alliance (ensures interoperability of devices from different manufacturers)
- Peer-to-peer communication is possible, but WLAN is usually used in *infrastructure mode*: all communication is routed through a central *access point* (AP).



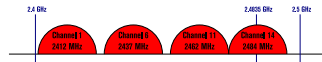
Wireless LAN - PHY

- Basic resource unit: *channel* of 20 MHz bandwidth
- A channel consists of 64 *subcarriers*, combined by OFDM modulation (Orthogonal Frequency Division Multiplexing).
- Narrow-band subcarriers allow easy equalization of the frequency-selective channel.
- Up to 8×20 MHz channels can be combined to increase the bandwidth/throughput.
- Air is shared medium $\Rightarrow$ a channel is occupied by one device (*station*) at a time.

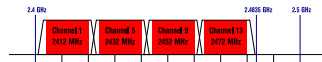
Image source: Rbeede, Liebeskind, CC BY 3.0 via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:NonOverlappingChannels2.4GHz802.11-en.svg>

Non-Overlapping Channels for 2.4 GHz WLAN

802.11b (DSSS) channel width 22 MHz



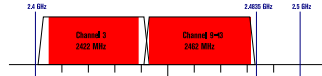
802.11g/n (OFDM) 20 MHz ch. width – 16.25 MHz used by sub-carriers



20MHz ch. width, without ch. 12 & 13 (United States customary):



802.11n (OFDM) 40 MHz ch. width – 33.75 MHz used by sub-carriers



Wireless LAN - Performance improvements

- Channel encoding adds redundancy bits to allow error correction at RX. Added overhead can be between 16.7% and 100% (code rate R_c of $5/6$ down to $1/2$), depending on wireless link quality.
- Also depending on link quality, all subcarriers are modulated by a N-QAM, allowing up to $M_b = \log_2(N)$ bits per symbol on each carrier, with N up to 1024. The combination of channel coding and modulation is called *Modulation and Coding Scheme* (MCS).
- Since 802.11n, multiple antennas can be used - *Multiple Input Multiple Output* (MIMO) - to increase reliability and/or data rate by having N_s *spatial streams* at cost of increased complexity.

Note: Modulation and channel coding is covered in lectures by INUE.

WLAN is one of experiments in the master's lab course at INUE.

Max. data rates in different exemplary 802.11n configurations (short guard interval):

N_s	R_c	M_b	Data rate Mbit/s	
			20 MHz	40 MHz
1	$1/2$	1	7.2	15
1	$3/4$	2	21.7	45
1	$1/2$	4	28.9	60
1	$5/6$	6	72.2	150
2	$5/6$	6	144.4	300
3	$5/6$	6	216.7	450

WLAN - Security

- Unlike wired Ethernet, WLAN works without physical access to the network.
- Authentication of users and access points Encryption of the transmitted data is necessary and part of the standard
 - Wired Equivalent Privacy (WEP) - considered insecure
 - Wi-Fi Protected Access (WPA), later WPA2 and WPA3
 - more advanced encryption protocols e.g. AES-CCMP in 802.11n
- Key distribution is a challenge, especially in large networks.
 - Pre-shared key (PSK) - same key for all users (WPA-Personal)
 - 802.1X - authentication server, e.g. Remote Authentication Dial-In User Service (RADIUS) - WPA-Enterprise

WLAN - Usage



- On embedded systems, WLAN is often used for configuration and maintenance - also from the Internet.
- Sometimes, WLAN is the only interface for user interaction. then usually a web-interface is offered, or an API to be accessed by an app.
- The drivers for the WLAN PHY and MAC are usually included in the OS (Linux).
- Management software is needed to configure the WLAN interface.
Desktop: configuration GUI; Embedded: text-based configuration files.
 - Client side: wpa_supplicant (Linux)
 - Access point side: hostapd (Linux)
 - On MCU-based systems, the management is part of the (single) application.

Image: Tibber Pulse, uses WLAN as a gateway between the energy meter interface and the electricity reseller server.

Source: <https://tibber.com/de/store/produkt/pulse-ir>

Bluetooth - General information

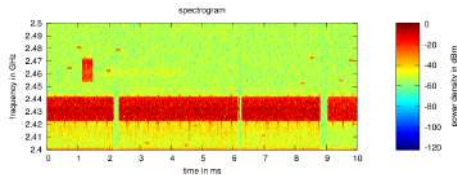


- Named after Danish king Harald Blåtand (Harald Bluetooth), who united Danish tribes into a single kingdom in the 10th century. The logo is a combination of the old Germanic runes for H and B.
- Used to build *personal area networks* (PAN)
- Standardized in IEEE 802.15.1, maintained by Bluetooth Special Interest Group (SIG).
- Used as a wireless replacement for serial short-range cables mainly for multimedia applications and for input devices.

Image source: bluetooth.com, public domain via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:BluetoothLogo.svg>

Bluetooth - Technology

- Operates in the 2.4 GHz ISM band (Industrial, Scientific, Medical), uses frequency hopping spread spectrum (FHSS) as modulation.
- Data rates up to 2.1 Mbit/s achievable with Enhanced Data Rate (EDR), introduced in 2004 with generation 2.0
- Bluetooth Low Energy (BLE) with data rates up to 1 Mbit/s and down to 10% of energy consumption, introduced in 2010 with generation 4.0
- BLE is also used for beacon applications, use cases: advertising, interaction (museums), indoor navigation, Invention with Covid-19: contact tracing
- Current generations 5.x introduce higher data rates and range on BLE, improved localization, and mesh networking.



Spectrum of WLAN and Bluetooth

- Zigbee defines network and application layer protocols, while PHY and MAC are based on IEEE 802.15.4.
- Data rates: 20 kbit/s up to 250 kbit/s in the 2.4 GHz ISM band
- Use cases: Wireless sensor networks, e.g.:
Home automation, environment sensing, industrial control, etc.
- Transmission distances: up to 100 m.
- A mains-powered device can act as a *router* to forward data between other devices, thus allowing to build large mesh networks.
- Battery-powered devices can save energy by only waking up at predefined times to send/receive data.



LoRaWAN

- Energy efficient wireless technology with very high range, specifically designed for the Internet of Things (IoT).
- LoRaWAN is a protocol stack on top of LoRa, which is a proprietary PHY and MAC layer technology developed by Semtech.
WAN = Wide Area Network, distances up to 15 km are possible, depending on environment.
- Data rates: 0.3 kbit/s up to 50 kbit/s in the 868 MHz (Europe) or 915 MHz (US) band.



A LoRa module, with antenna and SPI wiring.

Image source: Cnd, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:LoRa_Module_with_antenna_and_SPI_wires_attached.jpg

Thank you for the attention!

Next lecture: Power supply
Stable power sources for embedded systems

References

Bibliography i

- [1] *Universal Serial Bus Specification, Revision 2.0*, USB Implementers Forum, 4 2000. [Online]. Available: https://www.usb.org/sites/default/files/usb_20_20230224.zip
- [2] *USB Made Simple*, MQP Electronics Ltd, 2008. [Online]. Available: <https://www.usbmadesimple.co.uk/index.html>
- [3] C. Peacock, "Usb in a nutshell," *Beyond Logic*, 4 2018. [Online]. Available: <https://www.beyondlogic.org/usbnutshell/usb1.shtml>
- [4] S. Reemeyer, *UTMI+ Low Pin Interface (ULPI) Specification, Revision 1.1*, Philips, 10 2004. [Online]. Available: https://web.archive.org/web/20070226164413/http://www.ulpi.org/ULPI_v1_1.zip
- [5] K. Oestreich and M. Geiselhart, "Physical Layer Performance Measures," Institute of Telecommunications, University of Stuttgart, Germany, Nov 2023, webdemo. [Online]. Available: <http://webdemo.inue.uni-stuttgart.de>

Bibliography ii

- [6] M. Ziegler, “802.11 WLAN spectrum analysis,” Institute of Telecommunications, University of Stuttgart, Germany, Nov 2023, webdemo. [Online]. Available: <http://webdemo.inue.uni-stuttgart.de>

Hardware Platforms and Programming of Embedded Systems

Lecture 7 - Power Supply

Stable power sources for embedded systems

Dr.-Ing. Alexander Knaub

2024-12-05

Institute of telecommunications (INÜ)

Contents

Power sources

- AC to DC conversion

- Batteries

 - Non-rechargeable batteries

 - Rechargeable batteries

Power regulators

- Linear regulators

- Switching regulators

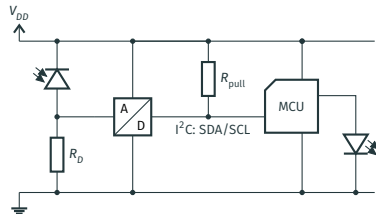
Power sources

General considerations

All digital circuits need *stable* DC supply voltage, also analog amplifier circuits. Reasons:

- Reliably distinguish between high and low logic levels
- Sensors usually contain a AD converter which needs a stable reference voltage
- Some components don't work at all if the voltage is too low
 - Example: LED
 - needs a minimum voltage to light up
 - Example: Flash memory
 - needs a minimum voltage to induce tunneling
 - ...

Image: Exemplary circuit with components depending on a stable supply voltage - light sensor (photo diode), AD converter, LED, microcontroller, I²C bus



Power sources

- AC Supply: Energy comes from the electricity grid
 - AC voltage has a quite stable level and frequency (230V, 50Hz in Europe; 120V, 60Hz in the US)
 - stabilized by the grid and the power plants
 - Needs to be converted to DC voltage for digital circuits: Transformer, Rectifier, LP Filter (see next slide)
- DC sources: Battery, Solar cell, Fuel cell
 - All generate a DC voltage of unstable magnitude
 - Obviously, no need for AC/DC conversion

Note: The topics of generating, storing and distributing electrical energy are covered in more depth by lectures of the institutes ILEA, IEH and IEW.

Image source: Substation: Dietmar Rabich, CC BY-SA 4.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Dülmen,_Umspannstation_--_2014_--_0005.jpg

Image source: Ingenuity helicopter: NASA, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Mars_helicopter_on_sol_46.png



Small electrical substation, converts 110kV to 20kV



Ingenuity helicopter on Mars, powered by solar cells and a battery

AC to DC conversion (classical)

- Down-conversion by a transformer:

$$V_{lo} = \frac{N_2}{N_1} \cdot V_{hi}; I_{hi} = \frac{N_1}{N_2} \cdot I_{lo}$$

- Rectifier: Converts AC to DC

Example: Bridge rectifier (4 diodes)

- Smoothing filter: Reduces the ripple

Usually: Large capacitor, sometimes with an inductor

- Reduce form factor: Switching power supply (next slide)

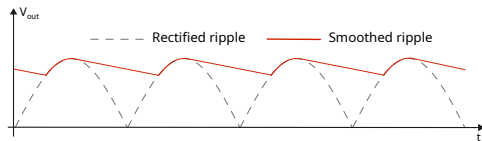
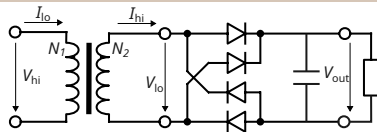


Image source: adopted from Wdwd, CC BY 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Gratz_rectifier.en.svg

Image source: adopted from SpinningSpark, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Smoothed_ripple.svg

AC to DC conversion - Switched mode power supply (SMPS)

- Transformers are more efficient at higher frequencies
⇒ Less iron ⇒ smaller form factor is possible
- Additional rectification, filtering and on/off switching before the transformer (primary side)
- Often: control feedback loop to regulate the output voltage by adjusting the on/off switching behavior

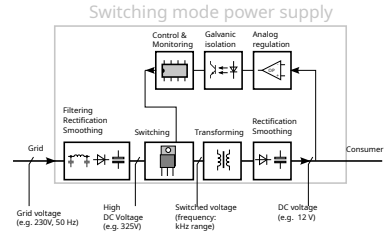


Image source: Adopted from Quark48, CC BY-SA 2.0 via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:Schaltnetzteil.svg>

Image source: Raimond Spekking, CC BY-SA 4.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:USB_power_adapter_for_Apple_iPod,_Model_A1205,_by_Foxlink_Technology_Ltd-2045.jpg



Fundamentals about batteries

- Batteries are electrochemical cells, more specifically *galvanic cells*
- Chemical *reduction/oxidation* reactions produce a voltage difference between the electrodes: Surplus of electrons at the anode, deficit at the cathode. Positive and/or negative ions moving in the electrolyte compensate the charge difference.
- If no current is drawn, the voltage is stable (equilibrium)
- If electrons and ions are allowed to flow and complete a circuit, the concentrations of reactive chemicals decrease $\Rightarrow$ Battery *discharges*
- For some battery types, the reaction can be reversed by applying a voltage $\Rightarrow$ Battery can be *recharged* (secondary cell, in contrast to primary cell which can only be discharged)

Image source: User:Brianac, public domain via Wikimedia Commons

<https://commons.wikimedia.org/wiki/File:Batteries.jpg>

Image source: Rehua, CC BY 3.0 via Wikimedia Commons

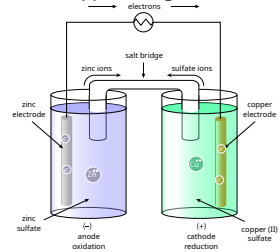
https://commons.wikimedia.org/wiki/File:Galvanic_cell_labeled.svg

Examples:

Variety of battery types



Basic copper-zinc galvanic cell



Batteries: Properties

- Nominal voltage and recharge capability is specific to the chemical composition of the electrodes and the electrolyte
- Voltage can differ from the nominal value due to different parameters:
 - The concentration of the chemicals (i.e. state of charge),
 - Temperature (influences the reaction),
 - Current drawn from the battery (internal resistance),
 - Age of the battery (state of health), ...
- The voltage is not stable over time, and not stable among different batteries $\Rightarrow$ Voltage regulation/stabilization is needed (see later)
- Elementary cells can be combined to increase the voltage or the capacity



Image source: Lead holder, CC BY-SA 3.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:9V_innards_3_different_cells.jpg

Batteries: Capacity

Battery capacity

Amount of electric charge it can deliver at the *rated* voltage.

Defined as product of 20 hours multiplied by the current that a new battery can consistently supply for 20 hours at 20 °C, while remaining above a specified terminal voltage per cell.

- Amount of charge Q (number of electrons that can be drawn)
Basic unit: Ampere-second, Coulomb; $1 \text{ As} = 1 \text{ C}$, **derived units:** Ah, mAh
- Other way to express capacity: Energy E (work that battery can deliver)
Basic unit: Joule, Watt-second; $1 \text{ Ws} = 1 \text{ J}$, **derived units:** Wh, kWh
- Relationship (assuming constant nominal voltage):
 $E = V \cdot Q$, with $Q = I \cdot t$ and $E = P \cdot t$ and $P = V \cdot I$
- Basically, capacity is defined by the *amount* of chemicals in the battery
- Real-life usable capacity depends on e.g. the duty cycle, temperature, and the cutoff voltage of the application.



Image: Differently sized batteries with the same chemistry (alkaline) and voltage (1.5 V), but different capacity

Image source: Adopted from Lead holder, CC BY-SA 3.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Batteries_comparison_4,5_D_C_AA_AAA_A23_9V_CR2032_LR44_matchstick-vertical.jpeg

Batteries: Energy density

- Capacity depends on the size of the battery
- Energy density: Amount of usable energy per volume or mass, characteristic of the battery chemistry, independent of the size
 - Energy density actually defined per volume, e.g. Wh/l
 - Also commonly used: Energy per mass (gravimetric energy density) referred to as *specific energy*, e.g. Wh/kg

Some examples:

Material(s)	Energy density [Wh/l]	specific energy [Wh/kg]	usage
Alkaline	360	130	not rechargeable, recyclable
Lithium metal	1200	500	not rechargeable, recyclable
Lithium-ion	700	300	rechargeable, recyclable
Supercapacitor	16.7	<i>approx</i> 10	rechargeable, recyclable
Gasoline	9500	12900	combustion, non-recyclable
Diesel	10700	12700	combustion, non-recyclable
Hydrogen gas	2.8	33300	combustion/fuel cell, non-recyclable

Batteries: Power density

- Amount of current that can be delivered or absorbed by the chemical reaction is limited.
- Beyond the limit either the voltage drops to unusable values, or the battery can be damaged.
- This is defined by the chemistry and the construction of a battery
⇒ *Power density* is another useful characteristic of a battery type

Some examples (rechargeable storage):

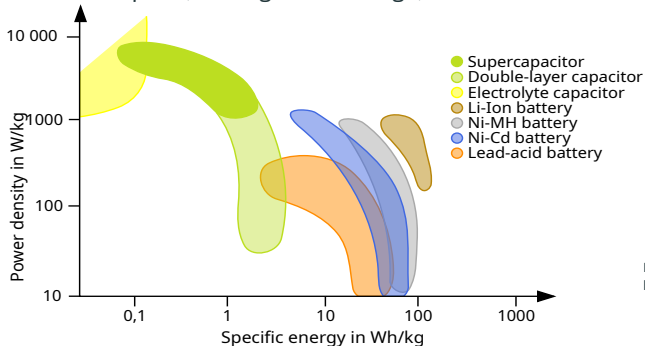


Image source: Adopted from MovGP0, CC BY-SA 2.0 via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:Energiespeicher2.svg>

Capacitor: another way to store electric charge

Related meaning as battery capacity, but not to be confused with the *capacitance of a capacitor* (condenser)

- In capacitors, charge is stored in an *electric field*, not in the state of chemicals
- The capacitance C is the property of the device

Unit: Farad, $1\text{ F} = \frac{1\text{ As}}{1\text{ V}}$; range used in electronics: pF ...mF

Voltage, charge, and energy are variable: $Q = C \cdot V$ and $E = \frac{1}{2} C \cdot V^2$

- There are use cases for capacitors instead of batteries.

Then, large capacitance is needed

- Large capacitance (range up to kF) $\Rightarrow$ *Supercapacitors*
- Benefit: High *power* density and higher cycle stability
- Examples: Flash of a camera (supplied by a battery),
Short-term energy supply for electric vehicles

(supplied from grid)

Image: Paris tram line T3, stores energy in supercapacitors when braking to reduce grid energy usage



Image source: Gérard Delafond, CC BY-SA 3.0 via Wikimedia Commons

<https://commons.wikimedia.org/wiki/File:Paris-tramway.jpg>

Alkaline batteries

- "Alkaline" refers to the electrolyte with $\text{pH} > 7$: Potassium Hydroxide (KOH)
- Same electrodes as the older Zinc-Carbon batteries (Zn anode "-", MnO_2 cathode "+"), but with a different electrolyte. Compensation for electron flow by OH^- ion flow.
- Nominal voltage: 1.5 V, capacity: 1 Ah to 10 Ah (AA cells: ≈ 2 Ah)
- Commonly used in consumer electronics in form factors AAA, AA, C, D, 9V (PP3) and button cells
- Also available with silver oxide (Ag_2O) cathode, offering better voltage stability. Usually as "button cell": less usage of expensive material
- Label encoding: **LRxx** for MnO_2 cathode, **SRxx** for Ag_2O cathode, "R": Round, "xx": the diameter in mm

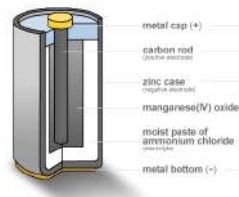
Image source: Adopted from Mcy jerry, CC BY 2.5 via Wikimedia Commons

[https://commons.wikimedia.org/wiki/File:Zincbattery_\(1\).png](https://commons.wikimedia.org/wiki/File:Zincbattery_(1).png)

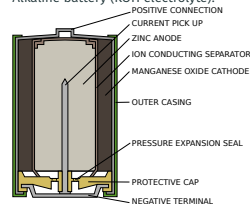
Image source: Lead holder, Tucvbf, Tympanus, public domain via Wikimedia Commons

<https://commons.wikimedia.org/wiki/File:Alkaline-battery-english.svg>

Zinc-Carbon battery (NH_4Cl electrolyte):



Alkaline battery (KOH electrolyte):



Lithium metal batteries

- Metallic lithium as anode, various cathode materials.
Compensation for electron flow by Li^+ ion flow.
- Form factor: Button cell, cylindrical cell
- Characteristics
 - *Manganese oxide* cathode (MnO_2):
Voltage 3 V, Capacity: ≈ 1.5 Ah (AA cell), ≈ 0.2 Ah (button cell)
 - *Iron disulfide* cathode (FeS_2): Voltage 1.5 V, Capacity: ≈ 3 Ah (AA cell)
- Label encoding (round cells):
 - **CRxx** for MnO_2 cathode, e.g. CR2032 (20mm diameter, 3.2mm height)
 - **FRxx** for FeS_2 cathode
- Commonly used in consumer electronics, e.g. watches, calculators, remote controls; FeS_2 often compatible to alkaline batteries.

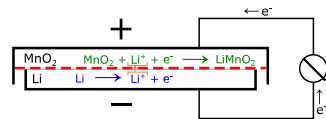


Image source: Nothingserious, Belumat, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Lithium_manganese_dioxide_battery.svg

Image source: Krzysztof Woźnica, public domain via Wikimedia Commons

<https://commons.wikimedia.org/wiki/File:Battery-lithium-cr2032.jpg>

Nickel-Cadmium batteries Ni-Cd

- Electrodes: Nickel(III) oxide-hydroxide cathode (NiOOH), cadmium anode, Electrolyte: potassium hydroxide.
Compensation for electron flow by OH^- ion flow.
- Nominal voltage: 1.2 V, capacity: under 1 Ah (AA cell form factor)
- Applications: Consumer electronics, power tools, model vehicles
- Partially compatible with alkaline batteries: in applications that tolerate alkaline batteries discharging down to 0.9 V
- Disadvantages: "memory effect", Toxic heavy metal cadmium - almost completely prohibited in EU



Image source: LordOider, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:NiCd_accumulator_with_recycling_pictograms.jpg

Nickel-Metal Hydride batteries

- Similar to Ni-Cd, but hydrogen-absorbing metal alloy instead of cadmium. Compensation for electron flow by OH^- ion flow.
- Nominal Voltage: 1.2 V, Capacity: up to 2.7 Ah in AA cell form factor
- Usage: consumer electronics (replacement for alkaline cells), non-plug-in hybrid electric vehicles
- Higher self-discharge rate than Ni-Cd, but low self-discharge (LSD) variants exist



Image: Eneloop LSD batteries

Image source: Raimond Spekking, CC BY-SA 4.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Turbo_Akku_Vac_Easy_Home_VC_618WP_-_battery_pack-1376.jpg

Image: NiMH battery pack for a vacuum cleaner (12 cells in series $\Rightarrow$ 14.4 V), 1300 mAh

Image source: Ashley Pomeroy, CC BY-SA 4.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Eneloop_6420.jpg



Lithium-ion batteries (1)

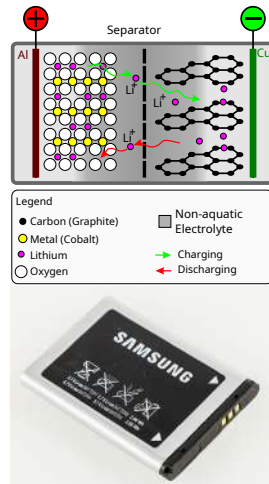
- Electrodes: metal oxide cathode, graphite anode, both capable to absorb lithium ions. Electrolyte: lithium salt in an organic solvent (no water due to lithium's high reactivity!)
- Charging and discharging: Lithium ions move in the electrolyte between the electrodes (similar to non-rechargeable Li-metal batteries)
- Nominal cell voltage: 3.6 V, cell capacity: 1 Ah to 10 Ah
- Usage: Consumer electronics, laptops, smartphones, power tools, electric vehicles, and for stationary energy storage (e.g. buffering solar power)
- Form factor: Cylindrical cell, pouch cell

Image source: Adopted from Cepheiden, CC BY-SA 2.0 via Wikimedia Commons

[https://commons.wikimedia.org/wiki/File:Li-Ion-Zelle_\(CoO2-Carbon,_Schema\).svg](https://commons.wikimedia.org/wiki/File:Li-Ion-Zelle_(CoO2-Carbon,_Schema).svg)

Image source: Raimond Spekking, CC BY-SA 4.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Samsung_E1200i_-_Lithium-ion_battery_AB463446BU-4036.jpg



Lithium-ion batteries (2)

Common liquid electrolyte cells, different cathode materials:

- LCO - Lithium Cobalt Oxide:
First commercialized, still in use. High specific energy
- NMC - Nickel Manganese Cobalt oxide:
High energy density, high power density, high cost.
High-performance cells for EVs.
- LFP - Lithium Iron Phosphate:
Lower energy density, lower cost, higher cycle stability.
Other label: LiFePO_4 - not to be confused with LiPo (see below)
Lower-performance cells for EVs, better safety than NMC.



Image source: Achimd, public domain via Wikimedia Commons

[https://commons.wikimedia.org/wiki/File:Lithium_polymer_battery_\(11.1_volts\).jpg](https://commons.wikimedia.org/wiki/File:Lithium_polymer_battery_(11.1_volts).jpg)

Lithium-ion batteries (3)

Polymer electrolyte cells (LiPo):

- Polymer electrolyte instead of liquid electrolyte
- Higher energy density, lower cost, lower cycle stability
- Laptops, smartphones, remote-controlled models



Image: LiPo battery pack (3 pouch cells) for a remote-controlled model car

Image source: Achimd, public domain via Wikimedia Commons

[https://commons.wikimedia.org/wiki/File:Lithium_polymer_battery_\(11.1_volts\).jpg](https://commons.wikimedia.org/wiki/File:Lithium_polymer_battery_(11.1_volts).jpg)

Lead acid batteries

- Electrodes: Lead & Lead oxide; Electrolyte: sulfuric acid
Compensation for electron flow by H^+ ion (proton) flow.
- Voltage: 2 V per cell, usually combined to 12 V or 24 V batteries
- Capacity: 20 Ah to 100 Ah
- Used as starter battery for combustion engines, when high currents are needed. Recharged from the combustion engine via a generator.
- In electric vehicles with Li-ion batteries as traction energy source, 12V batteries (lead-acid or Li-ion) are needed for board electronics (separate 12V grid)
- Also used in Uninterruptible Power Supplies (UPS) for computers/servers



Image source: Thomas Wydra, public domain via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:Starterbatterie.jpg>

Charging of batteries

- Charge rate in relation to the capacity: C-rate:
 - e.g. $1C$ = charge or discharge in 1 hour,
 - $< 0.1C$ = charge or discharge in >10 hours (trickle charge)
 - $> 1C$ = fast charge
- Different battery types need different charging methods
 - Ni-Cd, NiMH: Constant current (CC) charging, at a rate of $0.1C$ to $1C$
 - Li-ion: Constant current (CC) charging, followed by constant voltage (CV) charging.
In multi-cell batteries with a battery management system (BMS):
balancing phase between CC and CV.
Charge rates: $0.5C$ to $> 3C$ (fast charge)
 - Lead acid: CC, followed by CV, at a rate of $0.1C$ to $0.5C$
- Especially for fast charging, overcharge and damage to battery is possible.
⇒ monitoring of the battery temperature and voltage is needed.
- For the CV phase, a constant DC voltage is needed, independently of the current drawn by the battery ⇒ Another use case for voltage regulation

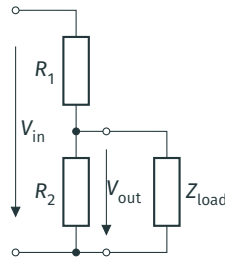
Power regulators

Not a regulator: voltage divider

Loaded voltage divider: $V_{\text{out}} = V_{\text{in}} \cdot \frac{R_2 \cdot Z_{\text{load}}}{R_1 R_2 + R_1 Z_{\text{load}} + R_2 Z_{\text{load}}}$

- R_1 and R_2 are fixed resistors (have a tolerance though)
- Output voltage scales with potentially unstable input voltage
- Output voltage drops if load current increases:
⇒ can still be used if the load current is low (e.g. gate of a FET)
- Energy loss due to constantly flowing current through R_1 and R_2 :

$$P_{\text{loss}} > \frac{(V_{\text{in}} - V_{\text{out}})^2}{R_1}$$



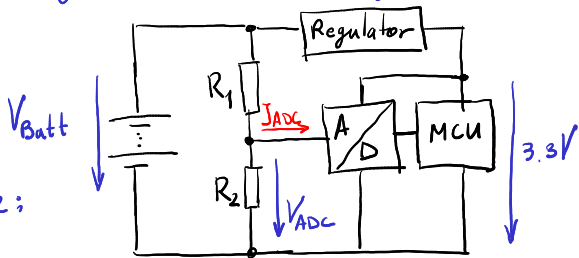
Example for meaningful use of voltage dividers

Task: Measure voltage of the power supply (battery, range e.g. $V_{\text{Batt}} = 4V \dots 6V$) but the available A/D converter can only measure in range $0V \dots 3.3V$ (with $3.3V$ being derived from battery by a "proper" regulator)

Solution: Since ADC usually has high input impedance ($I_{\text{ADC}} \approx 0A$), we can use a v. divider with $R_1 = R_2$ to scale down the voltage:

$$V_{\text{ADC}} = \frac{R_1}{R_1 + R_2} V_{\text{Batt}} = \frac{1}{2} V_{\text{Batt}}$$

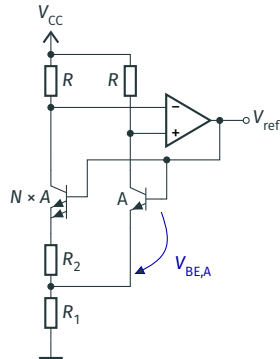
Power loss with e.g. $R_1 = R_2 = 1k\Omega$: $P_{\text{loss, max}} = \frac{(6V)^2}{2k\Omega} = 18mW$



Reference voltage generation

Purpose

- ~~Core~~ of all regulators:
Generate a stable DC voltage at a desired level
- Needed: A reference voltage which does not depend on the input voltage or load current, and ideally not on temperature
- Basic idea: *Band gap voltage reference* in semiconductor circuits, depends only on the materials and temperature.
- Example: Brokaw bandgap voltage reference [1]
generates a temperature-stable voltage of $V_{\text{ref}} \approx 1.2 \text{ V}$
- Another example: Zener diode (not shown)
If operated in reverse breakdown, the voltage is *sufficiently* stable, but higher than the bandgap voltage.



$$V_{\text{ref}} = V_{\text{BE,A}} + 2 \frac{kT}{q} \ln(N) \frac{R_1}{R_2}$$

Image source: Adopted from Paul Rako, public domain via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Brokaw_cell_theory.gif

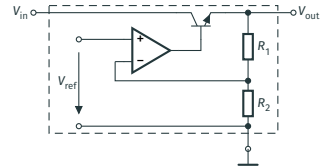


Low Drop-Out (LDO) - Functionality

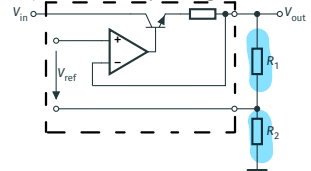
- Op-Amp with the reference voltage as one input, and the (scaled) output voltage as the other input [2]
- Amplifier adjusts voltage over the pass transistor, so that the relation between the output voltage and the reference voltage is stable.
- Resistors R_1 and R_2 scale the output voltage
 - Integrated into the regulator IC
⇒ Fixed output (e.g. 78xx series)
 - Placed outside the regulator IC
⇒ adjustable output voltage (e.g. LM317)

e.g. 7805 → $V_{out} = 5V$

Fixed output regulator:

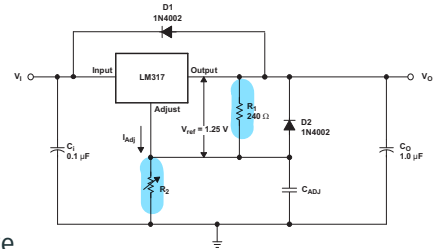


Adjustable output regulator:



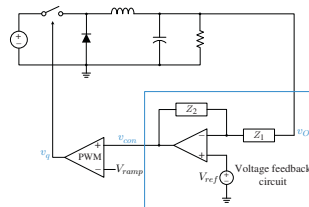
LDO - Properties and usage

- Low complexity - three pins, low cost
- Potentially low efficiency:
Power loss dissipated as heat. The loss is proportional to the voltage drop ($V_{\text{drop}} = V_{\text{in}} - V_{\text{out}}$) and the output current.
- Can be efficient if the goal is to provide low voltage and low current, while the input voltage is only slightly higher than the output voltage
- Typically used surrounded by capacitors to reduce "transient" effects from quick changes on supply or load side, and a diode from output to input to prevent reverse current flow, see the circuit in the picture (source: [3]).



Switching regulators - Functionality

- Similar feedback loop as in LDO. Instead of a pass transistor, a switch transistor is used [4]
- Switch is turned on and off at a high frequency (range: 100 kHz to 10 MHz).
Regulation is performed by adjusting the duty cycle
⇒ Pulse Width Modulation (PWM)
- No power loss when the switch is off, almost all power transferred to output when the switch is on
⇒ Efficient also for large voltage drops and high output currents
- Filtering on the output is needed to reduce the ripple
- Due to capability of bringing one DC voltage level to another, also called "DC-DC converter"



Buck converter with feedback loop.
Source: [4]

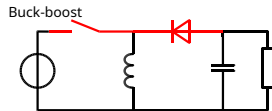
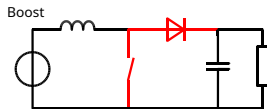
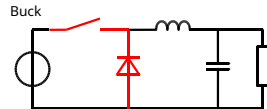
Switching regulators - Boost and Buck-Boost converters

Previous slide: Buck converter (Output voltage is lower than input voltage).

Other possible circuits:

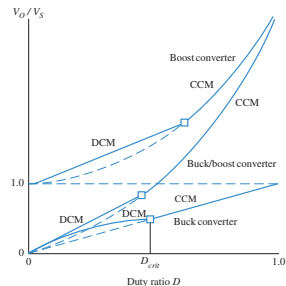
- Boost converter: Output voltage is higher than input voltage
- Buck-Boost converter: Output voltage can be higher or lower than input voltage, depending on the duty cycle.

Image source: Adopted from Cyril BUTTAY, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Commutation_cell_in_converters.svg



Switching regulators

- Higher efficiency even for large voltage drops and high output currents
- Higher frequency $\Rightarrow$ Smaller form factor, but also higher switching losses
- Potentially high noise on the output voltage and on the input voltage $\Rightarrow$ Can influence measurements, noise is increased by analog amplifiers
- Ongoing research:
Reduce the noise, increase the efficiency, reduce size



Relationship between PWM duty cycle and output voltage.
CCM = continuous conduction mode,
DCM = discontinuous cond. mode.
Source: [4].

References

Bibliography i

- [1] A. Brokaw, "A simple three-terminal IC bandgap reference," *IEEE Journal of Solid-State Circuits*, vol. 9, no. 6, pp. 388–393, 1974.
- [2] S. J. G. Gift and B. Maundy, *Electronic Circuit Design and Application*. Springer, 2022. [Online]. Available: <https://doi.org/10.1007/978-3-030-79375-3>
- [3] *LM317 3-Terminal Adjustable Regulator*, Texas Instruments, 2020, rev. Y. [Online]. Available: <https://www.ti.com/lit/ds/symlink/lm317.pdf>
- [4] B. Choi, *Pulsewidth Modulated DC-to-DC Power Conversion: Circuits, Dynamics, Control, and DC Power Distribution Systems*, 2022. [Online]. Available: <https://ieeexplore.ieee.org/servlet/opac?bknumber=9562692>

Hardware Platforms and Programming of Embedded Systems

Lecture 8 - Software for microcontrollers

Dr.-Ing. Alexander Knaub

2024-12-12

Institute of telecommunications (INÜ)

Contents

General Concepts

- Toolchain

- Memory layout

MCU software development

- Source code structure

- Concepts of object oriented programming in C

General Concepts

Main difference between MCU and PC software

C programs for an MCU follow the same syntax as for a PC CPU, but there are some "peculiarities" that make the software development slightly different.

PC CPU	MCU
• Clear definition of program end, at least for a clean system shutdown	No defined end, main loop usually implemented with a <code>while(true)</code> section
• Code is ideally portable, Access to hardware through drivers in OS	Code is usually platform-specific and has direct access to hardware
• Code can be compiled on the target	Code is cross-compiled on a <i>host</i> PC for the architecture of the <i>target</i> MCU
• A program shares the CPU with other programs and the OS kernel	Only one application runs on an MCU (the one app can have different tasks though)
• OS and memory management unit (MMU) HW provide virtual memory for each process	No virtual memory: Instructions, data, and peripherals all live in one memory space
• Instruction code is loaded from disk into RAM ("virtually" unlimited space)	Code remains in flash ROM, its size is limited by flash memory

Toolchain

(Cross-)Compiler toolchain is a set of programs needed to translate the source code into a binary file that can be executed on the MCU.

Minimum toolchain components for MCU development, in order of execution:

- **Preprocessor:** Expands macros, includes header files, etc.
- **Compiler:** Converts the source code into assembly code
- **Assembler:** Converts the assembly code into machine code
- **Linker:** Combines the machine code with the startup code and libraries, and generates the binary file

Sometimes people use "toolchain" for describing *all* development tools, e.g. also the debugger, the HW-access tools (programmer), or the IDE. Often the terms *toolchain* and *compiler* are used interchangeably.

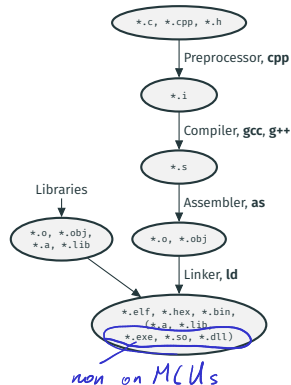


Image: Toolchain for C/C++ development with GNU Compiler Collection (GCC). Program names are set in **bold**.

ELF file format

- **ELF** = Executable and Linkable Format
- Very common file format for executable files (and more)
- Originates from Unix-like operating systems (Linux, BSD, Solaris, etc.)
- On MCU: ELF is generated by the toolchain (more precisely, the *linker*) and contains all information needed to load and run the program
 - Memory layout of the program
 - Program and data sections
 - Symbols (function names, variable names, etc.)
 - Libraries that are needed by the program
 - Other information e.g. for debug purposes

ELF format - example

Analysis of section headers within an ELF file using the GCC tool **objdump**:

```
$ arm-none-eabi-objdump.exe -h .pio/build/stm32/firmware.elf
```

```
.pio/build/stm32/firmware.elf:      file format elf32-littlearm

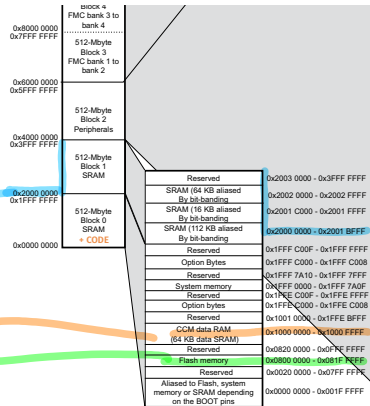
Sections:
Idx Name          Size      VMA           LMA           File off  Algn
  0 .isr_vector    00000194  08000000  08000000  00001000  2**0
    CONTENTS, ALLOC, LOAD, READONLY, DATA
  1 .text          00003804  08000194  08000194  00001194  2**2
    CONTENTS, ALLOC, LOAD, READONLY, CODE
  2 .rodata        0000055c  08003998  08003998  00004998  2**2
    CONTENTS, ALLOC, LOAD, READONLY, DATA
...
  8 .data          00000074  20000000  08003f14  00005000  2**2
    CONTENTS, ALLOC, LOAD, DATA
  9 .bss           00000404  20000074  08003f88  00005074  2**2
    ALLOC
10 .noinit         00000000  20000478  0800438c  00000000  2**0
    ALLOC
11 .user_heap_stack 00000600  20000478  20000478  00005478  2**0
    ALLOC
```

Comments:

- VMA - "virtual memory address", address of the section at runtime
- LMA - "load memory address", address of the section in the flash ROM
- **.isr_vector** - interrupt vector table, contains addresses of interrupt service routines (ISRs)
- Other sections: see next slides

Memory layout

- Defined by the hardware architecture
- Regions for accessing the peripherals (memory-mapped registers)
- Regions/sections for flash ROM
 - Program code **".text"**
 - Constants **".rodata"**
- Regions/sections for RAM
 - "ramfunc" instructions - e.g. **".textwr"** or **".ccmram"**
 - static variables - **".data"** and **".bss"**
 - RAM not otherwise reserved = **heap and stack**



The flash and RAM regions can be either in the same address space (example: STM32/ARM), or in different address spaces (example: AVR).

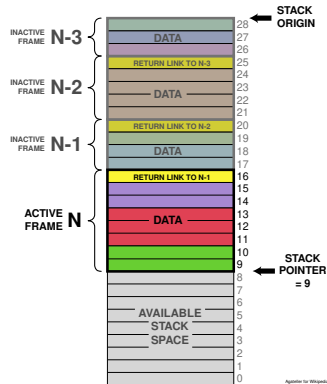
Image: Memory map example: STM32F429 ARM Cortex-M4 MCU. Excerpt from datasheet [1]

RAM initialization

- Static/global variables with non-zero initial values: `".data"` section
- Static variables without an initial value or zero-initialized: `".bss"` section
- It is possible for RAM to contain part of the program code, e.g. for faster execution or for surviving/performing flash erase section.
Special linker sections must be defined, e.g. `".textrw"`
- Initialization of RAM contents is done by the startup code inserted by the compiler in the `".text"` section (flash) before the `main()` function is called:
 - If necessary, copy the `".textrw"` code section from flash to RAM
 - Copy `".data"` section from flash to RAM
 - Clear `".bss"` section to all-zeros:
faster than copying the values, and only the length of the section must be stored in flash
 - Region beyond `".bss"` is usually available for the *heap* and *stack* defined at runtime.
(see later)

Stack

- Memory region with variable (not predefined) size
- Stack pointer register *SP* is initialized to a high address (top of the stack) at startup.
This is usually at the end of the RAM range (depends on the compiler and MCU architecture).
- Used for constant-size local variables and for keeping track of function calls
- Grows from high to low addresses (*SP* is decremented when objects are placed/*pushed* onto the stack)
- If a value on the stack is no longer needed, it is *popped* from the stack (*SP* is incremented)



Agente for Wikipedia
Public Domain 2008

Stack - usage

- Exemplary life cycle of a function $f_N()$, ensured by compiler:
 - Push the *return address* onto the stack
= next program counter after PC of the caller function $f_{N-1}()$
 - Place local variables of $f_N()$ in registers.
If no registers are available, push the new variables onto the stack as well.
 - If $f_N()$ needs to overwrite the registers that were used by $f_{N-1}()$, ensure that the values are saved on the stack after entering $f_N()$, and restored (popped) before $f_N()$ returns.
 - Return: Pop the return address from the stack into the program counter (PC)
- The stack behavior is specific to compiler and optimization settings, e.g. the caller $f_{N-1}()$ can be responsible for saving and restoring its registers.
- Function $f_N()$ can access the local variables of $f_{N-1}()$ on the stack, but not the other way around. ($f_{N-1}()$ needs to pass addresses/pointers to $f_N()$ then)
- When the function $f_N()$ returns, all local variables of $f_N()$ are discarded.

Example: <https://godbolt.org/z/j97n6Gbq4>

Heap

- Memory region with variable (not predefined) size
- Used for dynamically *allocated* memory outside the stack, e.g. `malloc()`, `new`, `new[]`
- Grows from low to high addresses, starts usually after the end of the `.bss` section.
- If a value on the heap is no longer needed, it can be *deallocated* from the heap, e.g. `free()`, `delete`, `delete[]`
- Implementation is highly platform- and toolchain-specific (provided via the standard library `stdlib.h` / `libc.a`):
 - Whether heap is below or above the stack
 - Whether heap is really dynamic or just a fixed-size pool
 - How the available size is determined
 - ...

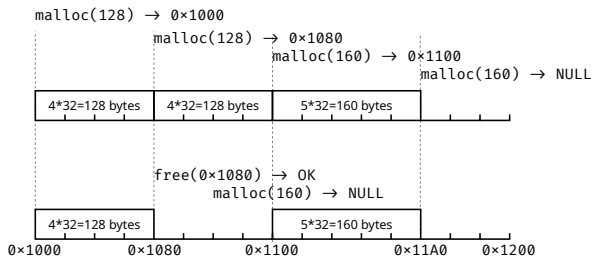
Heap - fragmentation

- Deallocation can happen at any time, not necessarily in the order of allocation, and not predictable by the compiler => Fragmentation of the heap is possible
- Due to fragmentation, in case of low total RAM size, the heap can be reported as full if there is no gap large enough for the requested allocation
=> makes programs less deterministic

Example with 512 bytes of available

heap RAM:

Once 416 bytes were used, there is no gap large enough for the next 160-byte allocation, even after total free space of $7 \times 32 = 224$ bytes after freeing the block at address **0x1080**.



⇒ avoid dynamic memory (**de**-)allocation of **variable** size in MCU programs.

(Defragmentation is the typical task of a *garbage collector*, which is not available in most MCU environments.)

MCU software development

MCU software development

- MCU software is usually written in C
C++ is also offered by some toolchains and frameworks, but not all standard library functions can be used
- The code is compiled on a PC for the target MCU architecture
- Usual PC-based C compilers often are (or can be made) capable to compile for MCU architectures:
 - GNU Compiler Collection (GCC), Clang/LLVM,
- There are also platform-specific compilers, e.g.:
 - Keil, IAR for ARM
 - Microchip XC8, XC16, XC32 for PIC

MCU code structure

- Two blocks: *initialization* and *main loop* inside the `main()` function
- Arduino framework: Developer needs to implement functions `setup()` and `loop()`
- Main loop => repeated execution of tasks
- Invisible to developer: System initialization before entering/calling `main()` (e.g. RAM initialization described earlier)
- Work with interrupts for event-based tasks (see later)

MCU code structure - Example

Example application: Read ADC, control LED brightness

```
1 #include <Arduino.h>
   const int ledPin = 5; // PWM-capable GPIO pin for LED
   const int potentiometerPin = 2; // analog input A2
   const int pwmResolution = 8; // bits per value
5   const int adcResolution = 10; // bits per value
   const int pwmUpdateInterval = 100; // in ms

   // non-const global variables
   int potentiometerValue = 0;

10 void setup() {
   // sets the PWM pin as output
   pinMode(ledPin, OUTPUT);
   // configure analog pin A2 for reading the potentiometer
15   pinMode(potentiometerPin, INPUT);
}

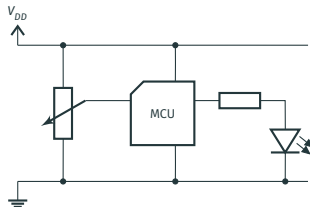
   void loop() {
   // read the potentiometer value - 10 bit ADC
20   potentiometerValue = analogRead(potentiometerPin);
   // adjust the LED brightness based on the potentiometer value
   const int dutyCycle =
       potentiometerValue >> (adcResolution - pwmResolution);
   analogWrite(ledPin, dutyCycle);

25   delay(pwmUpdateInterval); // wait before repeating
}
}
```

...with GPIO functions declared in `Arduino.h`, and the functions **setup()** and **loop()** called from within the Arduino framework:

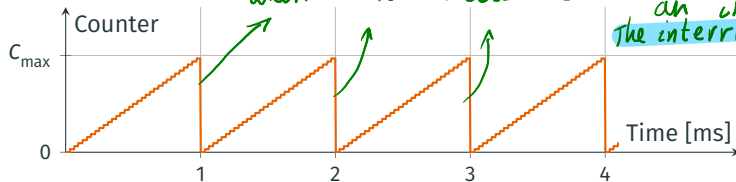
```
1 //...
   int main(void)
   {
       init();
       initVariant();

5       setup();
       for (;;) {
           loop();
           if (serialEventRun) serialEventRun();
10      }
       return 0;
   }
```



Keeping track of time

- Clock frequency can be made known to software
⇒ Elapsing time can be measured with a counter that increments with each clock cycle
- For this, dedicated hardware is usually placed on the MCU chip, usually called *timer* or *counter*, and often multiple instances are available.
- One of the timers is usually configured for the **system tick** - a periodic interrupt, which triggers incrementing a global variable, **counting number of "ticks"** since startup.
- The tick frequency is usually set to a value that allows counting milliseconds.
- Example: Arduino function `uint32_t millis(void)`



The interrupt handler increments the tick counter

Example: $f_{\text{clk}} = 16 \text{ MHz}$,
 $C_{\text{max}} = f_{\text{clk}} \cdot 1 \text{ ms} = 16000$

Tick-based task scheduling

- The tick counter can be used to trigger periodic tasks in the main loop
- Example: Read temperature/humidity from sensor, display on LCD, refresh every 5s.
- Challenge:
Reading sensors and displaying on LCD takes time.
How to maintain the 5s interval?
- Solution:
Use the tick counter to measure the durations, and to calculate the sleep time.
- Improvements: Stay reactive to e.g. user input
→ next slide

```
1 #include <Arduino.h>
  #include <LiquidCrystal_I2C.h>
  //...

5 const unsigned long refreshPeriod = 5000; // in ms

  // Use the tick counter to calculate the sleep time
void loop() {
  unsigned long start = millis();

10  int humidity = readHumidity();
    int temperature = readTemperature();
    lcd.setCursor(0,0);
    lcd.print("Hum:␣␣" + String(humidity) + "%");
15  lcd.setCursor(0,1);
    lcd.print("Temp:␣" + String(temperature) + "C");

    unsigned long end = millis();
    delay(refreshPeriod - (end - start));

20 }
```

Ticks and user input

- UI improvement to last example:
Pressing a button selects the next sensor to display.
⇒ Two concurrent tasks, different frequency:
Sensor refresh and button polling
- Challenge: *read the button state frequently*
Fast polling of button state prevents the MCU from sleeping
- Solution:
Measure time since last sensor refresh,
refresh if enough time has passed.
- Further improvements:
 - Avoid continuous polling of button state
 - What if button is pressed during sensor refresh?
 - → next slide

```
1 //...
  const unsigned long refreshPeriod = 5000; // in ms
  bool nextSensorIsHumidity = true;
  unsigned long lastRefresh = 0;

5 // Poll a button to switch between sensors
  void loop() {
    unsigned long now = millis();
    // read state of a button:
10    int buttonState = digitalRead(buttonPin);
    if (buttonState == HIGH) {
      nextSensorIsHumidity = !nextSensorIsHumidity;
      // force refresh:
      lastRefresh = now - refreshPeriod;
15    } // else: keep the current sensor

    if ( (now - lastRefresh) > refreshPeriod) {
      lastRefresh += refreshPeriod;
      const int value = nextSensorIsHumidity
        ? readHumidity() : readTemperature();
      lcd.setCursor(0,0);
      lcd.print(nextSensorIsHumidity
        ? "Hum.:␣" : "Temp:␣");
      lcd.setCursor(7,0);
      lcd.print(String(value));
20    }

25    // no delay for faster reaction to button press
  }
```

Interrupts for user input

- Improvements to last example:
 - Button press is detected by an interrupt, independently of the main loop
 - Main loop is cleaner
- Further possible improvements:
 - Ensure the button press is not detected multiple times due to bouncing
 - ...

```
1 //...
  const unsigned long refreshPeriod = 5000; // in ms
  volatile bool nextSensorIsHumidity = true;
  unsigned long lastRefresh = 0;

5
  void handleButtonPress() {
    nextSensorIsHumidity = !nextSensorIsHumidity;
    lastRefresh = millis() - refreshPeriod;
  }

10
  void setup() {
    lcd.init();
    lcd.backlight();

15    attachInterrupt(
      digitalPinToInterrupt(buttonPin),
      handleButtonPress, RISING);
  }

20 void loop() {
  unsigned long now = millis();
  if ( (now - lastRefresh) > refreshPeriod) {
    lastRefresh += refreshPeriod;
    const int value = nextSensorIsHumidity
      ? readHumidity() : readTemperature();

25    printValueOnLcd(lcd, value, nextSensorIsHumidity);
  }
}
```


Interrupts

- When some event happens, the MCU can interrupt/pause the main program and execute an interrupt service routine (ISR)
- The ISR location is either
 - fixed - must be placed at a specific address (not common on today's MCUs)
 - configurable - can be placed anywhere in the flash memory. The address must be placed in the interrupt vector table - a list of addresses for all ISRs
- During the ISR, the main program is paused, and the MCU is not able to react to other interrupts. *↳ usually*
Implication: Functions relying on interrupts don't work during ISR execution (e.g. Arduino's `delay()` - needs timer interrupts for counting milliseconds)
- ISRs should return as fast as possible. If lengthy operations are needed, this should be signalled by the ISR, and executed in the main loop.

Object oriented programming in C

Object oriented programming (OOP) concepts

- **Encapsulation:** Grouping of data and functions that operate on the data
- **Inheritance:** A class can inherit data and functions from another class
- **Polymorphism:** A function can have different implementations depending on the type of the object it is called on
- Usually achieved by using *classes* and *objects* - both not really available in C.
But: C **struct**'s can help.

Encapsulation in C (1)

- A **struct** defines the data, like member variables (attributes) of a C++ **class**
- Functions operating on the data take a pointer to the **struct** as an argument, like member functions (methods) of a C++ **class**
- Example listing: Data and functions defined in the same header file
→ **public** member variables
- For **private** member variables, see next slide

```
1 #pragma once
   #include <stdint.h>

   // --a--
5 // f      b
   // --g--
   // e      c
   // --d-- h

10 typedef struct {
    int pins[8]; // handles to GPIO pins
    uint8_t cleared; // if true, all segments are off
    // the number currently being displayed
    uint8_t currentDigit;
15 } OneDigit7Segment;

   // "constructor"
   OneDigit7Segment seven_segment_init(const int pins[8]);

20 // "methods"
   void seven_segment_display(
       OneDigit7Segment* digit,
       const uint8_t number);

25 void seven_segment_clear(OneDigit7Segment* digit);
   uint8_t is_cleared(OneDigit7Segment* digit);
   uint8_t get_current_digit(OneDigit7Segment* digit);
```

Encapsulation in C (2)

- For "**private**" member variables, forward-declare the **struct** in the header file, and *define* it in the source file.
- The **struct** definition is not visible to the user of the library.
- Without a definition (size is unknown), users cannot create variables of the **struct** type, only pointers.
→ initialization on the stack is not possible, only on the heap with **malloc()**.

my_7seg.h

```
1 #pragma once
  #include <stdint.h>

  // Forward declaration - members not visible
5 struct OneDigit7Segment;
  typedef struct OneDigit7Segment* OneDigit7SegmentPtr;

  OneDigit7SegmentPtr seven_segment_init(const int pins[8]);
10 void seven_segment_display(OneDigit7SegmentPtr digit, uint8_t number);
  void seven_segment_clear(OneDigit7SegmentPtr digit);
  uint8_t is_cleared(OneDigit7SegmentPtr digit);
  uint8_t get_current_digit(OneDigit7SegmentPtr digit);
```

my_7seg.c

```
1 #include "my_7seg.h"
  #include <stdlib.h> // for 'malloc'

  struct OneDigit7Segment {
5     int pins[8];
      uint8_t cleared; uint8_t currentDigit;
  };

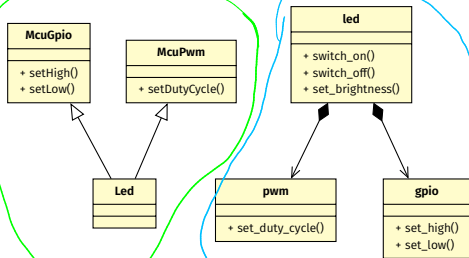
  OneDigit7SegmentPtr seven_segment_init(const int pins[8]){
      OneDigit7SegmentPtr digit = (OneDigit7SegmentPtr) malloc(sizeof(*digit));
10     for (int i = 0; i < 8; i++) digit->pins[i] = pins[i];
      digit->cleared = 1; digit->currentDigit = 0;
      return digit;
  }
```

Inheritance in C

Inheritance: A derived class can reuse the data and functions of a base class.

C has no concept of *Inheritance*

- Behavior of inheritance can be achieved through composition
→ instead of a "is-a" relationship, use a "has-a" relationship
- A **struct** can contain another **struct** as a member variable, possibly as a pointer, and delegate function calls to it.
- Example: **struct led** contains a **struct gpio** and a **struct pwm**
→ **led** "inherits" the **gpio** and **pwm** functions



Note: Inheritance is sometimes called bad practice, or at least often misused also in languages that support it. It is recommended to always consider whether composition can be preferred over inheritance. In C code, composition is anyway the only option.

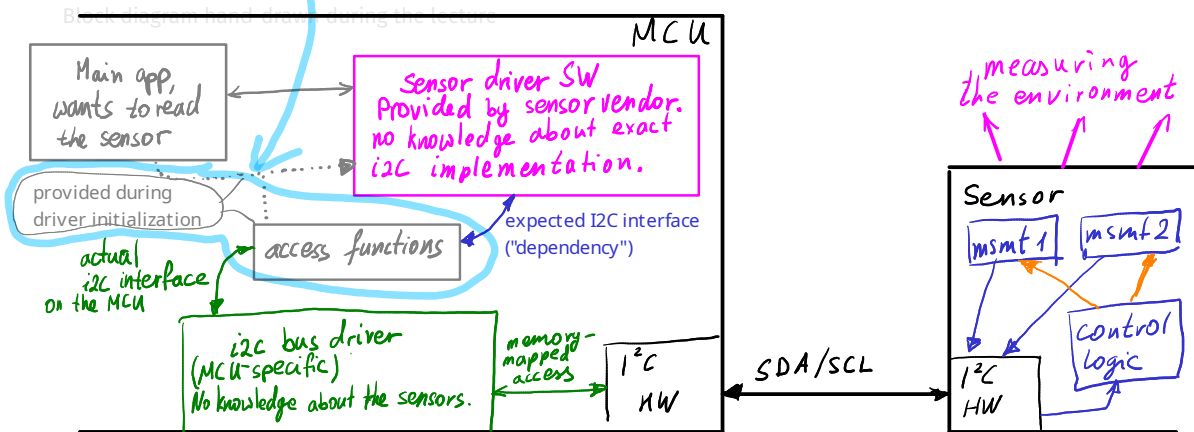
Polymorphism in C

Polymorphism: A function can have different implementations depending on the exact object on which it is called.

- C has no concept of *polymorphism*
- But: Behavior of polymorphism can be achieved through *function pointers*
- A **struct** can contain function pointers as member variables.
- Example: **struct led** contains a function pointer **set_brightness()**
→ **led** can be used with different implementations of **set_brightness()**

```
1 struct led {  
    void (*set_brightness)(uint8_t);  
    void * private_data;  
    // ...  
5 };  
  
void set_brightness_pwm(struct led* led_ptr, uint8_t val);  
void set_brightness_gpio(struct led* led_ptr, uint8_t val);  
  
10 int main(void)  
{  
    struct led led_pwm = {  
        .set_brightness = set_brightness_pwm  
    };  
    struct led led_gpio = {  
        .set_brightness = set_brightness_gpio  
    };  
    led_pwm.set_brightness(&led_pwm, 50);  
    // ...  
20 }
```

Note: This realization of polymorphism is often used in libraries that offer platform-independent interfaces and platform-specific implementations. Widely used in the Linux kernel (see next lecture).



Real-world example: <https://github.com/boschsensortec/BMP3-Sensor-API/tree/master>

Dependency injection in C

Concept of *Dependency injection*:

- Function "A" expects that another function "B" exists, the exact behavior is not important.
- Function "A" takes a pointer to function "B" as an argument, and calls B through the pointer.
- The caller of function "A" provides the actual implementation of function "B".

```
1 // header file "A" - declaration of "B" function pointers
typedef void (*i2c_write_t)(
    uint8_t addr, uint8_t reg, uint8_t *data, uint8_t len);
typedef void (*i2c_read_t)(
5     uint8_t addr, uint8_t reg, uint8_t *data, uint8_t len);

struct i2c_access {
    i2c_write_t write;
    i2c_read_t read;
10 };

struct sensor {
    struct i2c_access *i2c; // depends on functions "B"
    uint8_t addr;
15     // ...
};

void sensor_read(struct sensor *sensor, uint8_t reg, uint8_t *data, uint8_t len);
```

```
1 // source file "A"
void sensor_read(struct sensor *sensor,
    uint8_t reg, uint8_t *data, uint8_t len){
5     sensor->i2c->read(
        sensor->i2c, sensor->addr, reg, data, len);
}

// Other source file, provides functions "B"
10 struct i2c_access i2c = {
    .write = i2c_write,
    .read = i2c_read
};
struct sensor sensor = {
15     .i2c = &i2c,
    .addr = 0x12
};
```

not shown here

injection

Thank you for the attention!

Next lecture: RTOS and Linux
Multithreading on MCUs, Introduction to Linux Kernel

References

Bibliography i

- [1] STMicroelectronics, *STM32F427xx STM32F429xx Datasheet*, Jan 2018. [Online]. Available: <https://www.st.com/resource/en/datasheet/stm32f429zi.pdf>

Hardware Platforms and Programming of Embedded Systems

Lecture 9 - Embedded operating systems

RTOS and Linux

Dr.-Ing. Alexander Knaub

2025-01-09

Institute of telecommunications (INÜ)

Contents

- OS on microcontrollers - RTOS

 - Non-preemptive multitasking: "Bare-metal"

 - Preemptive multitasking - RTOS

- Embedded Linux

 - Introduction

 - Linux kernel

 - Configuring and building the Linux kernel

 - Board support package

 - Linux distributions

OS on microcontrollers - RTOS

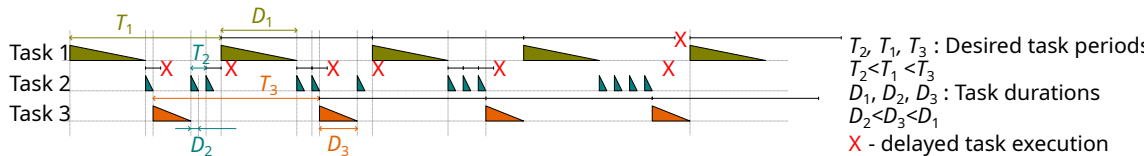
Non-preemptive multitasking - 1

Recapitulation: Tick-based operation, allowing simple multitasking

- Periodic check whether it is time to run one of the defined tasks
- If so, run it once; otherwise do nothing
- Continue with the next task, executing all in a round-robin fashion such that all tasks share the CPU time
- Such implementation, where the tasks are executed in one main loop, is called *super-loop*
- Potentially, an interrupt can occur at any time.
Once the handler function returns, continue with the interrupted task.

Non-preemptive multitasking - 2

- If the execution of tasks is short enough, the desired periodicity of the tasks can be maintained
- If there are many tasks, and/or the execution of tasks is lengthy, the total period grows, reducing the effective frequency of all the tasks, and thus potentially the system's responsiveness.

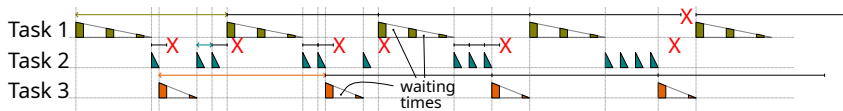


Non-preemptive multitasking - 3

Often, the execution of tasks is lengthy because of waiting for some event to occur:

- Completing a communication transaction by the hardware on the MCU or on a peripheral (e.g. reading a value from a sensor on I²C)
- Erasing or writing a flash memory page
- Just (busy) waiting for a certain amount of time (e.g. constant-length GPIO pulse - LED or buzzer)

In this time the CPU is not doing anything useful, but it is not available for other tasks either.



Blocking tasks - example

Example "alarm" task:

Periodically trigger execution of

runBuzzerTask() in the listing on the right for the pattern "beep-beep-beep", "pause", "beep-beep-beep", "pause",...

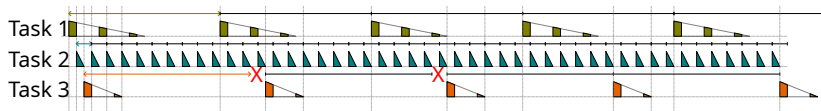
Full code for this example:

https://github.tik.uni-stuttgart.de/INUE/es4ee_example_multitasking/blob/main/src/BuzzerTask.c

```
1 static void buzzerOn(const BuzzerTaskData *data) {  
    ledcWrite(data->pwmChannel, data->volume);  
}  
  
5 static void buzzerOff(const BuzzerTaskData *data) {  
    ledcWrite(data->pwmChannel, 0);  
}  
  
static void buzzNTimes(const BuzzerTaskData *data) {  
10     // This function blocks the main loop for the duration  
    // 2 * pulseDuration * numPulses, although the CPU is  
    // only busy during buzzerOn and buzzerOff,  
    // which only configure the PWM output and return immediately.  
    for (int i = 0; i < data->numPulses; i++) {  
15         buzzerOn(data);  
        delay(data->pulseDuration); // busy wait  
        buzzerOff(data);  
        delay(data->pulseDuration); // busy wait  
    }  
20 }  
  
static void runBuzzerTask(void *dataGeneric) {  
    BuzzerTaskData *data = (BuzzerTaskData *)dataGeneric;  
    buzzNTimes(data);  
25 }
```

Non-preemptive multitasking - 4

- Possible solution:
A task exits early and *yields* the CPU to other tasks before starting to wait.
- To continue from where it left off, the task needs to store its current state (e.g. "Requested transaction with sensor X, no response received")
- This is called *cooperative* or *non-preemptive* multitasking.



Cooperative tasks - example

Same "alarm" task as in the previous example:

- The execution of `runBuzzerTask()` is triggered in *irregular* intervals:
`data->taskData.taskPeriod` is changed, depending whether the task is in the "beep" or "pause" phase.
- The task yields the CPU to other tasks directly after switching the signal on or off
- The task stores additional state data:
`data->isBuzzing`,
`data->numPulsesRemaining`,
`data->taskData.taskPeriod`

Full code for this example:

https://github.tik.uni-stuttgart.de/INUE/es4ee_example_multitasking/blob/cooperative_multitasking/src/BuzzerTask.c

```
1 static void runBuzzerTask(void *dataGeneric) {  
    BuzzerTaskData *data = (BuzzerTaskData *)dataGeneric;  
    // whatever was the last pulseDuration, it has elapsed  
    if (!data->isBuzzing) {  
        buzzerOn(data);  
        data->isBuzzing = true;  
        data->numPulsesRemaining--;  
        data->taskData.taskPeriod = data->pulseDuration;  
    } else {  
        buzzerOff(data);  
        data->isBuzzing = false;  
        if (data->numPulsesRemaining == 0) {  
            // restart the sequence at next period start  
            data->taskData.taskPeriod =  
                BUZZER_TASK_PERIOD -  
                (data->pulseDuration * (data->numPulses * 2 - 1));  
            data->numPulsesRemaining = data->numPulses;  
        } else {  
            // continue "beeping" until numPulsesRemaining == 0  
            data->taskData.taskPeriod = data->pulseDuration;  
        }  
    }  
    // Return and yield the CPU to other tasks:  
    // This function will be called again after  
    // `data->taskData.taskPeriod`,  
    // in the meantime the CPU is free to do other things.  
}
```

Non-preemptive multitasking - 5

Drawbacks of directly implemented cooperative multitasking in a super-loop:

- The tasks have to be written in a way that they yield the CPU to other tasks.
- Maintaining the state of the tasks is the responsibility of the programmer. This makes the tasks more complex and can be error-prone.
- The task yielding the CPU will continue only after all other tasks have been executed, thus potentially waiting longer than necessary.

⇒ This can be a viable solution for simpler systems with few lengthy tasks, which are not time-critical and of low complexity.

OS on microcontrollers - RTOS

Non-preemptive multitasking: "Bare-metal"

Preemptive multitasking - RTOS

Embedded Linux

Introduction

Linux kernel

Configuring and building the Linux kernel

Board support package

Linux distributions

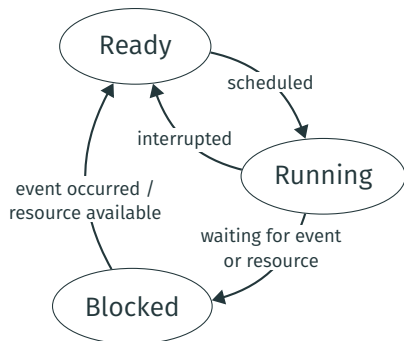
Preemptive multitasking

Main purpose of a *real-time operating system (RTOS)* on a microcontroller is the implementation of *preemptive* multitasking (like processes and threads on a PC).

- A *scheduler* task is in place, and is responsible for managing the tasks.
- Tasks need to be individually registered with the scheduler
- Scheduler can interrupt and resume tasks - similar to hardware interrupts.
- Unlike with hardware interrupts, usually another task is executed after the interrupt.
⇒ Current state of the interrupted task must be stored by the RTOS, and restored when the task is resumed (*context switch*).
- An RTOS imposes an overhead on the system, but simplifies the implementation of concurrent tasks.

RTOS task states

- *Running*: The task is currently being executed. Not more than one task for each CPU core can be in this state at a time.
- *Ready*: The task is ready to be executed, but is not currently running. The scheduler chooses one of the ready tasks to be executed next.
- *Blocked*: The task is waiting for an event to occur, and is not currently running. Once the event occurs, the task is moved to the ready state.



- Whenever waiting is needed in a task, special functions, provided by the RTOS, should be used in order for the task to be moved to the *blocked* state and the CPU to be given to one of the *ready* tasks.
- Since a task can be interrupted during its execution, developer needs to ensure that no data *race conditions* occur
 - Shared resources (e.g. peripherals) should be protected by *synchronization* mechanisms (e.g. mutexes, semaphores), also provided by the RTOS.
 - A task requesting a resource currently locked by another task is moved to the *blocked* state and later to the *ready* state - when the resource is released.
- An RTOS can provide additional services, e.g.:
 - Inter-task communication (e.g. synchronized message queues)
 - Higher-level functions (e.g. file systems, network stacks)
 - RTOS-aware drivers for peripherals

Scheduling strategies - 1

- Tasks compete for the CPU time if more than one task is in the *ready* state.
- Tasks can have a *priority*, so the scheduler can execute the highest-priority tasks first.
- Task preemption can take place when a task with higher priority becomes *ready* while a lower-priority task is *running*.
- If all *ready* tasks have the same priority, a scheduling strategy is used to decide which task to execute next.

Scheduling strategies - 2

The scheduler can be configured to use one of following strategies for selecting one of the *ready* tasks if multiple tasks have the same priority:

- *First-in, first-out (FIFO)*: Tasks are executed in order of becoming *ready* - until they complete or voluntarily leave the *running* state.
- *Earliest deadline first*: Tasks are executed in order of increasing deadline - until they complete or voluntarily leave the *running* state.
- *Round-robin*: Tasks are forcibly moved to the *ready* state a certain time period after becoming *running* (time slice), unless they voluntarily leave the *running* state earlier.

Note: "Voluntarily leaving the running state" brings a cooperative multitasking aspect to the scheduling. However, this is as easy as e.g. calling a function like **vTaskDelay()**, and RTOS takes care of context switching.

RTOS - Example

Example "alarm" task based on *freeRTOS*
(the basis of the Arduino core on ESP32):

- The task is registered with higher priority than the main loop task:
xTaskCreatePinnedToCore() within the main **setup()**
- The task is moved to the *blocked* state while waiting inside **buzzNTimesAndWait()**
- The task is moved to the *ready* state when delays have passed.
- When the task is not *running*, the functions in **loop()** are executed (not shown).

Full code for this example:

https://github.com/tik.uni-stuttgart.de/INUE/es4ee_example_multitasking/blob/freertos/src/BuzzerTask.c

```
1 void buzzerRtosTask(void *param) {  
    BuzzerTaskData *data = (BuzzerTaskData *)param;  
    while (true) {  
        if (data->taskData.isRunning) { runBuzzerTaskOnce(data); }  
        else { vTaskDelay(1); } // yield to other tasks  
    }  
}  
void setup() {  
    initBuzzerTask(&buzzerTaskData, /*...*/);  
10 xTaskCreatePinnedToCore(buzzerRtosTask, /*...*/);  
    /*...*/  
}
```

```
1 static void buzzNTimesAndWait(const BuzzerTaskData *data) {  
    // Arduino's delay() is mapped to a freeRTOS function on ESP32,  
    // which automatically yields the CPU to other tasks.  
    for (int i = 0;  
        i < data->numPulses && data->taskData.isRunning; i++) {  
        buzzerOn(data); delay(data->pulseDuration);  
        buzzerOff(data); delay(data->pulseDuration);  
    }  
    if (data->taskData.isRunning)  
        delay(BUZZER_TASK_PERIOD  
            - data->pulseDuration * data->numPulses * 2);  
13 }  
void runBuzzerTaskOnce(BuzzerTaskData *data) {  
    buzzNTimesAndWait(data);  
}
```

- *freeRTOS* documentation:
https://www.freertos.org/Documentation/RTOS_book.html
- Another RTOS - *Zephyr Project*:
<https://docs.zephyrproject.org/latest/index.html>
In addition to the scheduling, includes also drivers for many peripherals, and a devicetree mechanism for configuring the system.
- Books about concepts of RTOS and embedded systems (among others), all available online via the university library: [1], [2], [3]

Embedded Linux

OS on microcontrollers - RTOS

Non-preemptive multitasking: "Bare-metal"

Preemptive multitasking - RTOS

Embedded Linux

Introduction

Linux kernel

Configuring and building the Linux kernel

Board support package

Linux distributions

Reasons for using a full-featured OS

A full-featured operating system is needed for complex systems, where most of these characteristics are present:

- Application-level processor with a memory management unit (MMU) is used (e.g. ARM Cortex-A or x86)
- Program storage and operative memory outside the CPU (e.g. eMMC and DRAM)
- External high-speed peripherals (e.g. USB, Ethernet, WiFi, Bluetooth, display, audio)
- The tasks are organized in processes which are loaded into RAM from executable program files on file systems
- Execution of high-level programming languages is desired (C++, Python, Java, JavaScript)

GNU/Linux in embedded systems

GNU/Linux¹ is a popular operating system for embedded systems:

- Open-source, maintained/reviewed by a large community
- Available for many architectures, including ARM
- Drivers for many peripherals are included in the OS kernel.
- The OS is highly configurable to meet the needs of the system
 - at build time and/or at run time.
- Many management and debugging tools are available, especially useful when interacting with a system which does not have a user interface suitable for development or service - due to limited resources or security reasons.

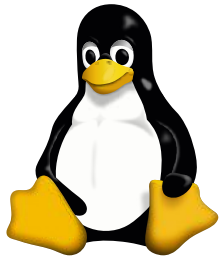


Image source: lewing@isc.tamu.edu Larry Ewing and The GIMP, public domain via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:Tux.svg>

¹"GNU/Linux" should be preferred over "Linux" when referring to the operating system, and not just the kernel: The Linux kernel is used only with GNU user-space programs on top of it, e.g. the C library **glibc**, the **bash** shell, tools like **tar** and **gzip**, or the **coreutils** package including **ls**, **cp**, **echo**, etc.

OS on microcontrollers - RTOS

Non-preemptive multitasking: "Bare-metal"

Preemptive multitasking - RTOS

Embedded Linux

Introduction

Linux kernel

Configuring and building the Linux kernel

Board support package

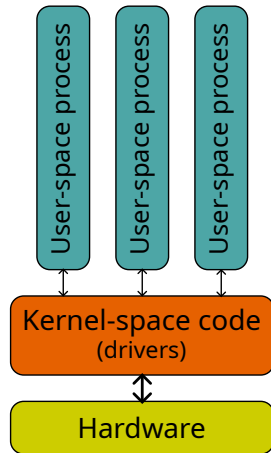
Linux distributions

The Linux kernel is the core of the operating system, responsible for managing the hardware resources and processes running on the system.

- Starts initially as a single process - like an application on an MCU
- Initializes the hardware within the processor and peripherals attached to it
- Dynamically shares the CPU time with other ("user space") applications, not known to the kernel compiler, by starting the first process (called **init**), which is responsible for starting all other processes.
- Implements mechanisms for process and thread scheduling, (virtual) memory management, inter-process communication, etc.
- Provides an interface for processes to access the hardware.

Kernel space and User space

- A hardware memory management unit (MMU) provides every process with its own virtual memory space mapped to distinct physical memory regions, preventing direct access to the address range reserved by memory-mapped peripherals.
- The MMU ensures that peripherals can only be accessed via program code located in the *kernel space* of the memory - where the OS kernel code is loaded at bootup.
- The kernel loads the application processes into the *user space* of the memory.
- User space processes can only access the hardware by calling functions provided by the kernel and residing in the kernel space – the so called *system calls*, see next slide.



Linux kernel - system calls

System call interface (SCI) is a set of functions provided by the kernel to the user-space processes. Many of the system calls are defined by the Portable Operating System Interface (POSIX) standard, IEEE 1003.1-2001, some are specific to Linux.

Linux kernel offers about 380 system calls for different purposes, the most important categories being:

- Process management: **fork()**, **exec()**, **exit()**,...
- File management: **open()**, **read()**, **write()**,...
- Communication - inter-process & over network: **pipe()**, **socket()**, **listen()**, **connect()**,...
- System information - time, name, hardware: **time()**, **uname()**, **gethostname()**,...

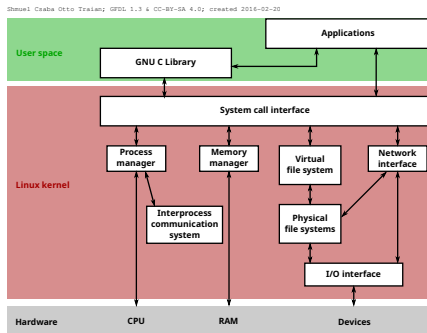


Image source: Shmuel Csaba Otto Traian, CC BY-SA 4.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Oversimplified_Structure_of_the_Linux_kernel.svg

System calls - continued

- Note: In the previous slide, no interface was shown for accessing the hardware, like `i2c_send()` or `gpio_set()`, as we would expect in microcontroller applications
- Reason: The device drivers implemented in the kernel do not expose interface functions for the user space processes.
- Instead, the drivers expose *device files* in the file system, which can be accessed by the user space processes using the file management system calls, reducing the variety of required system calls.

Note: More about device drivers and accessing devices through files in the next lecture.

OS on microcontrollers - RTOS

Non-preemptive multitasking: "Bare-metal"

Preemptive multitasking - RTOS

Embedded Linux

Introduction

Linux kernel

Configuring and building the Linux kernel

Board support package

Linux distributions

Configuring the Linux kernel

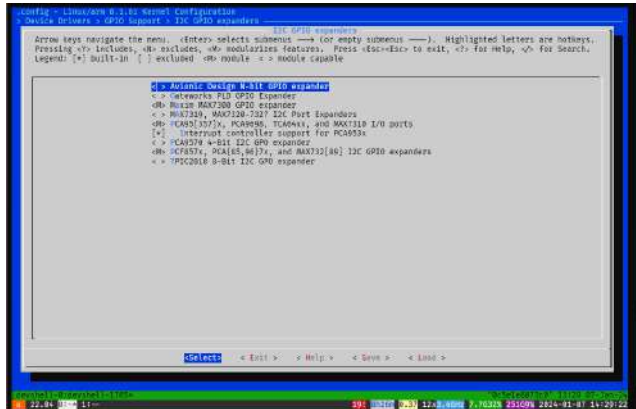
- The Linux kernel is highly configurable, allowing to select which parts of the source code are compiled and linked into the kernel binary.
- This is especially useful and important when using the Linux kernel in embedded systems, in order to get a kernel binary with the desired functionality and size.
- The configuration is achieved by defining compile-time macros which select C source files to be compiled (in Makefiles), and partially modify the behavior of the code within the C files.
- To manage the huge number of configuration options, the **Kconfig** system is used (see next slide)

Kconfig - continued

- The kernel source tree contains **Kconfig** files, in the directories of the components they configure.
- The **Kconfig** files specify the configuration options and their dependencies between each other.
- Text-based or graphical tools like **menuconfig** or **xconfig** can be used to build a configuration from the **Kconfig** files
- The configuration tool saves the resulting definitions in a **.config** file, which is then consumed by **make** during kernel compilation.

Kconfig - example

Example view of the tool **menuconfig**, editing the options defined in the file `drivers/gpio/Kconfig`, and the corresponding section in the resulting `.config` file.



.config file:

```
# CONFIG_GPIO_ADNP is not set
# CONFIG_GPIO_GW_PLD is not set
CONFIG_GPIO_MAX7300=m
# CONFIG_GPIO_MAX732X is not set
CONFIG_GPIO_PCA953X=m
CONFIG_GPIO_PCA953X_IRQ=y
# CONFIG_GPIO_PCA9570 is not set
CONFIG_GPIO_PCF857X=m
# CONFIG_GPIO_TPIC2810 is not set
```

Usage, e.g. in a C file (`drivers/gpio/gpio-pca953x.c`)

```
#ifndef CONFIG_GPIO_PCA953X_IRQ
static void pca953x_irq_mask(struct irq_data *d)
{
    // ...
    // (code used if CONFIG_GPIO_PCA953X_IRQ is defined)
#endif
```

OS on microcontrollers - RTOS

Non-preemptive multitasking: "Bare-metal"

Preemptive multitasking - RTOS

Embedded Linux

Introduction

Linux kernel

Configuring and building the Linux kernel

Board support package

Linux distributions

Board support package - BSP

Board support package (BSP) is a set of software components needed to run an operating system on a specific hardware platform.

- The BSP (or tools to compose it) is usually provided by the vendor of the target board, and includes these three components: ²
 - The *Linux kernel*, configured for the specific hardware as described above.
 - A *bootloader*, compiled to run on the target CPU, loading the Kernel from file storage into memory and starting it.
 - The *device tree* file, describing the hardware components of the system (see following slides)
- The BSP can be customized by the developer, e.g. by adding additional device drivers to the kernel or by customizing the boot behavior.

²Some definitions extend term *BSP* to mean a full disk image that also includes the user-space programs (RootFS, see below), and/or the toolchains for compiling the user-space software.

Device tree - 1

- Despite being compiled for a specific CPU architecture, the Linux kernel is by itself hardware-independent. Especially, the kernel does not need to know the addresses of memory-mapped peripherals at compile time (unlike on MCU applications).
- The device drivers are not initialized by the kernel until it is told how to access which hardware on the platform.
- On a PC, the BIOS or UEFI firmware is executed before the kernel, detects the connected hardware, and provides the kernel with the required information.
- On an embedded system where the hardware configuration usually does not change after building the device, the automatic discovery is dropped in favor of a more efficient and deterministic approach: the *device tree*.

Device tree - 2

- The device tree is a data structure describing the hardware components of the system.
- The definitions are stored in text files, usually with the extension **.dts** or **.dtsi** [4].
- In order to be used by the kernel, the device tree is compiled into a binary file **.dtb** (device tree blob) by the **dtc** tool.
- The device tree file is loaded to RAM and passed to the kernel at boot time by the bootloader.
- The entries in the device tree correspond to the device driver instances in the kernel, and can contain additional parameters passed to the drivers during their initialization [5]

Device tree - Example

Example: "PHYTEC phyBOARD-Mira Quad" (an evaluation board for an NXP i.MX 6 processor)

<https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/arch/arm/boot/dts/nxp/imx/imx6q-phytec-mira-rdk-emmc.dts>

File `imx6q-phytec-mira-rdk-emmc.dts` (the device tree for the board):

```
/dts-v1/;
#include "imx6q.dtsi"
#include "imx6qdl-phytec-phycore-som.dtsi"
#include "imx6qdl-phytec-mira.dtsi"
#include "imx6qdl-phytec-mira-peb-eval-01.dtsi"
#include "imx6qdl-phytec-mira-peb-av-02.dtsi"
#include "imx6qdl-phytec-mira-peb-wlbt-05.dtsi"

/ {
    model = "PHYTEC phyBOARD-Mira Quad Carrier-Board with eMMC";
    compatible = "phytec,imx6q-pbac06-emmc", "phytec,imx6q-pbac06",
        "phytec,imx6qdl-pcm058", "fsl,imx6q";

    chosen {
        stdout-path = &uart2;
    };
    // ...
};
```

Address range of this UART
in the memory map of the processor

File `imx6q.dtsi` (reusable definitions of the processor HW):

```
//...
#include "imx6qdl.dtsi"
/ {
    cpus {
        cpu0: cpu@0 {
            compatible = "arm,cortex-a9";
            //...
        }
        cpu1: cpu@1 {
            compatible = "arm,cortex-a9";
        }
    };
    //...
```

File `imx6qdl.dtsi` - reusable definitions of the HW common to a processor family -
defines also the memory-mapped device `uart2`:

```
uart2: serial@21e8000 {
    compatible = "fsl,imx6q-uart", "fsl,imx21-uart";
    reg = <0x021e8000 0x4000>;
    dmas = <&sdma 27 4 0>, <&sdma 28 4 0>;
    dma-names = "rx", "tx";
    // ...
};
```

Bootloader - 1

- The bootloader is a small program (comparable to an MCU application), usually stored in an unformatted region of the data storage drive (eMMC, SSD, ...) or on a separate storage device (e.g. SPI NOR flash).
- CPU automatically finds and executes the bootloader after reset, the bootloader is then responsible for loading the kernel into memory and starting it.
- The bootloader is usually only installed once - during device production, and is not updated during the lifetime of the system.
- However, the bootloader can have the ability to load the kernel from different locations, e.g. from external SD card or from network - useful for development and debugging.

Bootloader - 2

- In order to load the kernel, the bootloader needs to be able to access the storage devices or the network. Thus, drivers for the required storage devices, file systems and network interfaces need to be included in the bootloader.
- If not done by the CPU before starting the bootloader, the latter also needs to perform first-stage hardware initialization, e.g. configuring the clock sources and PLLs, and initializing the memory controller.
- A popular open-source bootloader for embedded systems is *U-Boot* [6].

OS on microcontrollers - RTOS

Non-preemptive multitasking: "Bare-metal"

Preemptive multitasking - RTOS

Embedded Linux

Introduction

Linux kernel

Configuring and building the Linux kernel

Board support package

Linux distributions

Linux distributions

- In order for a user-space application to usefully run on a Linux system, it needs to be accompanied by a set of other programs, libraries, and their configuration files.
- A *Linux distribution* is a collection of such software packages, which are tested to work together, and are installed on the so-called *root file system* (RootFS).
- A distribution needs to contain at least
 - the C library (e.g. **glibc**)
 - a shell (e.g. **bash**) with a set of basic tools, e.g. **ls**, **cp**, **echo**
 - an init system (e.g. **systemd**)
- Further packages are often also added, if needed for the application:
 - Higher-level programming language interpreters (e.g. Python, Java, JavaScript)
 - Wired and/or wireless network configuration
 - Remote access to the shell and/or to the filesystem
 - Logging facilities and update mechanisms

- For desktop and server systems, usually a pre-built distribution is used, e.g. Ubuntu, Debian, Red Hat, SUSE, ...
- On embedded systems, a custom distribution is often built, tailored to the needs of the application.
- The build process is usually automated by a build system, which
 - downloads the source code of the required packages,
 - configures and compiles them into packages,
 - and installs the packages into a RootFS, which is converted to a disk image file, ready to be flashed to the target storage device.

⇒ In this sense, the *partitioning* of the storage device, e.g. into a BSP partition, a RootFS partition, and a data partition, is also part of the distribution.
- Two very popular build systems are **yocto project** and **buildroot**.



Yocto Project

- The **yocto project** (grown out of the **OpenEmbedded** project) is a set of tools and meta-data for building an embedded Linux distribution.
- The meta-data is a collection of *recipe* files, organized in *layers*.
- The recipes describe the packages to be included in the distribution (also the Kernel and the bootloader), their interdependencies and configuration.
- The build process, based on the engine *BitBake*, is highly automated, and can be executed on a wide range of (Linux-based) host systems.

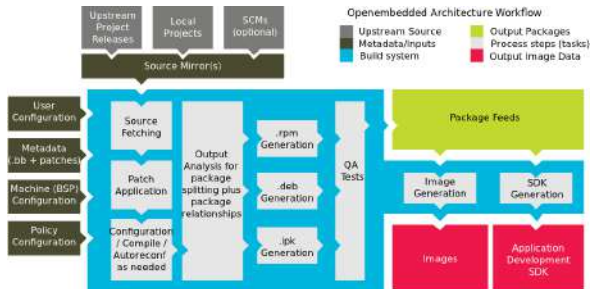


Image source: The Linux Foundation, CC BY-SA 2.0 <https://docs.yoctoproject.org/2.1/yocto-project-qs/yocto-project-qs.html>

Thank you for the attention!

Next lecture: Linux drivers and RootFS
Accessing the hardware from user space

References

Bibliography i

- [1] J. Beningo, *Embedded Software Design*. Apress, 2022. [Online]. Available: <https://doi.org/10.1007/978-1-4842-8279-3>
- [2] C. Ünsalan, H. D. Gürhan, and M. E. Yücel, *Embedded System Design with ARM Cortex-M Microcontrollers*. Springer Nature, 2022. [Online]. Available: <https://doi.org/10.1007/978-3-030-88439-0>
- [3] K. Murti, *Design Principles for Embedded Systems*. Springer Nature, 2022. [Online]. Available: <https://doi.org/10.1007/978-981-16-3293-8>
- [4] *Devicetree Specification, Release v0.4*, Devicetree.org, 2023. [Online]. Available: <https://github.com/devicetree-org/devicetree-specification/releases/tag/v0.4>
- [5] *Device Tree Usage*, Embedded Linux Wiki, 2023. [Online]. Available: https://elinux.org/Device_Tree_Usage

Bibliography ii

- [6] *Das U-Boot - the Universal Boot Loader*, DENX Software Engineering, 2024. [Online].
Available: <https://docs.u-boot.org/en/latest/>

Hardware Platforms and Programming of Embedded Systems

Lecture 10 - Linux kernel drivers and RootFS

Dr.-Ing. Alexander Knaub

2025-01-16

Institute of telecommunications (INÜ)

Contents

Linux kernel drivers - Introduction

- Relevance to this course

- Kernel driver architecture

File systems on Linux

- Storage devices, partitions and filesystems

- Linux Root filesystem (RootFS)

Kernel driver development

- Loadable kernel modules

- Kernel driver development

- Kernel driver source code structure

Linux kernel drivers - Introduction

Role of kernel drivers in Linux

- Linux was historically designed as a (free) replacement for (non-free) Unix, with the intention to be compatible with Unix and thus it took over the Unix philosophy *"everything is a file"*
- This means that the kernel exposes a filesystem-like interface to user-space programs, which can be used to access hardware devices
- The kernel drivers are software functions that react to userspace applications willing to access the hardware devices through the filesystem interface
- The kernel also exposes filesystem-like interfaces to observe and partially influence other kernel tasks, not related to hardware access, like process management, memory management, etc.

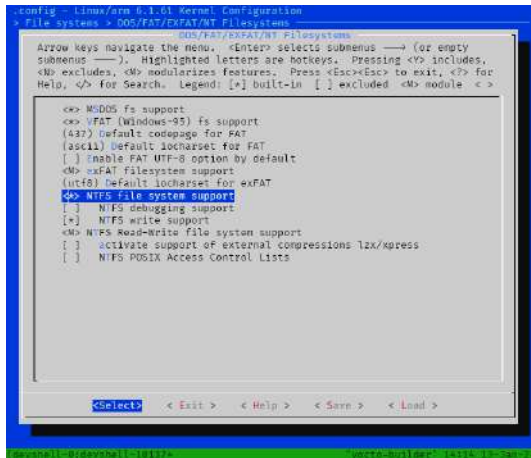
Use cases in embedded systems - drivers usable as-is

Many device drivers are already available in the kernel and can be used as-is.

They only need to be activated and maybe build-time parametrized in the kernel configuration (e.g. via menuconfig)

Example in the image:
menuconfig page for the FAT filesystem drivers.

- NTFS support will be compiled,
- the driver's option "NTFS write support" is activated,
- not the option "NTFS debugging support".



Use cases in embedded systems - parametrization via device tree

Some products you might be working on will use devices with available drivers that need (relatively simple) parametrization via the device tree file of your product's board.

Example in the code listing:

Snippet from a device tree file for the e-book reader "Tolino Shine 2 HD", showing an I²C bus and a touchscreen "Neonode zforce" connected to it.

The drivers are already available in the kernel, and the device tree file specifies the parameters for the drivers to use the hardware on this device correctly:

- I²C bus frequency (100 kHz)
- I²C slave address of the touchscreen controller (0x50)
- Touchscreen resolution (1072×1448)
- ...

Source: <https://github.com/torvalds/linux/blob/master/arch/arm/boot/dts/nxp/imx/imx6sl-tolino-shine2hd.dts>

```

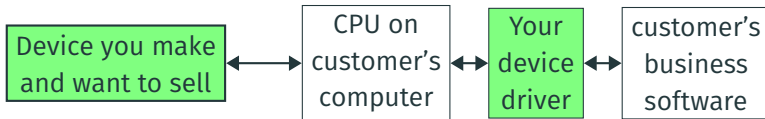
&i2c2 {
    pinctrl-names = "default", "sleep";
    pinctrl-0 = <&pinctrl_i2c2>;
    pinctrl-1 = <&pinctrl_i2c2_sleep>;
    clock-frequency = <100000>;
    status = "okay";

    zforce: touchscreen@50 {
        compatible = "neonode,zforce";
        pinctrl-names = "default";
        pinctrl-0 = <&pinctrl_zforce>;
        reg = <0x50>;
        interrupt-parent = <&gpio5>;
        interrupts = <6 IRQ_TYPE_EDGE_FALLING>;
        vdd-supply = <&ldo1_reg>;
        reset-gpios = <&gpio5 9 GPIO_ACTIVE_LOW>;
        x-size = <1072>;
        y-size = <1448>;
    };
};
```

Use cases in embedded systems - New drivers for new peripherals

You might be working on a new MCU or FPGA based product, offering an interface to (embedded) computers, like UART, SPI, I2C, USB, etc.

The customers will benefit from having a driver for your product in the kernel.



Use cases in embedded systems - New drivers for new hardware inside an SoC

You might be working on a SoC-FPGA based product¹, where a Linux distribution runs on the CPU part, and some product-specific hardware is implemented in the FPGA part.

There are IP cores usable in the FPGA part, for which kernel drivers can be already available (provided by the FPGA vendor - Intel/Altera or AMD/Xilinx), e.g. DMA engines.

For other very product-specific FPGA components you may design memory-mapped interfaces to the CPU and may need/want to write your own kernel drivers for accessing that interface.²

Example: See next slide

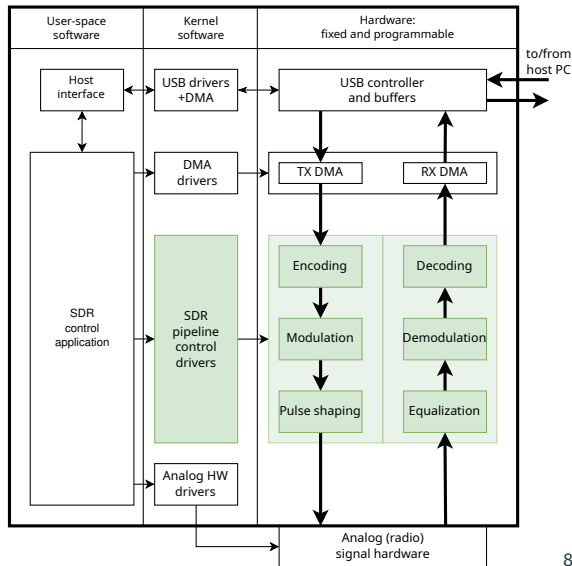
¹Remember lecture 3 on programmable hardware: An FPGA and a CPU on the same chip

²There are also ways to write user-space drivers, directly accessing the registers, and there are some reasons to do so, but this is out of the scope of this course.

SoC based application: Example

Example product: A Software-Defined Radio (SDR) device for customizable transmitting, receiving and processing of radio signals.

- The SDR device offers an USB 3.0 interface to a host computer
- The signal processing is implemented in the programmable (FPGA) part of the SoC.
- The CPU part runs a Linux-based *SDR control application* that configures and controls the signal flow in the FPGA part.
- The signal processing pipeline is controlled via a memory-mapped interface between the CPU and the FPGA.
- The *SDR pipeline control drivers* inside the kernel, provide access to this interface.



Linux kernel drivers - Introduction

Relevance to this course

Kernel driver architecture

File systems on Linux

Storage devices, partitions and filesystems

Linux Root filesystem (RootFS)

Kernel driver development

Loadable kernel modules

Kernel driver development

Kernel driver source code structure

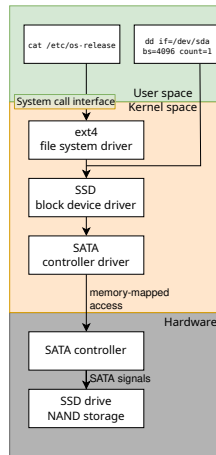
Hierarchical architecture of drivers within the kernel

- Every device has a driver, and the drivers can be used in a hierarchical way
- If a device A needs to be accessed through another device B, then the driver of device A uses the driver of device B.
- Within the kernel, the drivers don't use the filesystem interface, but directly access each other's functions.
- Despite being written in C, the source code shows many aspects of object-oriented programming, like encapsulation and inheritance.

Example: File access

Example: An actual data file, a stream of bytes on a storage device

- User application (e.g. `cat`) reads or writes a file (e.g. `/etc/os-release`) via a system call to the kernel
- The kernel initiates a sequence of function calls to drivers: Filesystem (ext4) → storage (SSD block device) → storage controller (SATA) → hardware
- The hardware controller accesses the storage device via the SATA bus
- The storage device driver can also be accessed directly from user-space via its own device file, e.g. `/dev/sda`, ignoring partitions and file systems (see later slides).



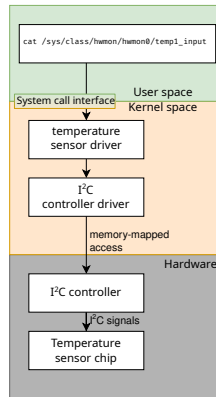
Example: Temperature sensor

Example: A temperature sensor connected via I²C, registered as hardware monitor in the kernel

- User application reads a file (e.g. `/sys/class/hwmon/hwmon0/temp1_input`), through which the kernel exposes the value of a temperature sensor.
- The kernel initiates a sequence of function calls to drivers: hwmon device class → this temperature sensor → I²C controller
- The I²C controller driver performs an I²C transaction via the controller's memory-mapped registers

Real-world example: The popular temperature sensor TMP103 from Texas Instruments

<https://github.com/torvalds/linux/blob/master/drivers/hwmon/tmp103.c>



File systems on Linux

Linux kernel drivers - Introduction

Relevance to this course

Kernel driver architecture

File systems on Linux

Storage devices, partitions and filesystems

Linux Root filesystem (RootFS)

Kernel driver development

Loadable kernel modules

Kernel driver development

Kernel driver source code structure

Storage devices

- A storage device is a device that can store data persistently³
- Usually, especially in embedded systems, a storage device is based on NAND, sometimes NOR, flash storage technology.
- For historical reasons, the term "disk" is often used for any storage device, even if it is, like flash, not based on rotating magnetic or optical disks.
- Due to the nature of flash memory, a storage device is usually divided into blocks, dictated by the page size of the device. Typical block size is e.g. 4096 bytes.
- "Block device" means that whenever one byte of data is written to the device, the whole block containing that byte is read, modified and written back to the device.
- Usually the total storage capacity of a storage device is divided into partitions, and each partition can be formatted with a filesystem (see later slides)

³Remember lecture 4 "Memory and storage"

Storage devices - raw access

- It is possible to read and write directly to individual blocks of a storage device, bypassing or ignoring potentially present filesystems and partitions. This is called *raw access* to the device.
- Raw (also: "bare metal") access can be necessary: Some processors look for a bootloader at a specific offset on a storage device, not in a file system.⁴
- On Linux, raw access to a storage device is possible via its device file, e.g.:
`sudo dd if=./flash.bin of=/dev/mmcblk0 bs=1k seek=32`
(writes 1-KiB blocks from the file **flash.bin** at offset 32 blocks of the device **/dev/mmcblk0**)
- Compare to a blank paper notebook: You can write to any page of the notebook, as long as you remember the page number when trying to read it again.
(For large notebooks, you'd want a table of contents or another kind of an index though)

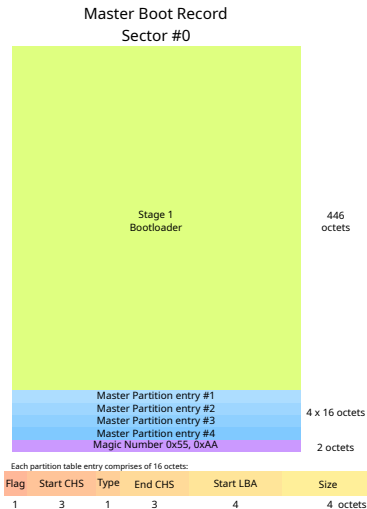
⁴Example: NXP i.MX8 processors expect the bootloader at offset 0x8000 (32 KiB) on the boot device
<https://community.nxp.com/t5/i-MX-Processors-Knowledge-Base/i-MX8-Boot-process-and-creating-a-bootable-image/ta-p/1101253>

Disk partitioning

- A storage device can be divided into partitions - internally contiguous and externally disjunct areas of the storage device.
- If a device is partitioned, the boundaries of the partitions are written into the *partition table*, which is stored in the first block(s) of the device,
 - like a table of contents.
- Other names for the partition table are *partition map* or *disk label*.
The latter term is used by the popular tool GNU **parted**.
- There are different partition table formats, the two most common ones are *MBR* (Master Boot Record) and *GPT* (GUID Partition Table)
- In Linux, the partitions of block devices are also exposed as (smaller) block devices, e.g. `/dev/sda1` on disk `/dev/sda`, or `/dev/mmcblk0p1` on disk `/dev/mmcblk0`

MBR partition table

- The MBR partition table format is one of the oldest (from 1983), and is still widely used due to its simplicity and compatibility with most "IBM PC compatible" systems.
- It is limited to 4 partitions and can handle devices with up to 2 TiB storage capacity (usually sufficient for embedded systems)
- The partition table is stored in the first 512 bytes of the storage device, after a small bootloader used by BIOS-based systems to boot from the device.
- An MBR partition table can be initialized with the tools **fdisk** or GNU **parted**, MBR is called "**msdos** disk label" in **parted**.



GUID partition table

- The GPT^a format is part of the UEFI standard (Unified Extensible Firmware Interface), from 2000.
- "G" in GPT stands for GUID (Globally Unique Identifier). These 128-bit pseudorandom numbers are used in the partition table to identify the partitions and their format types.
- GPT raises the limit to 128 partitions and can handle devices with up to 8 ZiB storage capacity (1 ZiB = 2³⁰ TiB).
- The first 512 bytes of the device are reserved for a small protective MBR partition table. Beyond that a header (512 bytes) and a table of partition entries (128 × 128B) are stored.
- A GPT table can be initialized with the tools **gdisk** or GNU **parted**, the latter refers to GPT as disk label "**gpt**".

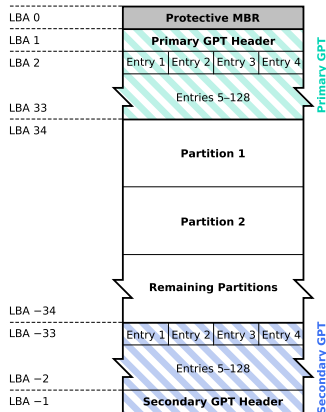
^aThe resemblance to ChatGPT is purely coincidental.

Image source: Kbolino, CC BY-SA 2.5 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:GUID_Partition_Table_Scheme.svg

LBA = Logical Block Addressing, uses typically 512 bytes per block

GUID Partition Table Scheme



Filesystems

- A filesystem is a method to organize data in a tree-like hierarchical structure of files and directories (folders)
- *Tree structure*: A filesystem contains one root directory, and each directory can contain files and subdirectories
- The most common use case for a filesystem (not the only one) is to keep an index of data files on a storage device
- Each storage device partition can be formatted with one filesystem.
- This means that some control data structures are written to the partition along with the actual data files, to keep track of the location of the files on the disk.
- These data structures are managed by filesystem kernel drivers, so that user-space applications don't need to know about the details of the filesystem.

```
/ #root directory#
├── bin
│   ├── bash
│   ├── cat
│   └── ...
├── boot
│   └── zImage
├── dev (empty)
├── etc
│   ├── network
│   │   └── interfaces
│   ├── fstab
│   └── ...
├── home
├── lib
├── mnt
├── proc (empty)
├── run (empty)
├── sys (empty)
└── ...
```

Image: Exemplary filesystem structure of the main partition of a Linux system (RootFS)

Directories shown as "empty" do not contain any files on the disk, and are populated at runtime by the kernel.

Filesystems for storage devices

- There are different filesystem types for storage devices on Linux.
The most common ones are:
 - Linux native file systems in different generations: ext2, ext3, ext4
 - Windows file systems: FAT (and its extensions FAT16, FAT32, exFAT), NTFS
 - Other special types: BTRFS, XFS, JFS, ...
- The filesystem types differ in various parameters, e.g.:
 - *Numerical limits*: File size, number of files in a directory, total number of files on the filesystem, maximum length of file names and paths, ...
 - *Features*: Journaling, access permissions control, compression, encryption, ...
 - The implementation the filesystem handling influences the *performance*:
Read/write speed, memory usage, size of the filesystem driver in the kernel, ...

Virtual filesystems

Since every filesystem-based access to a storage device is handled by a kernel driver, it is just as easily possible for the kernel to expose other information and functionality as filesystems to user-space applications → *Virtual filesystems*.

Examples on Linux:

- **tmpfs**: Files are "stored" in RAM
(RAM disk, used e.g. for temporary files disappearing after reboot)
- **procfs**: Information about processes and kernel tasks, exposed by the kernel
- **sysfs**: Provides information about hardware devices, exposed by the kernel
- **devtmpfs**: Access to character and block devices, exposed by the drivers in the kernel
- ...

Linux kernel drivers - Introduction

Relevance to this course

Kernel driver architecture

File systems on Linux

Storage devices, partitions and filesystems

Linux Root filesystem (RootFS)

Kernel driver development

Loadable kernel modules

Kernel driver development

Kernel driver source code structure

Linux Root filesystem (RootFS)

- The Root filesystem (RootFS) is the filesystem of the main partition of a Linux system
- Kernel refers to the root folder of the RootFS as `"/"`
- The RootFS must contain specific directories in order for the Linux system to work:
 - `/etc`: System configuration files
 - `/bin` and `/sbin`: Essential system executables
 - `/opt`, `/usr`: Additional executables and libraries
 - `/lib` and `/lib64`: Shared system libraries
 - used by the executables in `/bin`, `/sbin`, `/opt` and `/usr`,
 - `/lib` also contains loadable kernel driver modules - see later
 - Mountpoints for virtual file systems: `/dev`, `/proc`, `/sys`, `/run`, `/tmp`
 - `/root` and `/home`: Home directories of the root user and other users
- This directory structure is mandated by the Filesystem Hierarchy Standard (FHS), defined by the Linux Standard Base (LSB) specification [1]

Mounting of filesystems

- From the storage device's view, a partition formatted with a filesystem is not more than a sequence of bytes. Once a kernel runs, its drivers can interpret the data as a filesystem, and interact with it accordingly.
- The partitions not being the RootFS can be *mounted* to a directory somewhere within the RootFS tree.
 - This selected directory is then called *mount point*.
 - The root directory of the mounted filesystem is then mapped to the mount point, which makes the files and directories in the mounted filesystem accessible to the user.
- The RootFS is itself the first filesystem to be mounted - namely to the directory `/`.
- This way, other partition of the same storage device, or other storage devices (e.g. external flash drives) can be accessed via subdirectories of the RootFS.
- By convention, filesystems on external partitions are usually mounted to subdirectories of `/mnt` or `/media`.

Loop devices

- A *loop device* is a virtual device that maps a (binary) file to a block device driver.
- The file can contain partitions and filesystems. These partitions can be mounted to a directory in the RootFS.
- Loop devices are used e.g. when generating disk images for embedded systems:
 - Create an empty file, with a size sufficient to contain the desired data.
 - Attach the file to a loop device, partition it, and format the partitions with filesystems
 - Mount the partition filesystems to directories in the *host system's* RootFS
 - Populate the filesystems with the desired files and directories, especially with the *target* RootFS
 - After unmounting the partitions and detaching the loop device, the file can be 1:1 copied (flashed) to the target storage device
- These steps are usually automated by tools provided by the distribution build systems, e.g. **genimage** from Buildroot or **WIC** from Yocto

Initial RAM filesystem (initramfs)

- We know that the RootFS is the first filesystem to be mounted.
- We also know that files or areas in RAM can be mounted as filesystems, just like disk partitions (loop devices, resp. RAM disks).
- So, the kernel can be given a preinitialized filesystem in RAM, which it can mount as the RootFS.
- Such a filesystem image in RAM is called *initial RAM filesystem* (initramfs).
- If an initramfs is used, the bootloader loads it from the storage device to RAM, in addition to the kernel image itself, and passes the RAM address of the initramfs to the kernel.

initramfs - continued

- The initramfs is usually small (must fit into RAM and leave free memory for the operation), and contains only the essential executables.
- A use case for an initramfs can be to have userspace tools available for mounting the actual RootFS from a location, not accessible by the bootloader, e.g. a network drive, an encrypted partition requiring password provided at user input, or from a logical volume⁵ [2].
- In order to instruct the kernel to switch from the initramfs to the actual RootFS, a userspace program called **switch_root** is used.
- Not to be confused with **chroot**, which is a lightweight virtualization tool for changing the root directory of a single userspace process, while the kernel still uses the original RootFS. **chrootbf** is used e.g. for sandboxed environment like Docker containers.

⁵A logical volume is a virtual partition, provided by the Logical Volume Manager (LVM), which can span several partitions on multiple physical disks.

Kernel driver development

Linux kernel drivers - Introduction

- Relevance to this course

- Kernel driver architecture

File systems on Linux

- Storage devices, partitions and filesystems

- Linux Root filesystem (RootFS)

Kernel driver development

- Loadable kernel modules

- Kernel driver development

- Kernel driver source code structure

Built-in kernel drivers vs. loadable kernel modules

- Until now, we assumed that all kernel drivers are compiled into the kernel image.
- The kernel also supports *loadable kernel modules* (LKM), which are
 - compiled from the same sources, however not linked into the kernel image, but into separate *kernel object* (KO) files
 - placed in the directory `/lib/modules/<kernel version>/` in the RootFS
 - loaded into the kernelspace at runtime, as needed
- Some drivers *must be built-in*, e.g. the driver for the RootFS and its storage device
- Some *should be loadable* modules, e.g. the driver for a USB device if the device is not always connected to the system
- Some *must be loadable* modules, e.g. drivers that are responsible for loading configuration data to their devices, while the data is stored in the RootFS (`/lib/firmware`), available only after the kernel has started

Kernel driver modules

- Drivers that can be compiled as modules, show this via their Kconfig file: The option to enable or disable this driver is not a *boolean* but a so-called *tristate* option.
- The tristate option can have three values, selectable when configuring the kernel:^a
 - `<` or `n`: Do not compile this driver
 - `<*` or `y`: Compile this driver and link it into the kernel image
 - `<M>` or `m`: Compile this driver as a loadable kernel module

^aSee also menuconfig screenshots on earlier slides and in the previous lecture

```
menuconfig BLK_DEV
    bool "Block devices"
    depends on BLOCK
    default y
    help
        Say Y here to get to see options for various
        different block device drivers.
    ...

if BLK_DEV

source "drivers/block/null_blk/Kconfig"

config BLK_DEV_FD
    tristate "Normal floppy disk support"
    depends on ARCH_MAY_HAVE_PC_FDC
    help
        If you want to use the floppy disk drive(s)
        of your PC under Linux, say Y.
    ...

config BLK_DEV_FD_RAWCMD
    bool "Support for raw floppy disk commands"
    depends on BLK_DEV_FD
```

Code listing: Snippet from the Kconfig file of block device drivers.

The option `BLK_DEV_FD` is a tristate option, while `BLK_DEV_FD_RAWCMD` is a binary yes-or-no option.

<https://github.com/torvalds/linux/blob/master/drivers/block/Kconfig>

Linux kernel drivers - Introduction

Relevance to this course

Kernel driver architecture

File systems on Linux

Storage devices, partitions and filesystems

Linux Root filesystem (RootFS)

Kernel driver development

Loadable kernel modules

Kernel driver development

Kernel driver source code structure

Kernel driver source files - 1

A kernel driver usually has these source file types inside the kernel source tree:

- Source code - the actual C and header files, implementating the driver's functionality
- Kconfig file - allowing to (de)select and configure the driver when configuring the kernel before the build.
 - Either an own one for the driver itself if the driver has its own directory
 - Or a set of entries in the "parent" Kconfig file of the driver's category/directory
- Makefile - specifying the build rules for the driver
- Optionally, device tree bindings
 - Header files with named constants definitions, usable in the driver's source code and in the device tree file, where the driver's parameters are specified
 - Located in the source tree directory `include/dt-bindings/`

Kernel driver source files - 2

Location of useful files in the kernel source tree:
(example: A hypothetical driver for an RGB LED controller "we" have designed, accessible via I²C)

- `drivers/led/my-i2c-rgb-led.c` - contains the implementation of "our" driver's functions
- `drivers/led/Kconfig`, `drivers/led/Makefile` - contain the driver's Kconfig entries resp. build rules, along with that of other LED drivers
- `include/linux/*.h` - Useful header files declaring other driver's functions used by "our" C file
- `include/dt-bindings/leds/my-rgb-led.h`
 - An optional header file with constants that can be used in the device tree file to specify parameters valid for a specific instance of "our" LED controller device

```
/ #kernel source directory#
├── Documentation
│   └── leds
│       ├── index.rst
│       └── leds-class.rst
├── drivers
│   ├── leds
│   │   ├── my-i2c-rgb-led.c
│   │   ├── ...
│   │   ├── Kconfig
│   │   └── Makefile
│   ├── Kconfig
│   └── Makefile
├── include
│   ├── linux
│   │   ├── ...
│   │   ├── device.h
│   │   └── i2c.h
│   └── dt-bindings
│       ├── leds
│       │   └── my-rgb-led.h
│       └── ...
├── Kconfig
├── Makefile
└── ...
```

Out of tree modules

- It is possible to compile a kernel driver module without integrating it into the kernel source tree. This is then called an *out-of-tree* module.
 - The source files of the module are then located in a separate directory, defined by the developer.
 - The driver is per definition a module, and it is compiled independently of the Kernel, so a Kconfig file is not needed
 - There is no Makefile from the kernel source tree, so a dedicated Makefile for the module being developed is necessary.
- The module needs to be exactly compatible with the kernel it is loaded into, so the kernel headers and compiler options used for the build of the module must match those used for the kernel build.
- Linux distribution build systems assist with tools for building out-of-tree modules. For example [3] in the Yocto Project.

Linux kernel drivers - Introduction

- Relevance to this course

- Kernel driver architecture

File systems on Linux

- Storage devices, partitions and filesystems

- Linux Root filesystem (RootFS)

Kernel driver development

- Loadable kernel modules

- Kernel driver development

- Kernel driver source code structure

Kernel driver source code structure

During runtime, different parts of the drivers can be found in different regions of the kernel-space memory.⁶

- **.text** - The instructions of the functions defining the driver's behavior;
- **.data** and **.bss** - static variables, allocated in the RAM before the kernel startup. These variables hold global data structures reusable by all driver instances, or those used by the kernel to link the driver instances to the functions.
- **.heap** - variables separately allocated for *each* device compatible with the given driver, used by the kernel and by the functions to communicate with the concrete device instance and to keep track of the device's current state.

More details and some examples for the different components of driver source code will be given on following slides.

⁶Remember lecture 8 "Software for microcontrollers", where memory regions were presented

Kernel driver source code structure

1. Declaration of the driver's data types:

structs, enums, unions, typedefs for data structures representing one device instance

Random example: <https://github.com/torvalds/linux/blob/master/drivers/leds/leds-lm3697.c>

```
struct lm3697_led {
    u32 hvled_strings[LM3697_MAX_LED_STRINGS]; // the device name
    char label[LED_MAX_NAME_SIZE];
    struct led_classdev led_dev;
    // data for accessing the device, see below
    struct lm3697 *priv;
    struct ti_lmu_bank lmu_data;
    int control_bank;
    int enabled; // state of the LED: on or off
    int num_leds;
};
```

```
struct lm3697 {
    // dedicated GPIO pin to disable _this_ device
    struct gpio_desc *enable_gpio;
    struct regulator *regulator;

    // The I2C connection to _this_ device,
    // managed by the kernel's I2C subsystem
    struct i2c_client * client;
    // for easier access to the device's registers via `client`
    struct regmap *regmap;

    // Device identification data in kernel, here for logging only
    struct device *dev;

    // prevent concurrent access to _this_ device
    // from multiple processes
    struct mutex lock;

    int bank_cfg;
    int num_banks;

    struct lm3697_led leds[] __counted_by(num_banks);
};
```

Kernel driver source code structure

2. Definition of the driver's operation functions

Functions that will be mapped to the device-specific file operations, take a pointer to the (already initialized) device instance struct as argument.

Same random example: <https://github.com/torvalds/linux/blob/master/drivers/leds/leds-lm3697.c>

```
// Called on an initialized device to set the LED brightness
// Mapping to operation `write` on this device's `brightness` file:
// see below (later slides)
static int lm3697_brightness_set(struct led_classdev *led_cdev,
                                enum led_brightness brt_val)
{
    // get device-specific data structure defined above
    struct lm3697_led *led = container_of(led_cdev,
                                           struct lm3697_led,
                                           led_dev);

    // use the device-specific data to access the device
    // via i2c and set the brightness.
    // Namely, via the pointer led->priv->regmap

    //...
}
```

```
// Internal helper function. Called from the same file
// during initialization of the driver instance,
// once data structure lm3697 has been initialized on heap.
// Performs I/O operations necessary for the device to work
static int lm3697_init(struct lm3697 *priv)
{
    struct device *dev = priv->dev;

    if (priv->enable_gpio /* != NULL */) {
        gpiod_direction_output(priv->enable_gpio, 1);
    }

    // Send "enable" commands to the device via I2C
    // namely via the pointer priv->regmap
    // ...
}
```

Kernel driver source code structure

3. Definition of the driver's maintenance functions

Functions that are called by the kernel for the driver's life cycle management.

Same random example: <https://github.com/torvalds/linux/blob/master/drivers/leds/leds-lm3697.c>

```
// Called by the driver to initialize a new driver instance, when
// detecting a compatible device in the device tree.
// Parameter `client` provides access to the device on the I2C bus.
static int lm3697_probe(struct i2c_client *client)
{
    // Allocate memory for the device instance on the kernel heap
    struct lm3697 *led = devm_kzalloc(dev,
        struct_size(led, leds, count), GFP_KERNEL);

    // Make the connection I2C client -> driver
    i2c_set_clientdata(client, led);

    // Make the connection driver -> I2C client
    led->client = client;

    // Not on the slide: Read parameters from device tree

    // Link the brightness_set function to the "brightness" file:
    ...->led_dev.brightness_set_blocking = lm3697_brightness_set;
    // Register the device in the kernel's LED subsystem
    // so it appears below /sys/class/leds/
    devm_led_classdev_register_ext(dev, ...->led_dev, &init_data);
}
```

```
// Called by the kernel when the driver instance is no longer needed.
// Usually when the driver is a module and is unloaded via userspace.
static void lm3697_remove(struct i2c_client *client)
{
    struct lm3697 *led = i2c_get_clientdata(client);
    struct device *dev = &led->client->dev;
    int ret;

    // Send the "disable" command to the device via I2C
    ret = regmap_update_bits(led->regmap, LM3697_CTRL_ENABLE,
        LM3697_CTRL_A_B_EN, 0);

    if (ret)
        dev_err(dev, "Failed to disable the device\n");

    // Pull down the device's "enable" GPIO pin
    if (led->enable_gpio)
        gpiod_direction_output(led->enable_gpio, 0);

    // ...
}
```

Kernel driver source code structure

4. Definition and initialization of the driver's data structures:

Static global variables common to all instances of the driver

Provide meaning to the functions defined above by linking them to operations expected by the Kernel.

Same random example: <https://github.com/torvalds/linux/blob/master/drivers/leds/leds-lm3697.c>

```
// Complicated way for giving this driver an ID
static const struct i2c_device_id lm3697_id[] = {
    { "lm3697", 0 },
    { }
};
MODULE_DEVICE_TABLE(i2c, lm3697_id);

// Define how devices in the device tree can tell
// that they are _compatible_ with this driver
static const struct of_device_id of_lm3697_leds_match[] = {
    { .compatible = "ti,lm3697", },
    {},
};
MODULE_DEVICE_TABLE(of, of_lm3697_leds_match);
```

```
// Make the driver functions visible to the I2C subsystem:
// pack everything in to an expected data format
static struct i2c_driver lm3697_driver = {
    .driver = {
        .name = "lm3697",
        .of_match_table = of_lm3697_leds_match,
    },
    .probe = lm3697_probe,
    .remove = lm3697_remove,
    .id_table = lm3697_id,
};
// Pass the data to the I2C subsystem:
module_i2c_driver(lm3697_driver);
```

Organization of device files

- All devices known to the kernel are listed via SysFS - mounted on `/sys`
 - Through this interface, drivers can offer files influencing their behavior, e.g. changing the direction of a GPIO pin.
 - Indeed, some drivers base their entire operation on files in SysFS, for example the LED driver shown above, or temperature sensors that can be read out and configured (e.g. sampling modes) through files in SysFS
- For continuous or repeated I/O with a device, interface files are exposed in the `/dev` filesystem. Two types of drivers can be distinguished: *Block devices* and *Character devices* (see next slide)

Block and character device drivers

- *Block devices* allow reading and writing from arbitrary addresses. Usually used for representing storage devices and partitions, as presented on previous slides.
- *Character devices* allow reading and/or writing a linear stream of bytes. A practical representation of serial communication interfaces.
- Some special character devices:
 - `/dev/null` Accepts any data and just ignores it
 - `/dev/zero` Provides a stream of as many zero-bytes as desired.
 - `/dev/random` Provides a stream of random bytes. If available, hardware for generating true randomness is utilized (e.g. thermal noise inside the CPU)
 - `/dev/tty<N>`, `/dev/ttyUSB<N>`,...- real or virtual UART interfaces

Further reading

Topics we did not cover in this lecture that might be interesting for further learning:

- Deeper dive into properties of filesystems (journaling, copy-on-write behavior, etc.)
- Debug and management interfaces provided by the kernel (debugfs, procfs)
- Userspace device management (UDEV)
- Direct access to the memory-mapped registers from userspace via the special device file `/dev/mem`

Thank you for the attention!

Next lecture:

Tools and processes of continuous integration

References

Bibliography i

- [1] *Filesystem Hierarchy Standard*, LSB Workgroup, The Linux Foundation, 3 2015. [Online]. Available: https://refspecs.linuxfoundation.org/FHS_3.0/fhs-3.0.pdf
- [2] *Beyond Linux From Scratch - Chapter 5. File Systems and Disk Management*, The BLFS Development Team, 4 2021. [Online]. Available: <https://www.linuxfromscratch.org/blfs/view/cvs/postlfs/initramfs.html>
- [3] *Yocto Project Linux Kernel Development Manual - Incorporating Out-of-Tree Modules*, Linux Foundation, 2024. [Online]. Available: <https://docs.yoctoproject.org/kernel-dev/common.html#incorporating-out-of-tree-modules>

Hardware Platforms and Programming of Embedded Systems

Lecture 1 - Introduction

former "Embedded Systems
for Electrical Engineers"

Dr.-Ing. Alexander Knaub

2024-10-17

Institute of telecommunications (INÜ)

Contents

About us

What's in the lecture

What's *not* in the lecture

Introduction into embedded systems

- Examples of embedded systems

- Importance of embedded systems

- Requirements on embedded systems

- Hardware, software, firmware

About us

About the lecturer

Alexander Knaub

- 2011 - Graduated Electrical Engineering @ Uni Stuttgart
- 2011-2017 - Research at INUE, Uni Stuttgart
in area of signal processing for industrial communication systems
In parallel: Teaching - Exercises, lab course, student theses
- Since 2017 - Embedded software and firmware development for different FARO laser scanner families

Contact

Mail: alexander.knaub@inue.uni-stuttgart.de

Support by research assistants at INUE:

Maximilian Bauhofer

Mail: bauhofer@inue.uni-stuttgart.de

Tel: +49 711 685 67929

Marc Gauger

Mail: gauger@inue.uni-stuttgart.de

Tel: +49 711 685 69207

For contacting you if necessary, we will use ILIAS.

Contact us via ILIAS or E-Mail, use ILIAS forum.

About the lecture

- On-site lecture:
Thursdays 15:45 - 17:15, this lecture hall V47.01.
- Exercise format:
Problems solved on video, uploaded to ILIAS.
Interactive Q&A sessions on Thursdays 17:30 - 19:00, V47.03
Not every week: Dates will be announced through ILIAS.
- 15 lectures; first: 2024-10-17, last: 2024-02-06
- Lecture language: English, you can ask questions in German

Exam

- Written, 120 minutes, after the lecture period
Exact date will be announced. *≈ beginning of March*
- Registration via C@mpus.
- Format: Mostly single-choice questions, some open questions.
- (Probably) open book, no electronic devices allowed.
- Relevant material will be provided via ILIAS during the semester:
 - Lecture slides
 - Exercises
 - Supplementary material

WS 23/24 Test Exam "Embedded Systems for Electrical Engineers" Exam sheet 1, Page 1/6

2.2. Which of the following software components run in the kernel space?

- ☐ Process scheduler and user file system
- ☐ Terminal and the standard C library
- ☐ User file system and the standard C library
- ☐ C compiler and terminal

2.4. What is the maximum number of partitions that can be created on a disk with the MBR partitioning scheme? (Select one)

- ☐ 2
- ☐ 4
- ☐ 16
- ☐ 128

2.5. What are the typical output file formats of GCC toolchain components?

Compiler: ☐ .o ☐ .a ☐ .lib ☐ .out

Assembler: ☐ .o ☐ .a ☐ .lib ☐ .out

Linker: ☐ .o ☐ .a ☐ .lib ☐ .out

3.1. Consider the control graph in the picture. When starting in control "A", which sequence of CIL commands would create the graph?

- ☐ control, switch, control, merge
- ☐ control, control, switch, merge
- ☐ control, switch, control, release
- ☐ control, switch, control, control

3.2. Which operation is crucial in the Trunk Based Development (TBD) workflow?

- ☐ Merge
- ☐ Release
- ☐ Cherry-pick
- ☐ Fetch

4. Loose hardware-related questions

4.1. What does the acronym CPU stand for?

- ☐ central processing unit
- ☐ code processing unit
- ☐ central programming unit
- ☐ code programming unit

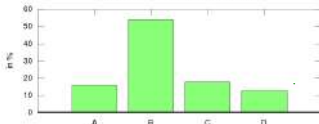
4.2. What does a CPU integrate?

- ☐ only processor
- ☐ processor and memory
- ☐ processor, memory and peripherals
- ☐ processor and peripherals

About you

Answers 119

Have you ever programmed a microcontroller (µC, MCU) or worked with embedded hardware before?



A: No, never

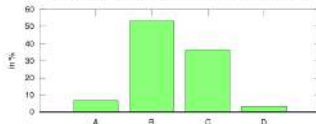
B: Simple programs for a microcontroller.

C: Moderate complexity embedded projects.

D: I have extensive experience with embedded systems.

Answers 120

With how many of these programming languages are you familiar? C/C++, Python, Rust, VHDL/Verilog, Shell scripts, others (count as 1)



A: None

B: 1-2

C: 3-4

D: 5 or more

Answers 119

How familiar are you with the area of programmable hardware? Buzzwords: FPGA, CPLD, VHDL/Verilog, ModelSim, Synthesis



A: Only heard of them

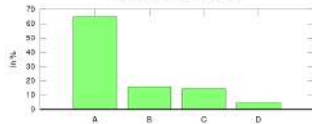
B: Have written and simulated some HDL code

C: Have synthesized a design for an FPGA/CPLD/ASIC

D: Have programmed an FPGA in the context of a more complex system

Answers 112

Have you worked with any real-time operating systems (RTOS) or embedded operating systems (e.g., FreeRTOS, Zephyr) in your previous studies or projects?



A: I have no experience with RTOS.

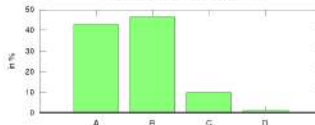
B: I have theoretical knowledge but no practical experience.

C: I have used an RTOS in a project or coursework.

D: I have extensive hands-on experience with RTOS and embedded OS.

Answers 114

Have you worked with the Linux kernel in any of your previous studies or projects?



A: The closest I got to a Linux system was using an Android device

B: I have typed some commands into the Linux terminal (bash)

C: I have compiled a Linux kernel that I have modified/reconfigured

D: I have contributed to the kernel or developed a kernel module

Quiz to get to

What's in the lecture

Definitions

Citations from [1]:

1

Embedded systems are things like microwaves and automobiles that run software but aren't computers.

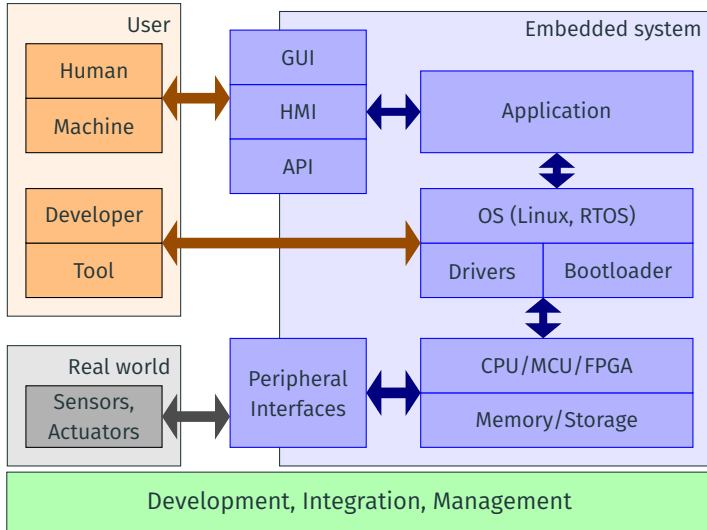
2

An embedded system is a computerized system that is purpose-built for its application.

3

Another way to identify embedded systems is that they use cross-compilers.

Scope



Contents

Topics	Lecture No.
1 Introduction	1
2 Hardware	2
2.1 Computing architectures	2
2.1.1 Architecture of microprocessors and microcontrollers	2
2.1.2 Custom-designed hardware (FPGA & Co.), other platforms	3
2.2 Memory/Storage	4
2.2.1 Volatile: On-Chip (Registers, Cache: SRAM), DRAM	4
2.2.2 Persistent storage devices: EEPROM, Serial Flash, eMMC, SSD	4
2.3 Input/Output interfaces and protocols	5
2.3.1 Exemplary sensors, actors	5
2.3.2 Board-level communication: I2C, SPI, PCI(e)	5
2.3.3 Inter-board communication: JTAG, UART, USB, Wireless PAN	5
2.3.4 Local Area Networks (LAN): Ethernet, WLAN	6
2.3.5 Wide Area Network (WAN): GSM, LTE	6
2.4 Power supply (AC/DC, battery,...)	7

...

...

3	Software	8
3.1	Microcontroller software, RTOS	8
3.2	Embedded Linux	9
3.2.1	Board support package: Kernel, Bootloader, Device Tree	9
3.2.2	Device drivers	10
3.2.3	Organization of data in file systems	11
3.2.4	Interface Kernel ↔ userspace through Linux RootFS	11
3.3	(Business) Application software	12
3.3.1	Converting C/C++ code into embedded software: GCC, Crosscompiling, Cmake	12
3.3.2	Potential of modern C++ on embedded systems	13
3.3.3	Cryptography, SecureBoot	13
3.3.4	Application interfaces	14
3.3.5	User interfaces	14
3.4	Software/Firmware management	15
3.4.1	Continuous integration: Versioning (git), Tooling	15
3.4.2	Linux distribution build system Yocto/Bitbake	15
3.4.3	In-field updates	15

Requirements

"Should" have for following the lecture:

- Basic understanding of electrical circuits
- Programming in C

Optional, nice to have:

- Knowing of C++ or other object-oriented language

What's *not* in the lecture

Not part of the lecture

- Programming course
- Assignments, Lab course
- Design of electronic circuits
- Control or systems theory

Introduction into embedded systems

Multimedia example - Analog

Vinyl record player

- Track position - manually with tone arm
- Rotation speed - constant $33\frac{1}{3}$ RPM
- Signal translation
 - Pick-up stylus in grooves on the record
 - Mechanical to electric signal conversion
- Sound tuning - analog RLC filters



Nice explanation by @TechnologyConnections: <https://www.youtube.com/watch?v=3DdUvoc7tJ4>

Image source: 32bitmaschine, Dydric, CC BY-SA 2.5 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Technics_SL-1200MK2-2.jpg

Multimedia example - Digital/Embedded

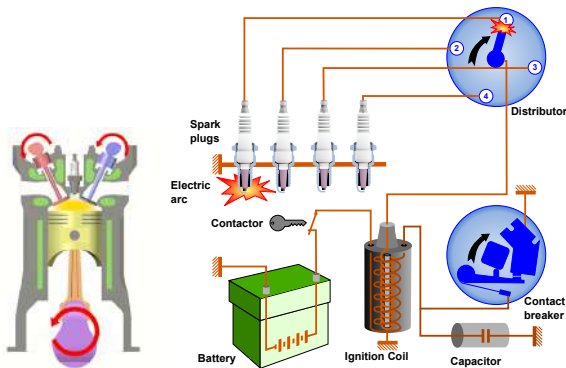
CD player

- Track position - servo motor, TOC in first track
- Rotation speed - controlled by data rate on the disk
inside: ≈ 500 RPM, outer edge: ≈ 200 RPM
- Signal translation
 - Optical sensor, AD converter
 - Error decoding, DA converter
- Sound tuning - digital filters in digital signal processors



Nice explanation by @TechnologyConnections <https://www.youtube.com/watch?v=3yJqLD9RxD4> Image source: Mk2010, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Panasonic_SL-S250C_with_Philips_LC0305_20110106.jpg

Automotive - Distributor ignition system



Goal:

Adjustment of ignition time to piston position and injection time.

Analog world:

- Mechanical adjustment directly connected to crankshaft rotation.

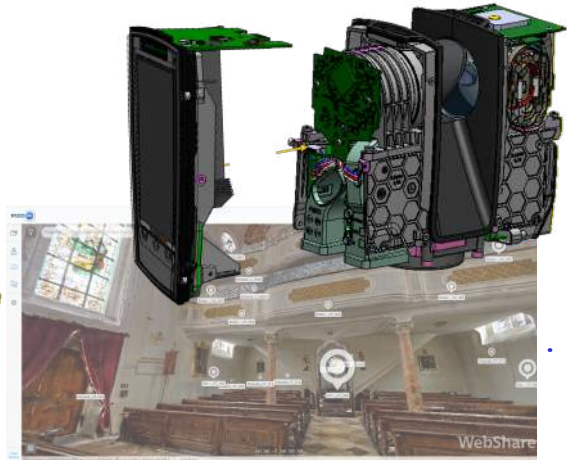
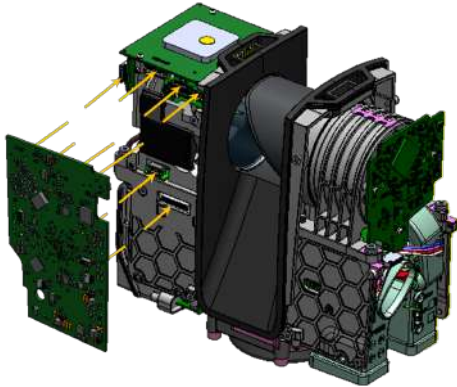
Digital/embedded world:

- Ignition time and injection time both controlled by (re)programmable Engine Control Unit (ECU) based on crankshaft position sensors.

Image sources: Wapcaplet; Frédéric MICHEL and PiRK via Wikimedia Commons - CC BY-SA 3.0
https://commons.wikimedia.org/wiki/File:Four_stroke_cycle_spark.png
https://commons.wikimedia.org/wiki/File:Car_ignition_system.svg
Animation: <https://www.youtube.com/watch?v=FQcX-NvySPk>

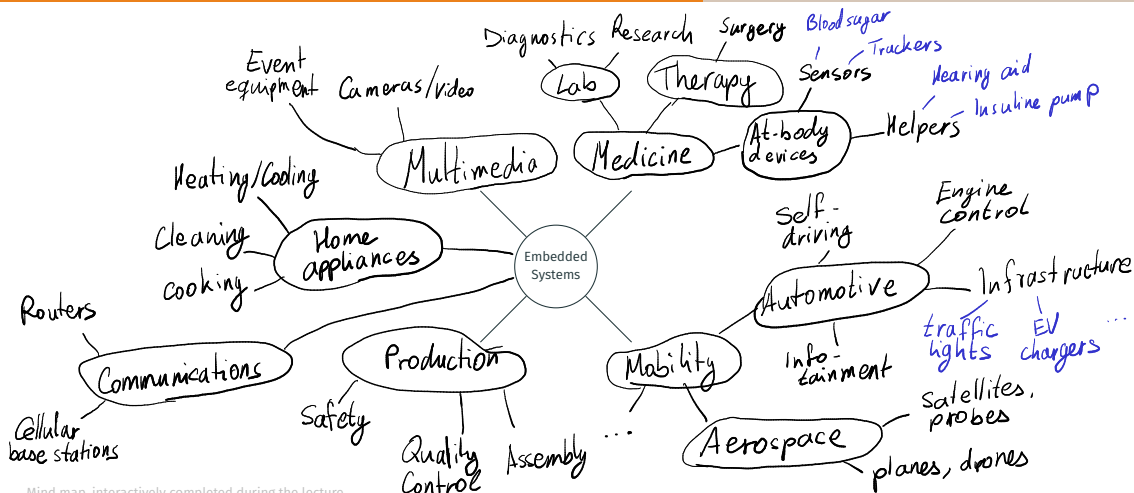
Measurement equipment

FARO®Focus 3D laser scanner



Example scan projects: <https://faro.websharecloud.com/ws-featured/#/>

Use cases for embedded systems



Mind map, interactively completed during the lecture

Example actuators

- Mechanical
 - Motors: DC, AC, stepper, servo
 - Valves, Push-pull solenoids
 -
- Optical
 - LEDs
 - Lasers
 - Displays
- Acoustic
 - Speakers
 - Buzzers
 - Bells (actually mechanical)
- Electrical
 - Relays, Switches
 - Wireless transmitters
 - General: DACs, PWM generators

Example sensors

- Environment

- Temperature
- Pressure
- Humidity
- Light
- Sound

- Position

- Absolute: GPS/GNSS
- Relative: Linear/rotary encoders, compass (magnetometer)
- Height (altimeter via pressure, GPS)

- Motion

- Linear: Accelerometer
- Rotary: Gyroscope

- Optical

- Single pixel: Photodiode, phototransistor
- Arrays (1D, 2D): Light curtains, Cameras

- Electrical

- Field probes
- Touch sensors
- Wireless receivers
- General: Comparators, ADCs

Advantages of embedded systems (1)

- (Partial) elimination of moving parts
 - Less manufacturing variance => constant product quality
 - Improved performance by avoiding inertial effects
 - Less aging effects (wear)
 - => Reduced maintenance efforts and increased longevity
 - Lower energy consumption
 - => improved sustainability and operational expenses
- Reduction in size
 - Improved or even enabled portability
 - Reduced material usage

Advantages of embedded systems (2)

- Usage of programmable software
 - Possibility to improve functionality without replacement of parts
 - Reuse functionality across different product families or generations
 - Shorter development cycles
- Automation: Less to no need of human operators
 - Reduce cost of operation
 - Reduce risk of human errors
 - High-availability autonomous operation is possible
Then, no need for a user interface

Challenges of embedded systems

- Software will have bugs
- Can be vulnerable to cyber threats
- Resource limitations (power supply, memory, CPU speed)
- Real-time requirements
- Maintenance or troubleshooting requires higher-skilled personell.

Non-Functional requirements

Architectural constraints not related to the purpose of the system [2]

- Reliability, availability - System delivers correct results and does not fail
- Energy efficiency - Especially if battery powered
- Size, weight, cost - To be appealing to customers
- Robustness against physical and electrical conditions
- Maintainability - Ease of repair or upgrade after selling a product
- Safety and security - No harm to humans or environment, no harm to the system itself
- Regulatory requirements - Ensure legal compliance (spectrum usage, EMC, safety ...)

Functional requirements

What should the system do?

Very specific to the application, therefore no definitive categorization.

- Input and output interfaces - e.g. ATM: Card reader, keypad, display, cash dispenser
- Expected roles of users - e.g. ATM: Customer, bank employee
- Expected behavior in normal cases and in error cases
 - e.g. ATM: Don't dispense cash without authorization, Don't debit account without dispensing cash
- ...

Hardware and Software

Hardware (HW)

Physical components of the system - copper, silicon, plastic, ...

Software (SW)

Collection of binary data that is executable on a processor.

Operation system, drivers, applications - usually generically replaceable.

Firmware

Firmware (FW)

- Binary data describing runtime behavior of a product.
 - Usually a piece of (embedded) software.
 - FPGA configuration (see later) is replaceable binary data.
Counts as firmware but not as software.
-
- Not as easily replaceable as generic software
Sometimes not replaceable during the whole lifetime of a device (e.g. washing machine).
 - Usually not portable to other devices
 - A device's firmware needs to implement a special mechanism in order to enable updates by the end user.

One last thing



Yearly Embedded Software Engineering Congress in Sindelfingen

<https://www.esk-kongress.de/>

- This year 2024-12-02 to 12-05
- Talks or compact seminars on topics related to embedded software
- Mostly in German, unfortunately. Only some talks are in English
- Somewhat expensive: Student price 350 EUR for 1 day, 1 person
Worth it if you are interested and have some experience.
- Keep it in mind for additional training if you start working in the area.

Also: Embedded World, Nuremberg : Fair/Exhibition
Usually early Spring every year

Thank you for the attention!

Next lecture: Computing architectures
(Microprocessors and microcontrollers - a crash course)

References

- [1] E. White, *Making Embedded Systems*.
O'Reilly, 2012.
- [2] W. Lange, M. Bogdan, and T. Schweizer, *Eingebettete Systeme*.
De Gruyter Oldenbourg, 2015.

Hardware Platforms and Programming of Embedded Systems

Lecture 2 - Computing architectures

Microprocessors and microcontrollers - a crash course

Dr.-Ing. Alexander Knaub

2024-10-24

Institute of telecommunications (INÜ)

Contents

Overview (recap)

Instruction Set Architectures

Processing architecture

Memory and peripherals

Processor packages

CPU and MCU families

Overview (recap)

Purposes of processors/microcontrollers

- Control of processes in a (embedded) system in a programmatic way
- React to inputs
- Generate outputs

Today: Quick introduction/recapitulation of the CPU architecture topic as far as necessary for understanding the interaction between software and hardware. The topic is in-depth covered by lectures of other institutes or books.

CPU block diagram

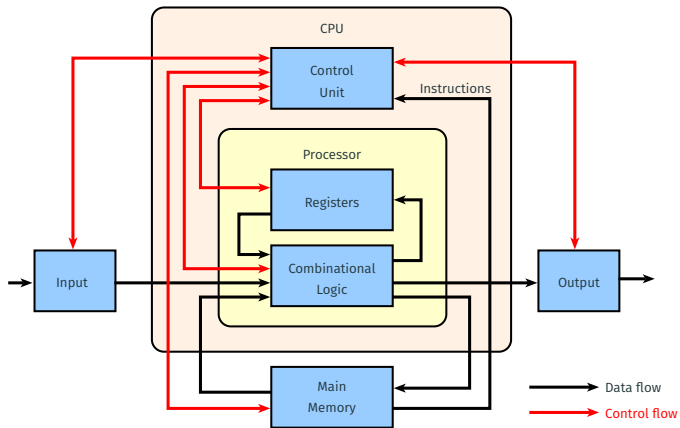


Image source: Adapted from User:Lambtron, CC BY-SA 4.0 via Wikimedia Commons
<https://commons.wikimedia.org/wiki/File:ABasicComputer.svg>

General operation mode

- Read instructions and input data from memory or from periphery
- Perform computation depending on instructions
- Write result to memory or to periphery
For immediate use of results: Registers inside the CPU
- Determine address of next instruction (jump if necessary)
- Repeat

Processor architecture characteristics

- Instruction set - how to program the CPU and what it can do
- Data format - word size, big/little endian,...
- Register layout - what can be stored directly in processor
- Memory access methods - read-write vs. read-only, alignment, caches etc.
- Access to peripherals - Which interfaces are available, how can the software use them

Instruction Set Architectures

Instruction Set Architecture (ISA)

The set of (assembler) instructions available to compilers (or advanced programmers) for a given processor.

The internal realization of a processor's behavior and capabilities is called *microarchitecture*.

Separation of ISA and microarchitecture allows to have a standardized set of instructions, while the processor's internal realization can vary between different models or even manufacturers (Example: x86 ISA supported by Intel and AMD).

The ISA depends itself on other aspects of the processor architecture, but it is the most relevant one from a software engineering point of view.

CISC vs RISC

There are two large classes of ISAs:

Complex Instruction Set Computer (CISC)

One instruction can describe a complex operation which can need multiple clock cycles to complete.

Reduced Instruction Set Computer (RISC)

Each instruction is a simple operation which is completed within one clock cycle.

The names can be misleading, to be read as rather "set of complex instructions", not "complex set of instructions". The variety of instructions in a RISC processor can be large as well.

CISC vs RISC: Example

The image displays a side-by-side comparison of C++ source code and its compiled assembly for two different architectures: x86-64 and ARM64. The interface is Visual Studio Code with the 'COMPILER EXPLORER' extension.

C++ Source Code (Left Panel):

```
1 #include <iostream>
2
3 int twoTimesN(const int n)
4 {
5     int b = 0;
6     for(int i = n; i > 0; i--)
7     {
8         b += 2;
9     }
10    return b;
11 }
12
13 int main()
14 {
15     std::cout
16         << twoTimesN(5)
17         << std::endl;
18 }
```

x86-64 gcc (trunk) (Editor #1) (Middle Panel):

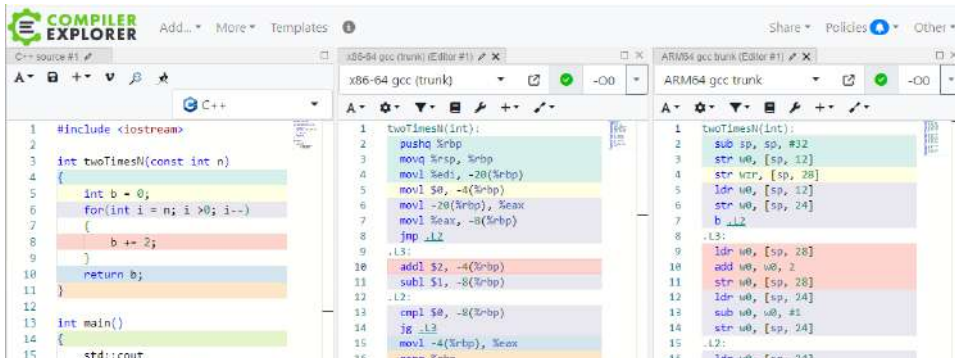
```
1 twoTimesN(int):
2     pushq %rbp
3     movq %rsp, %rbp
4     movl %edi, -20(%rbp)
5     movl $0, -4(%rbp)
6     movl -20(%rbp), %eax
7     movl %eax, -8(%rbp)
8     jnp .L2
9 .L3:
10    addl $2, -4(%rbp)
11    subl $1, -8(%rbp)
12 .L2:
13    cmpl $0, -8(%rbp)
14    jg .L3
15    movl -4(%rbp), %eax
16    popq %rbp
17    ret
18 > main:--
```

ARM64 gcc trunk (Editor #1) (Right Panel):

```
1 twoTimesN(int):
2     sub sp, sp, #32
3     str w0, [sp, 12]
4     str w12, [sp, 28]
5     ldr w0, [sp, 12]
6     str w0, [sp, 24]
7     b .L2
8 .L3:
9     ldr w0, [sp, 28]
10    add w0, w0, 2
11    str w0, [sp, 28]
12    ldr w0, [sp, 24]
13    sub w0, w0, #1
14    str w0, [sp, 24]
15 .L2:
16    ldr w0, [sp, 24]
17    cmp w0, 0
18    bgt .L3
19    ldr w0, [sp, 28]
20    add sp, sp, 32
21    ret
22 > main:--
```

Try it: <https://godbolt.org/z/65bPEPr71>

CISC vs RISC: Example



The screenshot displays the Visual Studio Code interface with three panels. The left panel shows the C++ source code for a program named `twoTimesN`. The middle panel shows the x86-64 assembly output, and the right panel shows the ARM64 assembly output.

```
#include <iostream>

int twoTimesN(const int n)
{
    int b = 0;
    for(int i = n; i > 0; i--)
    {
        b += 2;
    }
    return b;
}

int main()
{
    std::cout
```

x86-64 gcc (trunk) (Editor #1)

```
1  twoTimesN(int):
2      pushq %rbp
3      movq %rsp, %rbp
4      movl $edi, -20(%rbp)
5      movl $0, -4(%rbp)
6      movl -20(%rbp), %eax
7      movl %eax, -8(%rbp)
8      jnp .L2
9      .L3:
10     addl $2, -4(%rbp)
11     subl $1, -8(%rbp)
12     .L2:
13     cmpl $0, -8(%rbp)
14     jg .L3
15     movl -4(%rbp), %eax
```

ARM64 gcc trunk (Editor #1)

```
1  twoTimesN(int):
2      sub sp, sp, #32
3      str w0, [sp, 12]
4      str w2r, [sp, 28]
5      ldr w0, [sp, 12]
6      str w0, [sp, 24]
7      b .L2
8      .L3:
9      ldr w0, [sp, 28]
10     add w0, w0, 2
11     str w0, [sp, 28]
12     ldr w0, [sp, 24]
13     sub w0, w0, #1
14     str w0, [sp, 24]
15     .L2:
16     bne w0, w0, .L3
```

Note how calculated memory addresses are used in arithmetic operations on x86 (-> **addl \$2, -4(%rbp)**), while in ARM64 memory addresses are only used in load/store instructions (-> **ldr w0, [sp, 28]**), and arithmetic instructions only operate on registers (-> **add w0, w0, 2**).

Try it: <https://godbolt.org/z/65bPEPr71>

- For performance reasons, today's processors have internally a RISC (micro-)architecture, even if their ISA offers typically-CISC instructions.
- Such processors (most prominent: x86) contain a translation unit that converts CISC instructions of a program into RISC instructions required by the microarchitecture.
- In this lecture course, knowledge of assembler programming and usage of specific ISAs will *not* be required. When necessary for learning the basics of processor implementations, we will use RISC architectures as examples.

Processing architecture

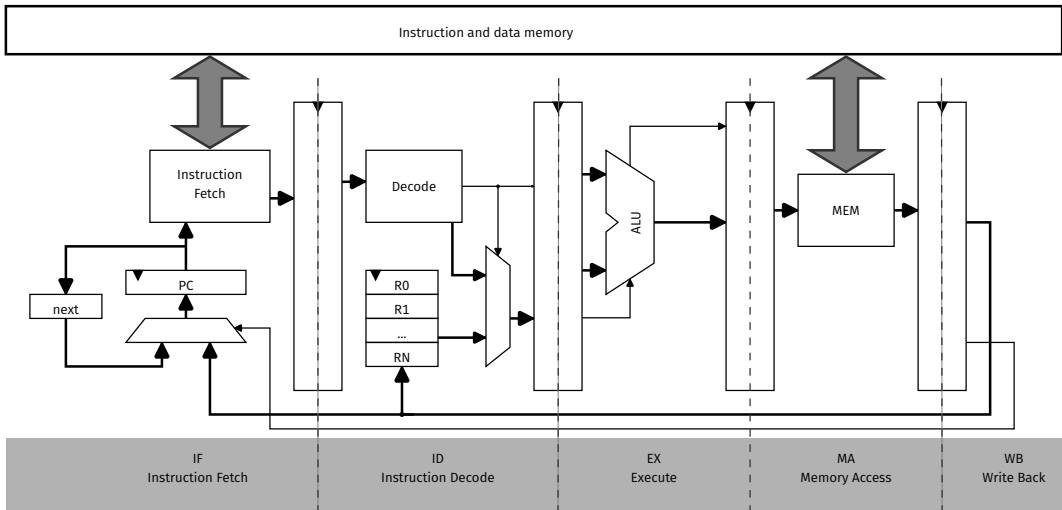
RISC pipeline

- The different steps of the instruction processing require dedicated hardware components. While one instruction is processed in one of the components, other components can be kept busy with previous or next instructions of the program.
- The components are connected in a pipeline, where each component (also called *pipeline stage*) is connected to the next one.



Image source: User:Audriusa, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Pipeline_device.jpg

RISC pipeline

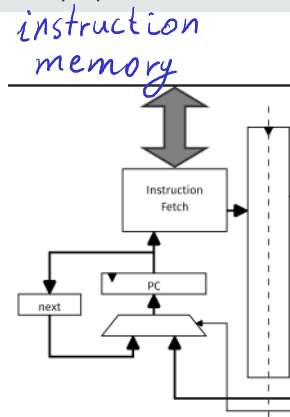


RISC pipeline - stage IF

Instruction Fetch (IF)

Fetches one instruction from memory, using the program counter (PC) as address.

- *Usually:* PC is incremented by the instruction word size in each cycle, e.g. x86-32: 4 bytes for 32-bit instructions, ESP32: 3 bytes for 24-bit instructions
- *Exception:* Branching instructions:
PC is set to the target address of the branch, potentially calculated by the ALU
- Modern CPUs contain caches to speed up memory access.
(More on this in a later lecture)



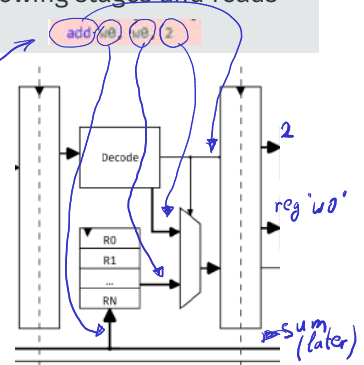
RISC pipeline - stage ID

Instruction Decode (ID)

Depending on the instruction, sets the control signals for following stages and reads the operands from registers if necessary

Registers:

- Small (individual values), fast memory inside the CPU
- Some have special purposes (e.g. stack pointer, status register), On some microcontrollers, special registers are used for accessing the binary in/out ports.
- In RISC, most registers are general purpose (can be used to hold any data)



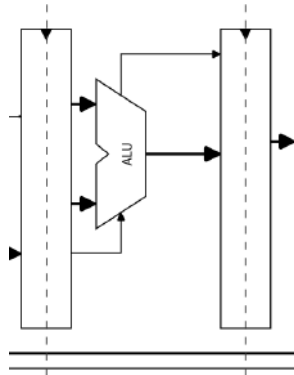
operation: Add
operand 1: content of reg. 'w0'
operand 2: constant '2', part of instruction
store result in reg. 'w0'

RISC pipeline - stage EX

Execute (EX)

Performs the operation specified by the instruction

- Arithmetic Logic Unit (ALU) performs the operation on the operands,
e.g. addition, multiplication, division, bitwise logic operations, etc.
- Special instructions: Memory access (load/store)
ALU calculates the address, memory access in next stage reads/writes the data
- In case of branching instructions, this stage decides whether to branch or not

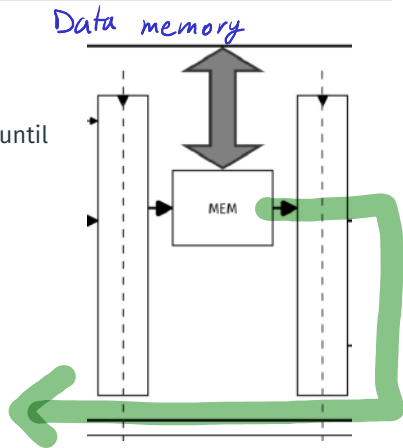


RISC pipeline - stage MEM

Memory access (MEM)

Reads or writes data from/to memory

- After reading, the data is written to a register
- Memory access is usually slow (multiple clock cycles).
 - To ensure consistency, the pipeline is stalled (paused) until the data is available.
 - To improve performance, a cache can be used (fast-accessible local copy of a small memory section).

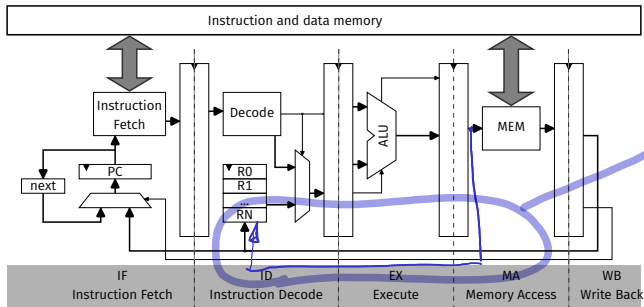


RISC pipeline - stage WB

Write back (WB)

Write the result of the MEM or EX operation to the register specified in the instruction

- If the instruction was an ALU operation without memory interaction, the result can be written already during the MEM clock cycle.



What we did not cover

- Interrupts - Switch of program flow by external events, not by sequentially read instructions
- Branch prediction - Attempt to improve performance by guessing which branch will be taken in conditional instructions
- Forwarding - Avoiding stalls (waiting for write-back of results of previous instructions) by forwarding the result of an ALU operation to the next instruction
- Implementation of caches - Achieving performance improvements while keeping consistency.

These and other aspects of processor architectures are covered in lectures of IKR and ITI.

Memory and peripherals

Basics of digital memory

- Memory is a collection of storage cells, each holding a binary value (0 or 1)
- The cells are grouped into bytes (8 bits) which are numbered (addressed)
- The memory offers an address input, to select which byte to read or write
- Additional control signals are used to select the operation (read or write)
- The bit width of the address signal W_{Addr} determines the maximum number of addressable bytes $N_{\text{Mem}} = 2^{W_{\text{Addr}}}$, e.g.

$$W_{\text{Addr}} = 16 \Rightarrow N_{\text{Mem}} = 2^{16} \text{ B} = 64 \text{ KiB} \quad \text{or} \quad W_{\text{Addr}} = 32 \Rightarrow N_{\text{Mem}} = 2^{32} \text{ B} = 4 \text{ GiB}$$

$$W_{\text{Addr}} = 64 \Rightarrow N_{\text{Mem}} \approx \infty$$

Memory access

- Harvard architecture: Separate memory for instructions and data
 - Higher performance (no interference between instruction and data access)
 - Two separate memory controllers necessary: Potentially more expensive
 - Usually: Instructions are read-only (Flash ROM)
- Von Neumann architecture: Unified memory for instructions and data
 - Instructions can be modified by the program
 - Instructions can be treated as data and vice versa → *dangerous*
 - A soft separation by using different address spaces is possible

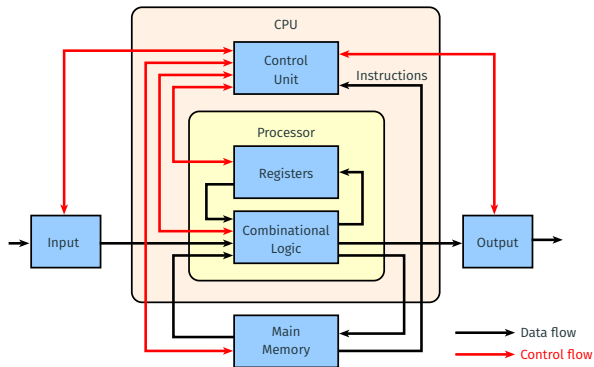


Load/Store architecture

```
// Where to store the operands and the result of an arithmetical operation?  
int c, a = 20, b = 22;  
c = a + b;
```

- Only access memory to transfer data to and from registers
- All ALU instructions can only operate on registers
- Simplifies pipelined instruction processing → common in RISC architectures
- Often combined with a larger number of registers than in CISC to reduce the number of memory accesses which are usually slower.
→ *Memory hierarchy* (more on this in a later lecture)

Access to peripherals

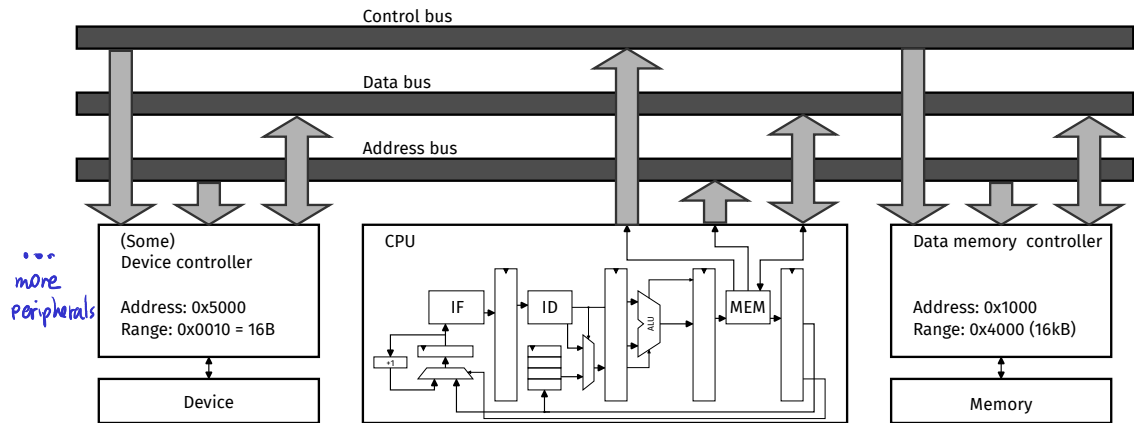


How to access the Input and Output blocks (I/O)?

(Actually: How do the device drivers do this?)

- For simple pin access, a special instruction can be available, like `MOVF PORTB, W;` (PIC assembly for "copy the 8 input pins from port B as 1 byte to the register W")
- For more complex peripherals, a memory-mapped I/O is used, see following slides

Memory-mapped I/O



Memory-mapped I/O

Memory-mapped I/O: Peripherals are on the same address space as the data memory

- Peripherals can be accessed with the same instructions as the data memory
- This technically reduces the available address space for the data memory
- With architectures that use 32-bit or even 64-bit address widths, the address space is large enough to avoid collisions
 - state of the art on most modern RISC architectures

Memory-mapped I/O (2)

- For simple binary input/output, like GPIO pins, the program just reads/writes the corresponding memory address
- For more complex peripherals that require a protocol, like UART, SPI, I2C, etc., the corresponding controller offers control and status registers as well as data registers within its assigned memory range.
- The program can then e.g. in a sequence
 1. read from a *status register* to check if data is available in a controller's receive buffer,
 2. read from the *data register* to actually get the data,
 3. write to a *control register* to trigger sending an acknowledgement on the peripheral interface.

Port-mapped I/O

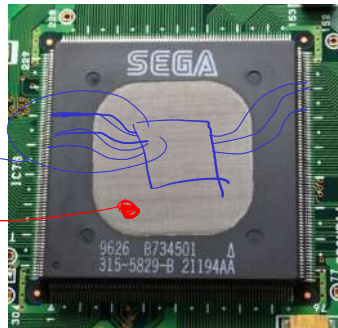
Port-mapped I/O: Similar to Memory-mapped I/O, but peripherals are on a separate address space, require specific instructions to access

- The address space of the data memory is not affected
- Can sometimes be found on architectures with small address widths (8-bit, 16-bit)
- Drawback: More instructions are required to access the peripherals, consuming the instruction space (number of possible instructions with a given instruction word width).

Processor packages

Purposes of processor packages

- Mechanical protection of the integrated circuit (IC)
- Provide electrical connections, a.k.a. bonding to the PCB (printed circuit board)
- Provide a standardized form factor
→ simplifies PCB design
- Often: Dissipate heat
- Sometimes: Provide electromagnetic shielding
(grounded single-piece metal cover)



A processor in a Sega arcade machine, probably "Model 3" (1996)

Image source: M. Pritchard, public domain via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Sega_304-pin_QFP.jpg

DIP: Dual In-line Package

- Can be soldered (through-hole mount) or plugged into a socket
- Standardized pin spacing: 2.54 mm ($\frac{1}{10}$ inch)
- Popular among hobbyists due to easy handling (e.g. can be plugged into a breadboard)



ATmega328P MCU in a 28-pin DIP socket on an Arduino UNO R2 board

Image source: Nick Hubbard, CC BY 2.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Arduino_UNO_unpacked.jpg

SOP: Small Outline Package

- Other names: Small Outline Integrated circuit (SOIC), or just SO
- SMD^a version of DIP - soldered directly onto the surface of a PCB → Easier routing of signals on multi-layer PCBs
- Standardized pin spacing: 1.27 mm (1/20 inch) (half) → higher packing density
- Variants: TSSOP (Thin Shrink), VSOP (Very Small),...
Then, smaller pin spacing, e.g. 0.65 mm for TSSOP

^aSurface Mounted Device



Microchip PIC 16HV615 MCU
(2006) in the 8-pin SOIC package

Image source: Nick Hubbard, CC BY 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Microchip_PIC_16HV615.jpg

QFP: Quad Flat Package

- Quadratic version of SOP with pins on all four sides
→ More pins on the IC without increasing its size
- Standardized pin spacing: 1.27 mm (1/20 inch)
- Variants: TQFP (Thin), LQFP (Low-profile),...
- Related: Pin leads bent under the package - Plastic Leaded Chip Carrier (PLCC)
Or no leads at all - Quad Flat No-leads (QFN)



AMD Am5x86-P75 CPU (1995) in a PQFP package (Plastic QFP)

Image source: Konstantin Lanzet, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:KL_AMD_Am5x86-P75_PQFP.jpg

PGA: Pin Grid Array

- Even more pins per used PCB area by having pins on the bottom of the package in multiple rows
- Used in early desktop CPUs, e.g. Intel and AMD
- High pin count required especially for memory interface (RAM)
- Can be soldered (through-hole mount), but usually plugged into a socket



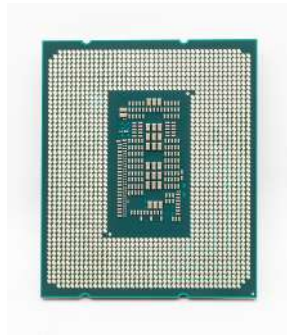
Cyrix (IBM) CPU 6x86MX PR200
(1998) in the PGA package

Image source: Eric Gaba, CC BY-SA 3.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Cyrix_IBM_CPU_6x86MX_PR200_bottom.jpg

LGA: Land Grid Array

- As PGA, but pins are replaced by flat contacts on the bottom of the package
- No pins that can bend or break
→ Improved mechanical stability
- Electrical contact by pressing the contact pads to pins in the socket or by soldering the package to the PCB.



Intel Core i7 12700KF CPU (2022)
in the LGA 1700 package

Image source: Jacek Halicki, CC BY-SA 4.0 via Wikimedia Commons
[https://commons.wikimedia.org/wiki/File:2023_Intel_Core_i7_12700KF_\(4\).jpg](https://commons.wikimedia.org/wiki/File:2023_Intel_Core_i7_12700KF_(4).jpg)

BGA: Ball Grid Array

- As LGA, but contacts are replaced by solder balls
- Always soldered to the PCB. Allows good heat transfer to the PCB.
- Used in Laptop CPUs, GPUs and in high-performance processors for embedded applications, as well as for storage and memory chips.



Altera MAX II EPM240G CPLD (Complex Programmable Logic Device) in a BGA package

Image source: KaiMartin, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Altera-CPGA_EPM240G.jpg

CPU and MCU families

Difference between PC CPU and MCU

Microcontroller Unit (MCU)

- Processor + memory + peripherals
- Lower performance than PC CPU
- Lower power consumption than PC CPU
- Runs a fixed program from memory (Flash ROM)
 - Optimization possible by using Harvard memory-access architecture

CPU¹ of a personal computer (PC)

- Processor only
- Operates at higher clock frequencies than MCU
- Designed to load arbitrary programs at runtime from memory
 - Usually Von Neumann memory-access architecture

¹Central Processing Unit. The processor core in an MCU is also called CPU.

Components usually integrated in an MCU

- CPU and Memory (Flash ROM, RAM)
- Timers - for keeping track of time while the CPU is busy with other tasks
- Peripherals (more on that in a later lecture)
 - GPIO (General Purpose Input/Output)
 - Serial interfaces (UART, SPI, I2C, ...)
 - Analog-to-Digital Converter (ADC)
 - Digital-to-Analog Converter (DAC)
 - PWM (Pulse Width Modulation)
 - Controllers for external storage devices (e.g. SD cards)

Atmel: Manufacturer of various microcontrollers and other embedded-oriented hardware (touchscreens, memory, ...). Acquired by *Microchip* in 2016.

- AVR = Alf and Vegard's RISC processor
 - Wide popularity thanks to Arduino HW platform and development environment
 - Proprietary 8-bit RISC architecture
 - Two big sub-families (differing in number of peripherals, memory sizes, ISA functionality):
tinyAVR - *ATtiny* and megaAVR - *ATmega*
- SAM = Smart ARM-based Microcontroller
 - 32-bit ARM Cortex M and A architectures (more on ARM in later slides)
 - Higher performance than AVR (word width, clock frequency)
- Both C friendly thanks to availability of GCC compiler

- Original microcontroller product of *Microchip Technologies* (long before acquiring Atmel with its AVR family)
- Changing interpretation of the name.
Current: PIC = Programmable Intelligent Computer
- RISC architecture, mainly 8-bit, performance comparable to AVR
- Mainly programmed in assembler.

ARM Cortex (1)

- ARM = Advanced RISC Machine. Most widespread ISA in portable devices .
- MCUs: Cortex-M, Cortex-R, CPUs: Cortex-A
- Not a manufacturer, but a family of 32-bit and 64-bit ISAs:
Company *ARM Limited* designs the ISAs and licenses them to other manufacturers - as predesigned processor core intellectual property (IP) blocks or as permissions to design their own cores based on the ARM ISA.
- Example manufacturers: STMicroelectronics (STM32), NXP (i.MX), Apple (M1, M2), Qualcomm (Snapdragon), etc.

ARM Cortex (2)

- Manufacturers usually connect specific peripherals to the licensed ARM cortex core. Usual for MCUs, but also CPUs can integrate low-level peripherals this way - useful for embedded systems.
- *Microcontrollers* are mainly programmed in C using GCC compiler, *CPUs* are operated with an operating system (usually Linux), which can run programs written in almost any language.
- ARM CPUs are very frequently used in embedded systems due to their low power requirements and integration of peripheral interfaces.
If they integrate memory (not common for a CPU), a Graphics Processing unit GPU and/or radio modem, they are called a System-on-Chip (SoC)

- ISA designed by *Tensilica*, today part of *Cadence Design Systems*
- Proprietary ISA, licensed to other manufacturers who can extend it by own operations/instructions
- C/C++ friendly thanks to availability of GCC compiler and debugging tools
- Used especially in the ESP8266 and ESP32 microcontrollers by Espressif
 - These MCUs integrate wireless connectivity and are very popular in IoT applications and in hobbyist projects
 - Known to some of you from German Electrical Engineering Bachelor study course
 - Compatible with Arduino IDE and libraries
 - Note: Most modern ESP32-C-Series MCUs feature a RISC-V core and not Xtensa

- Open-source ISA, applied in various microcontrollers and CPUs
- Designed by UC Berkeley to be modular and extensible
- Currently not very widespread, but gaining popularity
- Applied by SiFive, Western Digital, Nvidia, AMD, Microchip, NXP, STMicroelectronics, ...
Recent announcement: Qualcomm works on a RISC-V based *Snapdragon Wear SoC*
- C/C++ friendly thanks to availability of GCC compiler and debugging tools

x86 → 32 bit

x86-64 → 64 bit

- CISC ISA, used mainly in desktop and laptop processors by Intel and AMD
- Usually don't contain integrated peripheral controllers
- Less common in embedded systems.
Mainly when high performance is required and when peripherals are connected by high-level interfaces (PCI(e), USB, Ethernet, ...)

Thank you for the attention!

Next lecture: Programmable Hardware
(FPGAs, CPLDs, ASICs etc.)

Hardware Platforms and Programming of Embedded Systems

Lecture 3 - Programmable Hardware

FPGAs, CPLDs, ASICs etc.

Dr.-Ing. Alexander Knaub

2024-10-24

Institute of telecommunications (INÜ)

Contents

Basics of digital hardware engineering

FPGA technology basics

Applications and variants

Programming

- FPGA Design process

- IP cores

Basics of digital hardware engineering

Processing in software: Sequence of commands

Implementation in software executed on a CPU (example: $S = A + B$):

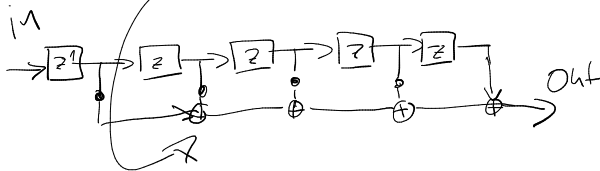
Try it out: <https://godbolt.org/z/nbGv8haxv>

High-level language (C):

```
int add(int a, int b)
{
    return a + b;
}
```

Compiled to assembly language (ARM64):

```
add(int, int):
    sub sp, sp, #16
    str w0, [sp, 12]
    str w1, [sp, 8]
    ldr w1, [sp, 12]
    ldr w0, [sp, 8]
    add w0, w1, w0
    add sp, sp, 16
    ret
```



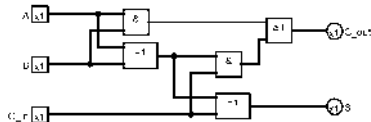
We see: a sequence of commands is executed, including memory accesses, which can consume unpredictable amount of time to complete.

Can we get better (faster, more reliable) with dedicated hardware?

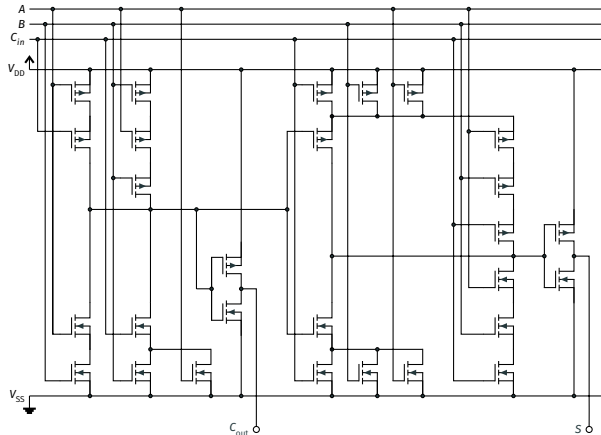
Processing in hardware: Digital logic circuits

Simple example: Adding two one-bit numbers (full adder)

Binary logic gates
(AND, OR, XOR, NOT):

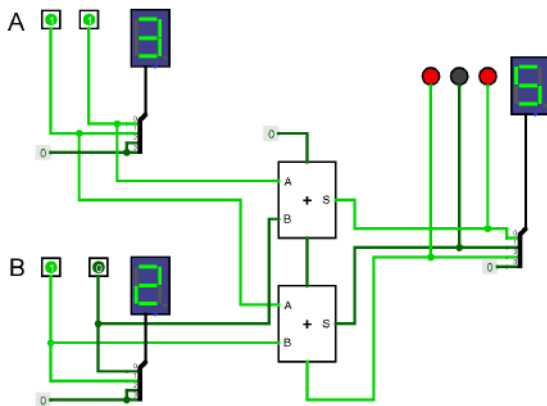


Possible realization as a CMOS
transistor circuit:



Increase complexity

From simple to complex: A two-bit ripple-carry adder build from two full adders

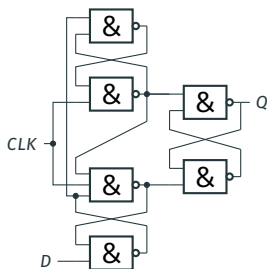


...and so on.

Memory cells

- Processing often depends also on previous inputs
 - Example: Counters (next = previous +1)
⇒ Storing the state of a system is required
- One-bit memory cell: “flip-flop” (FF), “bistable multivibrator”
 - Implementation: Binary logic with a closed feedback loop
 - Can be clocked: The input is taken to the output only on a rising edge of a periodic signal
(useful for synchronizing the outputs of logic circuits)
 - Can have a reset input and an input to enable/disable “memorizing” the output

⇒ Allows to build *sequential logic circuits*



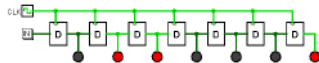
D-Flip-Flop: Assigns $Q = D$ on rising edge of CLK

Registers

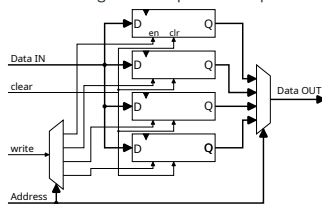
Register

Array of multiple clocked one-bit memory cells that are all read or written together.

- Usually registers hold a data word:
a number, bunch of status bits for a component, etc.
- Array of registers with “enable” inputs driven by a logic circuit for selecting/addressing one of them
⇒ *RAM*
- Array of registers with constant contents and a logic circuit selecting one of the outputs ⇒ *ROM*



Chase light circuit:
8-bit shift register with parallel output

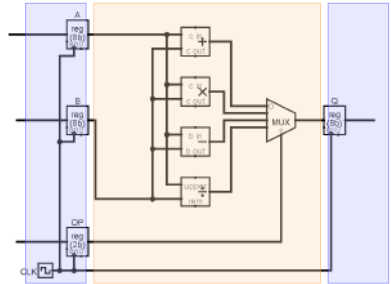


RAM with 4 registers, 2-bit address

Register Transfer Level (RTL)

Commonly used level of abstraction when describing digital hardware: RTL

- Registers, synchronized to a clock
- More or less complex combinational logic circuits between them
- The combinational logic provides well-defined outputs for all possible input combinations – after a propagation delay, depending on the complexity of the combinational logic and on the inputs.
- The propagation delay limits the clock frequency at which the registers can operate.



Digital circuit properties

Pros

- The delay from input to output is as small as transistor switching times and signal propagation delays. $\Rightarrow$ Very fast processing is possible.
- If implemented as integrated circuit (IC), low power consumption and small form factors are possible.

Cons

- Design process is complex and error-prone
- Manufacturing an Application-Specific Integrated Circuit (ASIC) is highly expensive.
- The designed circuit is only suitable for one purpose.

Describing a system by a truth table

Truth table (example: 2-bit adder) deterministically maps inputs to outputs

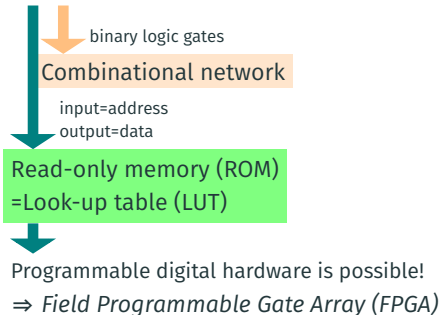
N	B	A	B_1	B_0	A_1	A_0	S_2	S_1	S_0	S
0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	1	0	0	1	1
2	0	2	0	0	1	0	0	1	0	2
3	0	3	0	0	1	1	0	1	1	3
4	1	0	0	1	0	0	0	0	1	1
5	1	1	0	1	0	1	0	1	0	2
6	1	2	0	1	1	0	0	1	1	3
7	1	3	0	1	1	1	1	0	0	4
8	2	0	1	0	0	0	0	1	0	2
9	2	1	1	0	0	1	0	1	1	3
10	2	2	1	0	1	0	1	0	0	4
11	2	3	1	0	1	1	1	0	1	5
12	3	0	1	1	0	0	0	1	1	3
13	3	1	1	1	0	1	1	0	0	4
14	3	2	1	1	1	0	1	0	1	5
15	3	3	1	1	1	1	1	1	0	6

Note: The design of sequential and combinational networks is covered in much more details in lectures of IKR and ITI.

Describing a system by a truth table

Truth table (example: 2-bit adder) deterministically maps inputs to outputs

N	B	A	B_1	B_0	A_1	A_0	S_2	S_1	S_0	S
0	0	0	0	0	0	0	0	0	0	0
1	0	1	0	0	0	1	0	0	1	1
2	0	2	0	0	1	0	0	1	0	2
3	0	3	0	0	1	1	0	1	1	3
4	1	0	0	1	0	0	0	0	1	1
5	1	1	0	1	0	1	0	1	0	2
6	1	2	0	1	1	0	0	1	1	3
7	1	3	0	1	1	1	1	0	0	4
8	2	0	1	0	0	0	0	1	0	2
9	2	1	1	0	0	1	0	1	1	3
10	2	2	1	0	1	0	1	0	0	4
11	2	3	1	0	1	1	1	0	1	5
12	3	0	1	1	0	0	0	1	1	3
13	3	1	1	1	0	1	1	0	0	4
14	3	2	1	1	1	0	1	0	1	5
15	3	3	1	1	1	1	1	1	0	6



Note: The design of sequential and combinational networks is covered in much more details in lectures of IKR and ITI.

FPGA technology basics

Basic building block: Programmable logic cell

- Contains hardware for implementing a small logic circuit: LUT, flip-flops, multiplexers, optional: additional logic
- Programmable by changing the LUT contents and routing within the logic cell
- The functionality varies among different manufacturers and product families
- Different names for the same thing:
 - Altera/Intel: Logic element (LE) or Adaptive Logic Module (ALM)
 - Xilinx: Configurable Logic Block (CLB)

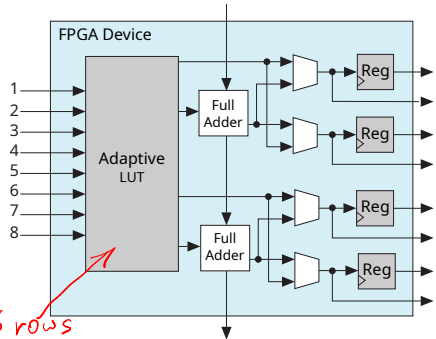
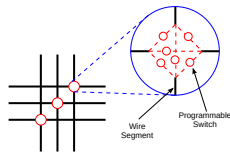


Image Source: <https://www.intel.com/content/www/us/en/docs/programmable/683694/current/adaptive-logic-module.html>

Making it a full-featured device

More electronics is needed for describing *any* digital system:

- The logic cells (thousand of them) are joined by programmable interconnects
- Dedicated routing for clock signals to ensure synchronicity of registers
- Dedicated hardware blocks for input/output ports, often with high-speed transceivers (Gbit/s)



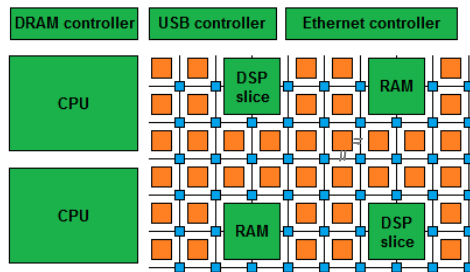
Switch box, a programmable routing element

Image source: User:Ignaciomella and User:Stannered, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Switch_box.svg

Making it a full-featured device

Optional but usually also present on today's FPGAs:

- Block RAM for efficient storing of data
- Digital signal processing (DSP) blocks (multiplier + adder + register)
- Phase-locked loops (PLLs) for generating clock signals
- Currently popular: Hard processor cores
⇒ System-on-Chip (SoC) variants



Modern FPGA: lots of **hard**, not-field-programmable gates

Image source: Yossi Kreinin via [1]

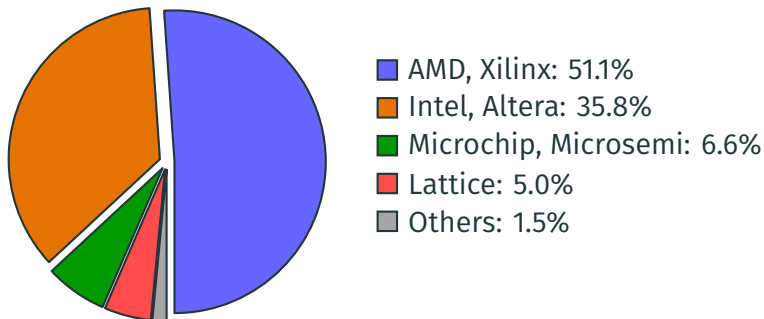
Applications and variants

Variants of FPGAs

- Vendors offer a wide range of products
- Configuration of the FPGA
 - *SRAM-based FPGAs*: The configuration is stored in volatile RAM cells and must be loaded from an external storage at power-up.
 - *Flash-based FPGAs*: The configuration is stored in persistent flash memory.
- Presence of Digital Signal Processing (DSP) blocks
 - Efficient processing of digital signals in filter designs
 - May be able to perform floating-point operations
- Some FPGAs contain dedicated processor cores, also called Hard Processor System (HPS)
- Different I/O capabilities, sizes and speeds

Market share by manufacturers

Total Market in 2018: 5.7bn \$



Source: <https://www.nextpcb.com/blog/top-10-fpga-companies-in-the-world>

Range of FPGA sizes

Example selection of different Intel FPGA devices

	Cyclone10 CL006	CycloneV 5CEA7	Arria10 SX160	Stratix10 SX5500²
Logic elements ¹	6000	150000	160000	5510000
Block RAM [Mbit]	0.270	6.86	8.8	137
DSP blocks	15	156	156	1980
I/O pins	176	480	288	1640
HPS	N/A	N/A	Dual ARM Cortex-A9	Quad ARM Cortex-A53
Price [€]	≈ 10	≈ 250	≈ 920	>30000

¹The number of logic elements is not easily comparable among different vendors because they use different implementations of a logic element.

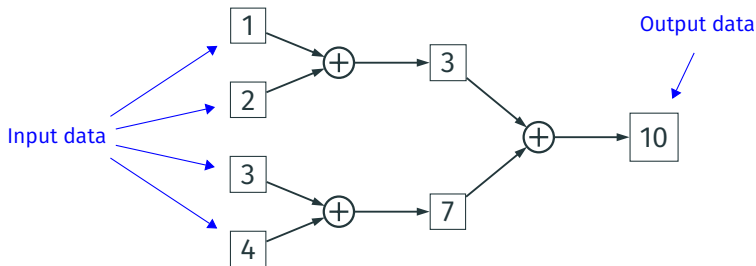
²The main application of high-end FPGAs is usually not in embedded systems, but rather as accelerators for complex computations (e.g. "mining" of cryptocurrencies) in desktop or server computers.

Application areas

- Transforming systems (Based on data flow models)
- Reactive systems (Based on control flow models)
- Reactive transforming systems
(Combination of reactive and transforming systems)

Transforming systems

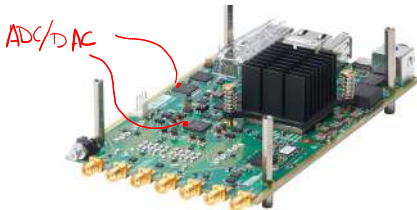
- Dominated by data flow, Represented by data flow graphs
- Work on data streams
- Well suited to be implemented on an FPGA because of parallelized and pipelined processing



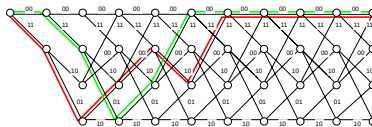
Transforming systems (2)

Examples

- Audio or video processing
- Signal processing in communications:
 - Modulation/demodulation,
 - encoding/decoding
 - encryption/decryption
 - equalization
- Often found in measurement equipment
- Nowadays also neuronal networks



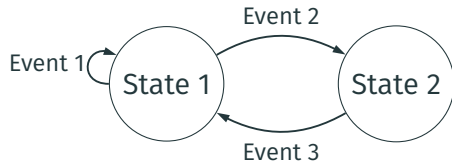
Software Defined Radio (SDR) USRP E320 with a Xilinx Zynq-7045 SoC FPGA
<https://www.ettus.com/all-products/usrp-e320-board/>



Trellis diagram - Maximum likelihood decoding of a convolutional code
Image source: User:Unomano, CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Trellis_Diagram.svg

Reactive Systems (1)

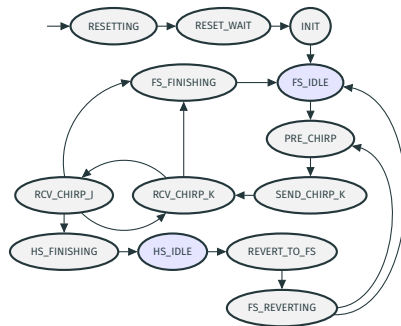
- Dominated by control flow
- Represented by state machines (FSM)
- Usually implemented in software
E.g. Soft-/Hard-Processor, Microcontroller, etc.
- Implemented in FPGA if:
 - Very tight timing bounds are required (real-time)
 - As part of a reactive transforming system
 - Low power required



Reactive Systems (2)

Examples

- Communication protocols
- Real-time critical (PID) control systems
- If part of a reactive transforming system:
e.g. control of data processing parameters
or for e.g. pausing the dataflow if no data
is available



State transitions during USB speed negotiation, see also https://www.usbmadesimple.co.uk/ums_6.htm

Contradictions

FPGAs are not well suitable for use cases with these requirements

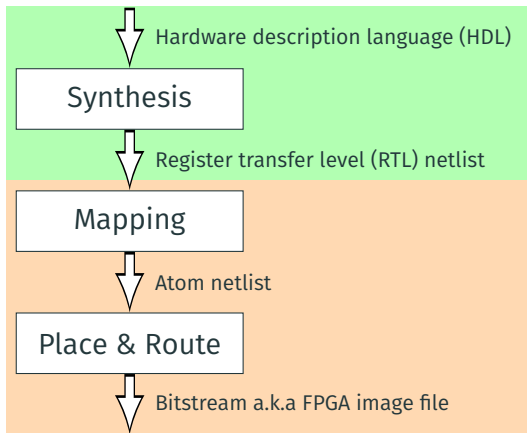
- Large buffers (in the range of megabytes)
- Complex user interactions:
graphical user interfaces (GUIs), displays, keyboards, etc.
- Large memory access,
especially difficult with random (unpredictable) memory access
- File I/O operations

Other variants of customized digital hardware

- Application-specific integrated circuits (ASICs)
 - Can be designed with a hardware description language like FPGAs
 - Not reconfigurable but manufactured for a specific purpose in a semiconductor fab
 - Very high cost
- Application-specific standard products (ASSPs)
 - Similar to ASICs but manufactured in large quantities for different customers
 - Potentially lower cost
- Complex programmable logic devices (CPLDs)
 - Similar to FPGAs but with a different (simpler) architecture
 - Non-volatile configuration memory

Programming

FPGA build flow



Technology agnostic

Vendor and device specific

Hardware description languages

- Abstract description of the structure and behavior of digital integrated circuits for
 - Simulation and Verification
 - Synthesis and Design
- Most popular languages
 - VHDL (Very high speed integrated circuit Hardware Description Language)
 - More popular in Europe
 - Strongly typed
 - Verilog
 - More popular in the USA
 - Weakly typed
 - SystemVerilog: Extension of Verilog with verification features on a higher abstraction level
 - No programming languages in the classical sense

Today's lecture: focus on VHDL as an example (lecturer's preference ;))

VHDL properties

- Strong modularization
 - Modules can only be accessed via their *ports*, interconnected by *signals* carrying values of basic types (essentially bit vectors)
 - Modules are thus loosely coupled
- Highly hierarchical
- Everything happens immediately¹
- Everything happens simultaneously²

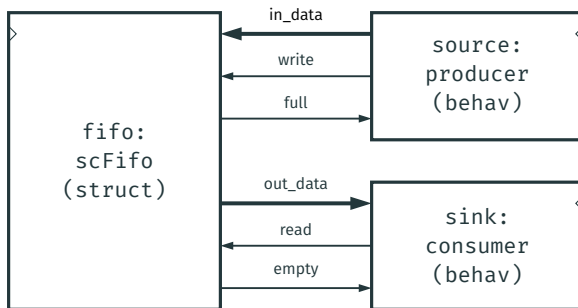
A comprehensive guide to VHDL is the book "The Designer's Guide to VHDL" by Peter J. Ashenden [2].

¹Unless there is a wait statement (simulation only)

²Except for statements in process bodies

VHDL entity

- Basic construct to describe *structure*
- Describes a module and consists of
 - An interface definition
 - One or multiple implementations called architecture
- Entities are instantiated and connected through signals



VHDL process

- Basic construct to describe *behavior*
- Sequential assignments are only allowed inside processes
- All processes run simultaneously
- A process consists of
 - sensitivity list
 - optional process-internal declarations
 - process body (sequential statements)
- Communicate with other processes through signals

VHDL process flow

1. Wait until any signal in the sensitivity list changes
2. Process sequential assignments:
 - 2.1 Process 1st assignment and schedule its result
 - 2.2 Process 2nd assignment and schedule its result
 - 2.3 ...
3. Assign new values to signals
4. Go to 1.

-- Does not what you might think

example : **process** (b)

begin

 a <= b;

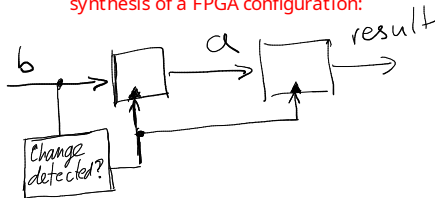
 result <= a;

end process example;

The code snippet has no real sense, and you will probably never find something like this in productive designs.

Here, it acts as example of surprising effects that are possible with VHDL

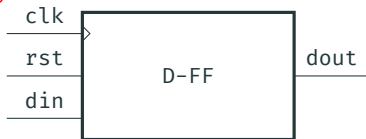
The code could be translated to a delay chain like this if used for synthesis of a FPGA configuration:



Example: D Flip-Flop

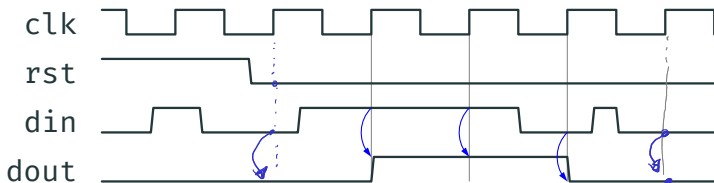
```
dff : process(clk, rst) is
  -- no declarations here
begin
  if rst = '1' then
    dout <= '0';
  elsif rising_edge(clk) then
    dout <= din;
  end if;
end process dff;
```

not all
signals listed



no "else"
branch

memory



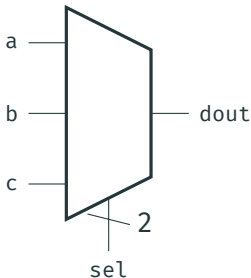
Example: Multiplexer, core logic

```
mux: process (a, b, c, sel) is
begin -- process mux
  case sel is
  when "00" =>
    dout <= a;
  when "01" =>
    dout <= b;
  when "10" =>
    dout <= c;
  when others =>
    dout <= '0';
  end case;
end process mux;
```

all relevant
inputs listed

} → "else"
(default
behavior)

combinational
circuit



Example: Multiplexer, entire entity

```
entity multiplexer is
port (
    a    : in  std_logic;
    b    : in  std_logic;
    c    : in  std_logic;
    sel   : in  std_logic_vector(1 downto 0);
    dout  : out std_logic
);
end entity multiplexer;

architecture rtl of multiplexer is
begin
    mux : process (all) is
    begin -- process mux
        case sel is
            when "00" => dout <= a;
            when "01" => dout <= b;
            when "10" => dout <= c;
            when others => dout <= '0';
        end case;
    end process mux;
end architecture rtl;
```

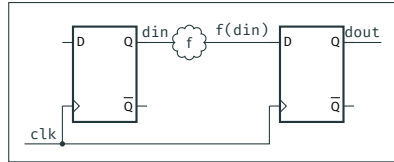
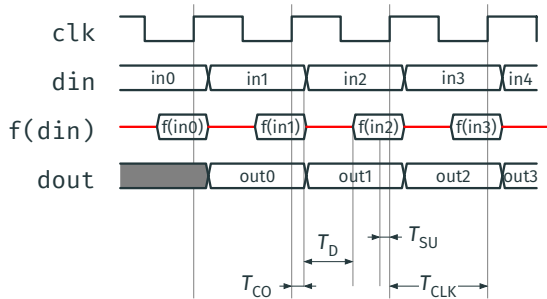


→ stopped here 24.10.24

FPGA design considerations

- Binary signals need to be stable when they are sampled - both, on the peripheral pins of the FPGA and internally between components
 - ⇒ All outputs should be driven by a clocked register
 - ⇒ Registered outputs change only during short time around the clock edge.
- The inputs of the registers must be stable at the clock edge (setup and hold times), also: The combinational circuits driving the register inputs have a delay.
 - ⇒ The signals at the input of combinational circuits must be stable earlier (previous clock edge)
 - ⇒ The input signals of combinational circuits must be driven by a clocked register
- ⇒ **Think in RTL:**
Registers provide data, combinational circuits process it, registers store the result before it is processed by other combinational circuits.

Timing considerations



T_{CLK} : clock period

T_{CO} : clock-to-output delay of a register

T_{SU} : set-up requirement of a register
(data stable before clock edge)

T_D : delay of combinational logic and routing

$$T_D + T_{CO} + T_{SU} \leq T_{CLK} = 1/f_{CLK}$$

T_{CO} and T_{SU} are fixed by the technology and known to synthesis tools

T_D represents the complexity of a sub-network between two registers, and it is the main factor limiting the maximum clock frequency. $\Rightarrow$ Timing analysis is part of design process.

Design verification: Challenges

- Goal: Verify that the design behaves as intended
- Challenges in digital hardware design:
 - On the hardware, the design behaves like a black box:
Unlike in software, where logging and/or debugging is possible, only the inputs and outputs can be observed in hardware.
 - Re-synthesis after rework before testing can slow down the development process
 - The hardware might not yet be available for real-world testing

Design verification: Solutions

- One of tools for FPGA design verification: *Simulation*

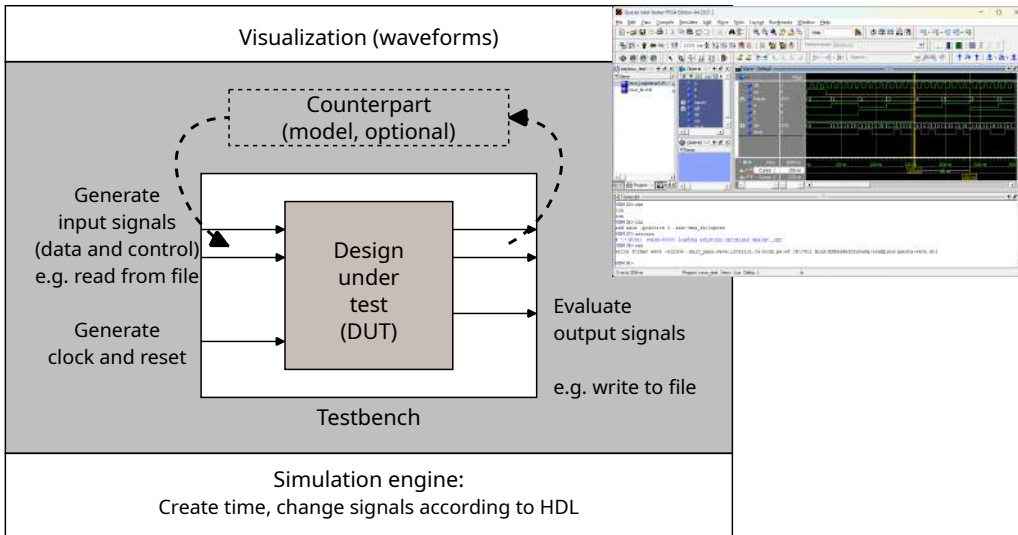
Benefits:

- No need for hardware, can be done on a PC or in the cloud
- Very detailed observation of the internal signals is possible
- Verification of submodules is easier than in real hardware
- Faster development cycles are possible:
No need to re-synthesize the whole design after every change.

- Limitations:

- Simulation is slower than real-time
- Side-effects of the hardware environment can not be simulated (e.g. power supply, temperature, electromagnetic interference)
- Special care is required regarding accuracy of the simulation model for the simulation results to be reliable.

Design verification: Simulation



Design verification: Simulation

- Simulation model is built around the design/device under test (DUT)
- The model (testbench) provides the inputs to the DUT and observes its outputs
- The testbench is written in a HDL language (VHDL, Verilog, ...)
- Additional software can be used for the following purposes:
 - Generate test data for the input, maybe generate expected results
 - Evaluate the collected output data (e.g. compare with expected results)
 - Automate the simulation process (e.g. run multiple test cases with different settings)

- Simulation, verification:
 - Modelsim, Questasim (Commercial, Mentor Graphics, belongs to Siemens)
 - GHDL (open source)
 - isim (Commercial, by Xilinx)
 - VUnit (open source testing framework)
Uses Modelsim or GHDL as backend, allows validating of components by Python scripts
- Synthesis, place & route, timing analysis:
 - Quartus (Intel, Altera)
 - Vivado (Xilinx)
 - Synplify (Synopsys)

IP cores

- Intellectual property (IP) cores are pre-designed and pre-verified modules
- Provided by the FPGA vendor or third-party vendors.
- Can be used as building blocks for larger designs (see next slides), thus saving development effort and ideally also improving quality and efficiency.
- The only way to utilize special HW features of an FPGA
- Can be shipped in different variants
 - As encrypted netlists (black boxes), then often with simulation models
 - As source code (VHDL, Verilog, SystemVerilog)
- Can usually be adjusted to project requirements by parametrization options.

IP cores - Mathematical computations

- Digital filters (FIR, IIR)
- Transforms
 - Fast Fourier Transform (FFT)
 - COordinate Rotation Digital Computer (CORDIC)
e.g. magnitude and phase of a complex number, or trigonometric functions
- Encryption/decryption operations

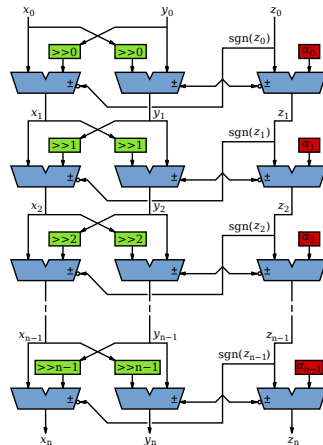


Image source: User:Inductiveload, public domain via Wikimedia Commons

[https://commons.wikimedia.org/wiki/File:CORDIC_\(Bit-Parallel,_Unrolled,_Circular_Rotation\).svg](https://commons.wikimedia.org/wiki/File:CORDIC_(Bit-Parallel,_Unrolled,_Circular_Rotation).svg)

IP cores - On-Chip Memory

Built either from registers within the logic elements (distributed RAM) or from dedicated regions (block RAM).

- RAM (Random Access Memory) - for storing and retrieving data at runtime
e.g. line buffers in image processing
 - Single-port RAM: The same address for reading and writing
 - Dual-port RAM: Separate addresses for reading and writing
Simultaneous reading and writing possible
- ROM (Read-Only Memory) - for retrieving data known at synthesis time
e.g. lookup tables for complex mathematical functions (sin, cos, ...)

IP cores - On-Chip Memory (2)

- FIFO (First In, First Out) - for buffering data streams

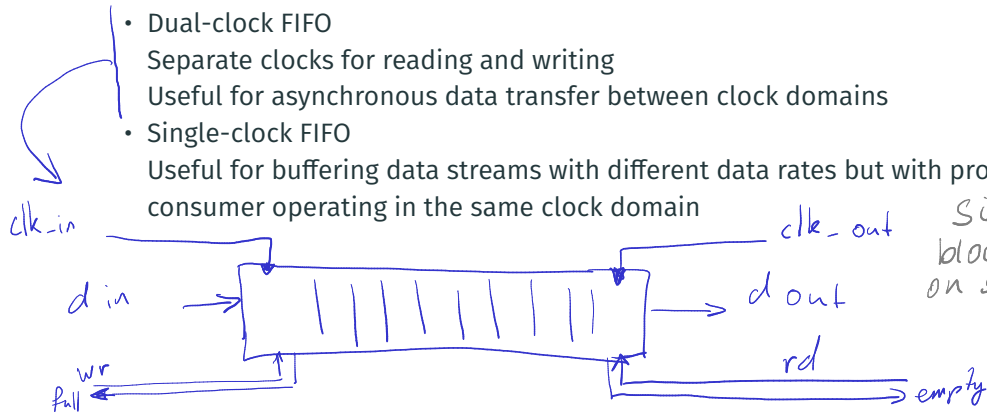
- Dual-clock FIFO

Separate clocks for reading and writing

Useful for asynchronous data transfer between clock domains

- Single-clock FIFO

Useful for buffering data streams with different data rates but with producer & consumer operating in the same clock domain



*similar to
block diagram
on slide # 27*

IP cores - Peripheral controllers

- Controllers for standard interfaces
- Examples:
 - Peripheral device interfaces
e.g. UART, SPI, I2C, Ethernet, USB, CAN, ...
 - Memory interfaces, e.g. DDR, Flash memory, ...
 - Audio and video interfaces
- Often available as a dedicated HW resource $\Rightarrow$ reduced logic cell usage and improved performance

IP cores - Microprocessors

- Complete microprocessor systems built from logic cells ("soft core") as opposed to "hard core" if a dedicated processor is placed on one chip with the FPGA
- FPGA components can be accessed by software via memory-mapped I/O
⇒ Faster/easier than accessing them from a separate CPU/MCU device
- Usual tasks of a processor core:
User interfaces, communication on higher-level interfaces, memory-intensive tasks, etc.
- Can use Block RAM IP resources for instruction and data memory
- Examples:
 - Nios II & NIOS V by Intel
 - Microblaze by Xilinx
 - MIV Risc V, e.g. in Microsemi FPGAs
 - Cortex M by ARM

RISC-V architecture

Intel's
Proprietary
RISC
architecture

The end

Thank you for the attention!

Next lecture: Memory and Storage

References

Bibliography

- [1] EmbeddedRelated.com, *How FPGAs work, and why you'll buy one*, Jun 2013.
- [2] P. J. Ashenden, *The Designer's Guide to VHDL, Volume 3, Third Edition (Systems on Silicon) (Systems on Silicon)*.
San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 3 ed., 2008.

Hardware Platforms and Programming of Embedded Systems

Lecture 4 - Memory and Storage

Dr.-Ing. Alexander Knaub

2024-10-30

Institute of telecommunications (INÜ)

Contents

Memory hierarchy

Volatile memory technologies

Persistent storage technologies

- Flash-based mass storage

Direct memory access (DMA)

Memory hierarchy

Definition

Memory is a component of a computing system, operated by a processing unit, that can store information to be accessed at a later time.

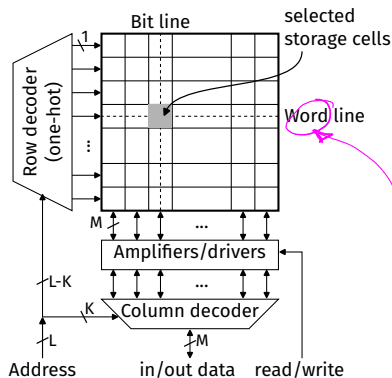
- Can be integrated into the processing unit or connected externally
- Can be volatile or persistent with respect to power loss
- For most operations, random access is required
 - After writing multiple pieces of data, reading can be in a different order.
 - As opposed to sequential access where reading order is defined by the order of writing (FIFO queue, LIFO stack etc.)
 - Data is accessed by an address to avoid ambiguity.
The smallest addressable unit is called a *word*.

Organization, addressing

Defined at interface of the memory (outside)

Usually this is meant, when the term "word" is used

- The address of a word in a memory is a L -bit number, allowing to access 2^L words of M bits each.
- The words are arranged in 2^K columns and 2^{L-K} rows.
- The address is split into a row address of $L - K$ bits and a column address of K bits.
- Row selection connects all one-bit cells in a row (one page) to their bit lines (BL)¹.
⇒ Efficient read/write of 2^K word bursts is possible.
- Amplifier circuits determine whether a word is read or written.



¹Internally, memory has no grouping of bits. The row selection signal is called word line (WL). Don't confuse this with M -bit wide data words selected by the column address outside the matrix.

defined by matrix representation (inside)

NOT the same

Example

Raspberry Pi 4B with e.g. a 4 GiB ($4 \times 8 = 32$ Gbit) LPDDR4 SDRAM [1].
Memory device: Micron MT53D1024M32D4 (labeled "D9WHV")

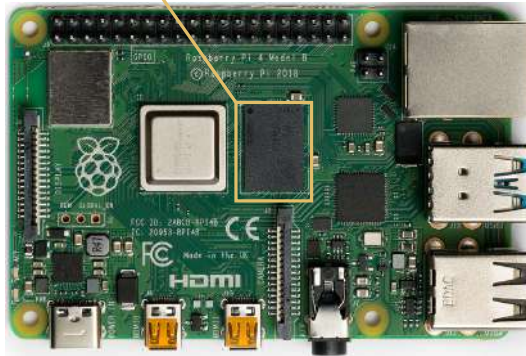


Image source: User:Laserlicht, CC BY-SA 4.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Raspberry_Pi_4_Model_B_-_Top.jpg

Example

Memory device: Micron MT53D**1024M32**D4. From the datasheet (pages 20, 23, 37, 73) [2]:

The device is internally configured with x16 I/O, 8-banks. Each of the x16's 1,073,741,824-bit banks is organized as 65,536 rows by 1024 columns by 16 bits.

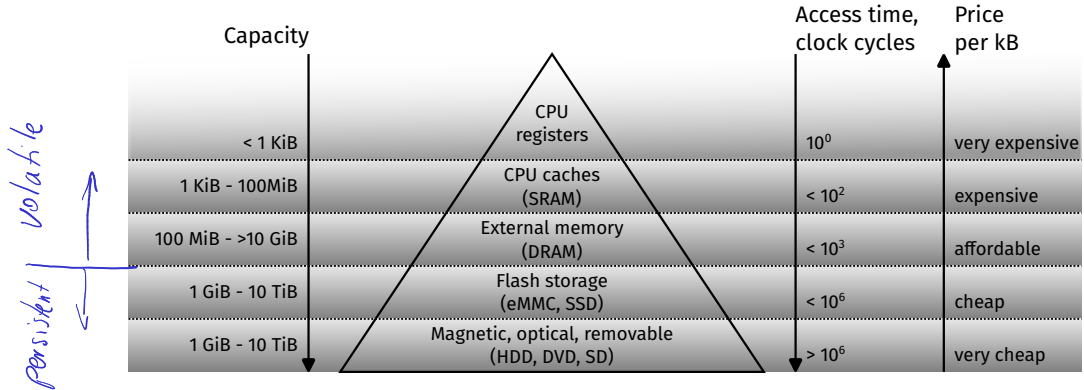
- $M = 16$, 1024 columns $\Rightarrow K = 10$, 65,536 rows $\Rightarrow L - K = 16 \Rightarrow L = 26$
 $\Rightarrow$ 1 Gbit is accessible within one memory bank. $2^{26} \cdot 16$

- The variant 1024M32 (32 Gbit) contains 4 dies in one package, each with 8 banks.
- For addressing, additional 3 bit are required to select a bank.
Also, each of the 4 dies has its own chip-select pin on the package.
- There are no 29 pins for bank/row/column selection though: The addresses are specified by sequentially transmitting up to 8 words over a 6-pin command interface.

$\rightarrow$ see also interface definition example on slide # 11

Memory hierarchy

It is desired but impossible to have unlimited and fast memory.
As a compromise, a hierarchy of different memory types is used.

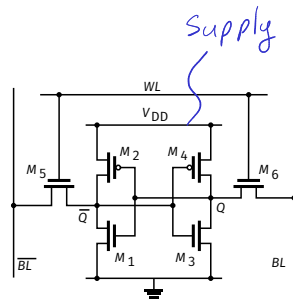


Volatile memory technologies

SRAM

Static RAM - Based on feedback loops (a.k.a. flip-flops).

- Expensive due to low density - 6 transistors per bit
- In the basic form, asynchronous (combinational). Synchronous variants exist (SSRAM)
⇒ higher clock frequencies by pipelined structure *mainly*
- Fast: Access time of a cell is limited by the capacitance of the bit line
- Used in low-density high-speed scenarios
 - Inside CPUs as caches, registers, etc.
 - In FPGAs in the LUTs of logic elements, block RAM



A one-bit SRAM cell with 6 CMOS transistors.

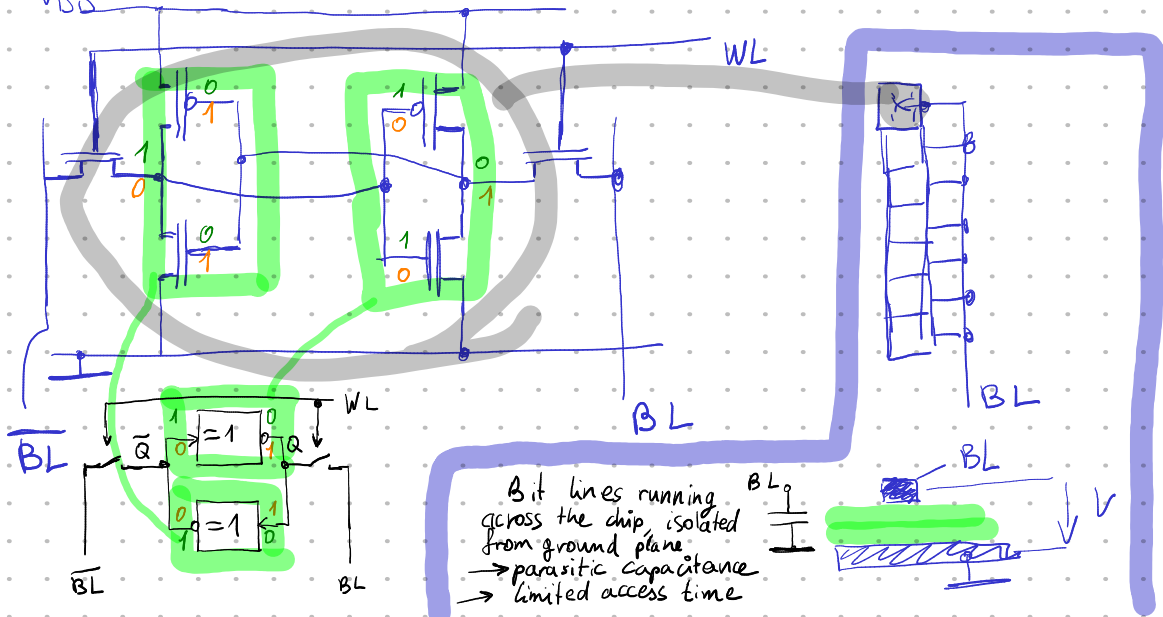
WL: word line. BL: bit line.

If WL is high, BL has the value of Q

A value is written by setting WL high and BL and $\overline{BL}$ to the desired value

Image source: User:Inductiveload, public domain via Wikimedia Commons
[https://commons.wikimedia.org/wiki/File:SRAM_Cell_\(6_Transistors\).svg](https://commons.wikimedia.org/wiki/File:SRAM_Cell_(6_Transistors).svg)

V_{DD} Two inverters in a positive feedback loop = "flip-flop"



Bit lines running across the chip, isolated from ground plane
 → parasitic capacitance
 → limited access time



DRAM

Dynamic RAM - Based on charged capacitors, periodic refreshing is necessary.

- Since 90s mostly used as SDRAM (synchronous DRAM) - synchronous to a clock signal
- Cheaper due to high density - 1 transistor and 1 capacitor per bit
- Slower:
 - Bit line capacitance *and* cell capacitance
 - Leakage current requires periodic refreshing
- Higher energy consumption due to necessary refreshing.
- Today popular: DDR (double data rate) SDRAM
⇒ 2 data words are transferred per clock cycle

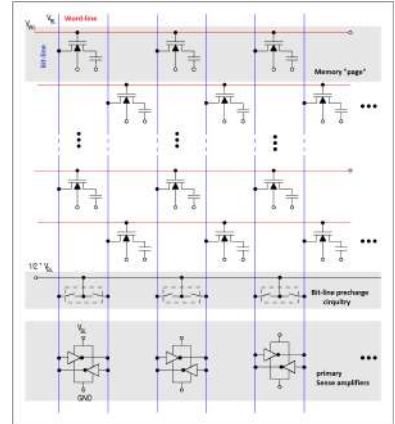


Image source: JürgenZ., Koza1983, CC BY-SA 3.0 via Wikimedia Commons
[https://commons.wikimedia.org/wiki/File:DRAM_cell_field_\(details\).png](https://commons.wikimedia.org/wiki/File:DRAM_cell_field_(details).png)

DDR-SDRAM

- Standardized by JEDEC^a as JESD79 to be interoperable between memory modules, CPUs, and mainboards.
- Standard interface: DIMM (Dual In-line Memory Module) and SO-DIMM (small outline DIMM), carrying multiple individual DRAM chips.
- Word width: 64 bit $\Rightarrow$ min. 8 chips with 8 bit width each
- Access via a command/address protocol on dedicated pins
- Prefetch: Read/write multiple words at once (e.g. one row), transfer in a sequence (burst) without own address commands
- Different generations (DDR, DDR2,..., DDR5):
increasing clock frequency and prefetch depth.
Also: Changes in command/address protocol

^aJoint Electron Device Engineering Council, organization of 365 companies

Image source: User:D-Kuru, CC BY-SA 4.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:RAM_SO-DIMM_8GB_by_Kingston-top_back_PNr%C2%B00565.jpg

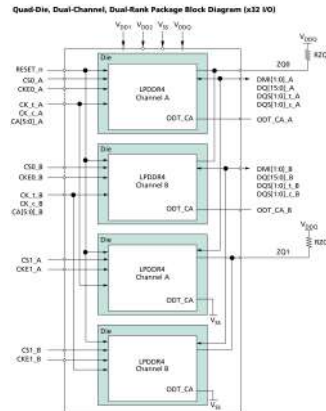


A DDR3 SO-DIMM with 8 GiB capacity.

Sometimes, with
8 chips
 $\rightarrow$ space for redundancy,
usable for forward
error correction (FEC) 10/32

LPDDR-SDRAM

- LPDDR = Low Power DDR SDRAM
Designed for mobile devices like smartphones, tablets, low-cost laptops, CPU-based *embedded systems*
- Smaller word widths possible
(e.g. 16 or 32 bit vs. 64 bit for DDR)
- More power efficient than DDR, but can be slower
- Soldered to PCB $\Rightarrow$ no need for DIMM socket
 - Lower cost, smaller/thinner form factor
 - Missing upgradeability is usually a drawback
 - But: HW-upgrades in field are rarely required on embedded systems



Block diagram of Micron MT53D1024M32D4 [2], used on Raspberry Pi 4B

Caches (1)

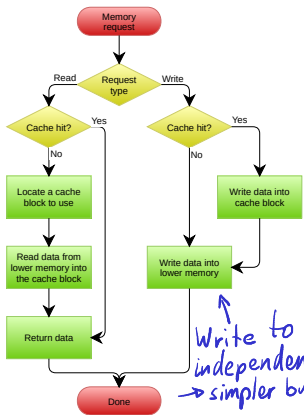
- DRAM access is slow compared to CPU clock frequency
But: Once a row is selected, multiple words can be read/written in a burst
- SRAM is faster than DRAM, but offers less capacity per chip area
- Usual approach: Use small SRAM inside a CPU as a (transparent) cache for DRAM
 - A cache contains a partial copy of the data in a slower memory
 - Cache is organized in *cache lines*, usually size of one DRAM burst
 - Whenever necessary, a cache line is read from DRAM and stored in the cache, or written back to DRAM

Caches (2)

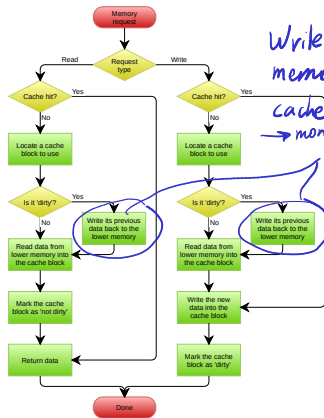
- Cache size is limited, so it is regularly overwritten by external RAM contents when addresses outside the cached range are accessed.
⇒ *Cache miss*, (as opposed to *cache hit*, when the cache contains the requested data)
- A cache manager ensures that data written by software is not lost when a new cache line needs to be read (see next slide).
 - either by immediately writing to DRAM (*write-through*)
 - or by storing the data in the cache and writing back to DRAM when another cache line needs to be read (*write-back*).
- Writing back and reading a new cache line takes time, during which the CPU is stalled.
⇒ Performance can be better if cache misses are avoided - by compiler or developer of a software, or by selecting a CPU with a bigger cache.

Note: Hardware aspects of the cache implementation are discussed in depth in e.g. courses by IKR .

Write policies



Write-through (with no-write allocation)



Write-back (with write allocation)

Image source: User:Flin00, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Write-through_with_no-write-allocation.svg,

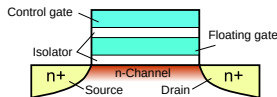
https://commons.wikimedia.org/wiki/File:Write-back_with_write-allocation.svg

- Operating the external memory requires a *memory controller* (MC) which is responsible for the command/address protocol.
- The MC is usually integrated into the CPU or SoC
- The software usually does not care about the memory access
The platform-dependent compiler or the CPU itself takes care of the details.
- If needed for an FPGA-based project, an MC needs to be implemented explicitly.
Usually IP cores can be used, provided by FPGA vendor or third parties.

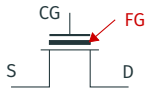
Persistent storage technologies

Semiconductor storage

- Based on floating-gate MOSFET transistors (FGMOS)
- Storage cell operation modes:
 - Charge: Apply *high* voltage to the drain (D) and control gate (CG), $\Rightarrow$ electrons are injected into the floating gate (FG)
 - Discharge: Apply *high* voltage to D only
 $\Rightarrow$ electrons are released from FG
 - Read-out: Apply a *lower* voltage to CG and D, measure current from D to S. $\Rightarrow$ If FG is charged, the transistor does not conduct and vice versa.
- If not charging/discharging, the charge in FG is stable for years.
- Precise injection and reading allows having more than two charge states.
 $\Rightarrow$ Multi-level cell (MLC), triple-level cell (TLC), quad-level cell (QLC) with 2, 3, 4 bits per cell.



Cross section of a FGMOS transistor



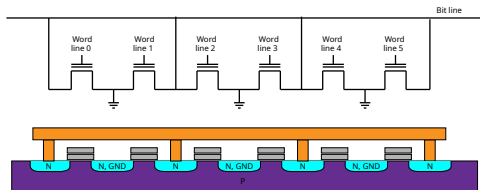
Circuit symbol of a FGMOS transistor

Flash memory (1)

Two types of flash memory, named after the logic gate they resemble in read mode.

NOR flash:

- Reading: Bit line is pulled to 0V, if any cell conducts current. (discharged and selected by WL).
- It is easy to read a single bit.



NAND flash:

- Reading: Bit line is pulled to 0V, if all cells in the row conduct current, especially the cell selected by WL.
- Reading in blocks is preferred.

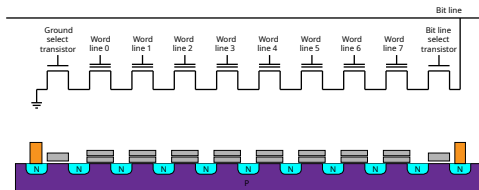


Image source: User:Cyferz at English Wikipedia, CC BY 2.5 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Nand_flash_structure.svg
https://commons.wikimedia.org/wiki/File:NOR_flash_layout.svg

Detailed process of reading and writing of the memory is not relevant for the lecture.

Further reading: [3] or <https://www.youtube.com/watch?v=V1dpqyUIhk> and <https://www.youtube.com/watch?v=sEs4MMQZqv4>

Flash memory (2)

- For both types, *erase* means discharging the FG
 - ⇒ The transistor will conduct current when reading
 - ⇒ By convention, this is interpreted as a logical '1'.
- Setting a bit to '0' is called *program*, and is achieved by charging the FG.
- Erasing is much slower than programming, and programming is slower than reading.
- NAND flash is cheaper than NOR flash because of higher density
- NAND flash is slower than NOR flash because of higher complexity and resistance of the cells connected in series.
- Typical usages: NOR = instruction memory in MCUs, NAND = data storage (USB drives, SSDs).

Flash memory (3)

- Programming granularity: *pages* - one row in the cell matrix, selected by a WL line, e.g. 512 bytes on NOR, 4 KiB on NAND
- Erasing granularity: *blocks*, e.g. 4 KiB on NOR, ≥ 128 KiB on NAND
- Data retention time: 10 years for both NOR and NAND
- Erasing prior to programming leads to wear-out of the insulation around the FG
 - Lifetime is specified in number of erase cycles
 - Typical: 10^5 to 10^6 on NOR and SLC NAND; 10^4 down to 10^3 on MLC/TLC/QLC NAND
- To avoid unnecessary erase cycles, DRAM buffers are used on NAND flash (to delay writing until it cannot be avoided). Also, wear leveling mechanisms (implemented by the storage controller) distribute erase cycles evenly over the whole memory.

Electrically Erasable Programmable Read-Only Memory (EEPROM)

- Similar to flash memory, different cell structure
- Two transistors per cell, ordered like in a NOR flash, allow programming and erasing of single bits.
- Slower and due to lower density more expensive than flash
- Single-bit cells allow more erase cycles than flash
- Used when small amounts of data need to be stored
- Usually provides a low-level serial interface (e.g. SPI or I²C) to the host CPU

Flash-based mass storage (1)

- Architecture: Flash memory (one or several chips, usually NAND flash) + controller
- Controller is responsible for
 - Accessing the flash memory chips (command/address protocol)
 - Providing a standard interface (e.g. USB, SATA, PCIe) to the host CPU
 - Wear leveling, Optionally: error correction (ECC), Sometimes: data encryption
- On the standard interface, the storage usually identifies as a block device (as opposed to a character device, e.g. a serial port)¹
- The software accesses the storage by reading/writing addressable blocks of a fixed size, comparable to *sectors* on a hard disk drive.
- The software *can* write *file system* metadata (e.g. FAT32, NTFS, ext4,...) to the storage, which allows to organize the data, rather than accessing the blocks directly¹.

¹More on this in a later lecture

Flash-based mass storage (2)

eMMC (embedded MultiMediaCard)

- One-chip device with integrated flash controller
- Special form factor of MMC, not removable but soldered directly to the PCB
- Serial interface, up to 8 data lines.
 - Accessed by a MMC controller in the CPU.
 - Slower than e.g. SSD (see later), but: Simpler interface, no need for a high-speed controller on the PCB
- Capacity: up to 256 GiB
- Frequently used as primary storage in embedded systems (including smartphones, tablets, etc.)



A 16 GiB eMMC used e.g. in a Samsung Galaxy Tab 2 10.1
Contains two 8 Gbit MLC NAND flash chips

Image source: Raimond Spekking, CC BY-SA 4.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Samsung_Galaxy_Tab_2_10.1_-_Samsung_KLMAG2GE4A-A002-3955.jpg

Flash-based mass storage (3)

SSD (Solid State Drive)

- Physical interfaces: SATA or PCIe, both possible in M.2 form factor
- Protocols: Advanced Host Controller Interface (AHCI) on SATA or PCIe, Non-Volatile Memory express (NVMe) on PCIe
- Capacity: up to 16 TiB (SATA), up to 100 TiB (PCIe)



Placeholders
allow more capacity
on same PCB

NAND flash

PCB and SATA interface of a 1 TiB SSD

Image source: Jacek Halicki, CC BY-SA 4.0 via Wikimedia Commons

[https://commons.wikimedia.org/wiki/File:](https://commons.wikimedia.org/wiki/File:2023_Dysk_SSD_Patriot_P210_1TB_wewn%C4%85trz.jpg)

2023_Dysk_SSD_Patriot_P210_1TB_wewn%C4%85trz.jpg



A 1TB SSD in M.2 2280, uses NVMe protocol over PCIe 4.0

Image source: D-Kuru, CC BY-SA 4.0 via Wikimedia Commons

[https://commons.wikimedia.org/wiki/File:](https://commons.wikimedia.org/wiki/File:Samsung_980_PRO_PCIE_4.0_NVMe_SSD_1TB-top_PNr%C2%B00915.jpg)

Samsu

Flash-based mass storage (4)

SD card (Secure Digital)

- Derived from MMC, but with more pins and higher density
- The most dominant removable memory card format. Has made all variants of MMC obsolete, except eMMC.
- Max. capacity: 2GB (original SDSC); 32GiB (SDHC); 2TiB (SDXC); 128 TiB (SDUC)
- Requires a card reader device implementing the SD protocol on the host side.

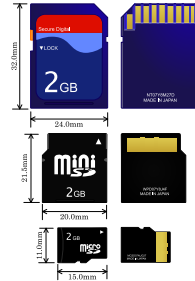


Image source: kenuki-commonswiki, Tkgd2007 (size comparison), CC BY-SA 3.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:SD_Cards.svg

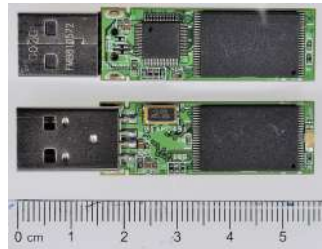
Image source: Manorainjan, CC BY-SA 3.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:SD-extremeMemory_2GB_alt_innen.jpg

Flash-based mass storage (5)

USB storage drives

- Protocol: USB mass storage device class (MSC)
- Usually a NAND flash chip and Flash+USB controller
- Sometimes, USB is an extra interface to access an SD card or an SSD with their own memory controller



USB drive with flash memory



USB card reader

Image source: Graham Beards (flash drive), CC BY-SA 4.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:USB_flash_drive2.jpg

Image source: Joydeep (card reader), CC BY-SA 3.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:A_Memory_Card_Reader_without_cover.jpg

Direct memory access (DMA)

Conventional memory access

Recapitulate memory-mapped access to peripherals:

- Software reads/writes a memory address in order to trigger a peripheral action or to transfer desired data
- If using load/store instructions, the data is transferred between a CPU register and the peripheral device
- Drawbacks when transferring chunk of data between a peripheral and RAM:
 - The CPU needs to be involved in every transfer ⇒ Slow and inefficient
 - The transfers have to be implemented in software
⇒ Error-prone, increases instruction memory usage.

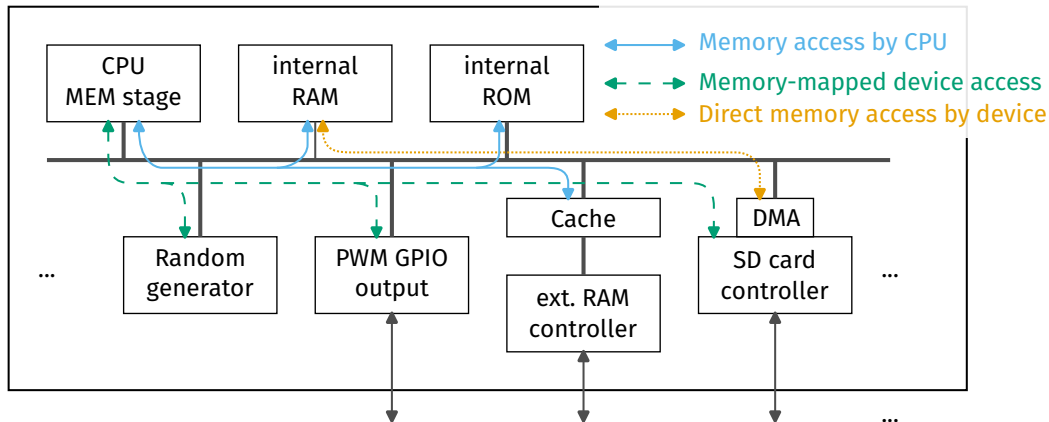
Direct memory access

Alternative to conventional memory access: Allow a peripheral device to access the memory directly, without CPU involvement

- The CPU configures the peripheral device (via a memory-mapped interface) to perform a transfer to or from the memory.
- The device performs the transfer, and signals the CPU when it is finished.
- Meanwhile, the CPU can continue with other tasks (processing the last transfer or preparing a new one)
- The device can e.g. maintain a constant data rate while the CPU processes the data in irregular intervals.

Direct memory access

Exemplary peripheral devices with different memory access modes in the ESP32 MCU



Direct memory access: Example

Network-to-audio streaming:

- A network interface controller (NIC) receives a data packet from the network at a random time and writes the packet to a memory location (DMA write)
- CPU is interrupted by the NIC, processes the packet with a random delay, and writes it to another memory location (DMA write)
- A digital-to-analog converter (DAC) reads the packet from the memory and converts it to an analog signal (DMA read)
- While the DAC is converting, the CPU can process the next packet, and/or do other tasks (e.g. user interactions)

Example: ESP32-based walkie-talkie project <https://github.com/atomic14/esp32-walkie-talkie>

Memory hierarchy (continued)

Memory hierarchy on a real-world MCU (example: ESP32)

- Compiler automatically makes the CPU use the fastest memory available for variables, i.e. registers first, SRAM second
- Optional external RAM is mapped to the address space, physically accessed via SPI
 - The software needs to explicitly tell the CPU, which variables should be placed in this memory (e.g. by special malloc() functions).
 - The hardware implicitly uses a cache in order to improve performance.
- Internal (and optionally external) flash memory for persistently storing variables is explicitly accessed by the software. For this purpose, driver functions are available, e.g. from Preferences.h or nvs_flash.h.

Memory hierarchy (continued)

Memory hierarchy on a real-world CPU (example: a Linux application)

- Compiler automatically makes the CPU use the fastest memory available for variables, i.e. registers first, (external) RAM second.
- The application operates in its own virtual RAM address space
 - Mapped to actual regions of the RAM by the operating system and the memory management unit (MMU) of the CPU.
 - Virtual memory is provided by the OS - address returned by calls to malloc().
- Storage devices (eMMC, SSD, removable storage) are accessed via drivers - both for the block device interface and for the file system.
- Application reads/writes the files via the file system driver to use storage space.
- If RAM is full, the OS can move data to the storage and back, transparently to the application (virtual addresses stay unchanged). Least recently accessed (LRA) data is moved first to reduce performance impact.

"paging",
"swapping"

On embedded, paging is not always enabled, then applications can be terminated "oom-killer"

Thank you for the attention!

Next lecture: Interfaces and protocols 1
(Device-local interfaces)

References

Bibliography

- [1] Raspberry Pi Ltd, *Raspberry Pi 4*, 2021.
- [2] Micron Technology, Inc, *Automotive LPDDR4/LPDDR4X SDRAM MT53D512M16D1, MT53D512M32D2, MT53D1024M32D4*, 2021.
- [3] K. Abbas, *Handbook of Digital CMOS Technology, Circuits, and Systems*.
Springer Nature Switzerland AG, 01/2020.

Hardware Platforms and Programming of Embedded Systems

Lecture 5 - Interfaces and protocols 1

Fundamental serial interfaces - UART, SPI, I2C & Co

Dr.-Ing. Alexander Knaub

2024-11-14

Institute of telecommunications (INÜ)

Contents

Pins of an IC

General Purpose Input/Output (GPIO)

UART

JTAG

Inter-integrated circuits I²C

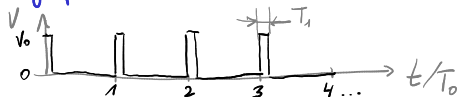
Serial Peripheral Interface (SPI)

Pins of an IC

Pins of an MCU IC

- Pins are the physical connections between the microcontroller and the outside world
- Some pins have special functions, e.g. power supply, clock, reset, etc.
- The function of most pins is driving or reading signals (digital or analog)
- If used for communication, software or hardware components implementing the communication protocol have control over a pin.

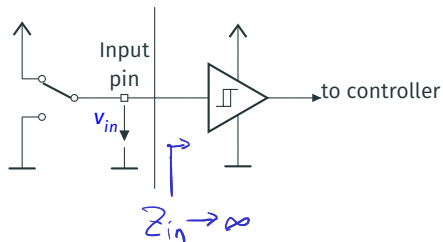
also: analog-appearing digital signals:
e.g. pulse width modulation (PWM)



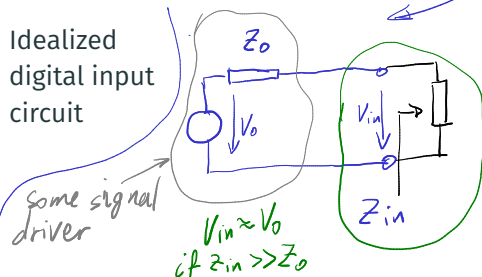
→ Varying the duty cycle T_1/T_0 in $V(t)$ changes the average intensity.

Digital inputs

- Digital input is an amplifier (more precisely, a comparator with a constant reference voltage), converting the potentially arbitrary voltage on the pin to a voltage value easily interpretable as high or low by the processor.
- An input pin should have a high input impedance Z_{in}
 - Very little current flows into or out of the pin
 - Does not load the signal source, i.e. does not change the voltage of the signal source

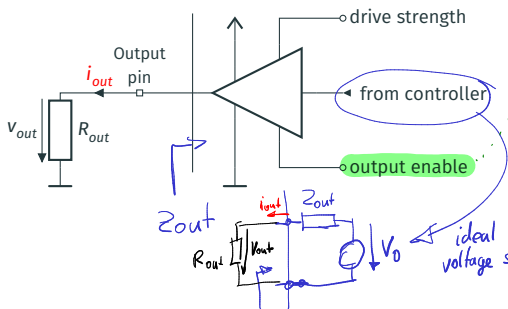


Idealized
digital input
circuit



Digital outputs

- Digital output is a driver circuit, forcing the pin voltage v_{out} to the high or low level.
- An output should have a low output impedance ($z_{out} \approx 0$)
 - Can drive a current into or out of the pin (shortcut to ground or to supply voltage)
 - Keeps the voltage of the pin constant despite the current (within appropriate limits)
 - Can be disabled to prevent undesired short-circuits

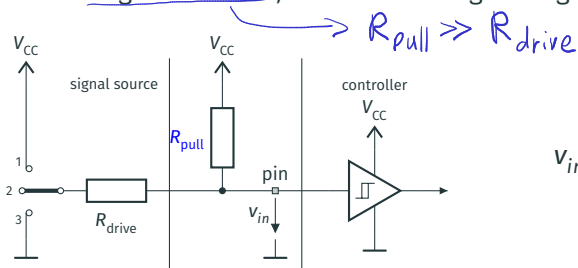


$$i_{out} = \frac{v_{out}}{R_{out}}$$

ideal voltage source, $v_{out} \approx V_0$ if $z_{out} \approx 0$

Pull-up and Pull-down

- *Pull-up* and *pull-down* are resistors that are connected to the pin
- Pull-up is connected to a positive supply voltage, pull-down is connected to GND
- Used to set a defined value of a pin, if no external driver is connected (floating pin).
- Have a high resistance, to avoid loading the signal source too much (e.g. 50 k Ω)



$$v_{in} = \begin{cases} V_{CC} & \text{if switch} = 1 \\ V_{CC} & \text{if switch} = 2 \\ V_{CC} \cdot \frac{R_{drive}}{R_{drive} + R_{pull}} & \text{if switch} = 3 \end{cases}$$

Voltage levels of pins

Different standardized voltage levels are used for digital pins. The standards define the voltage levels for high and low, the limits for driving a pin, and the thresholds for detecting a signal as high and low.

Name	High	Drive high	Read high	Low	Drive low	Read low
TTL	5 V	2.4 V	2 V	0 V	0.4 V	0.8 V
LVTTL	3.3 V	2.4 V	2 V	0 V	0.4 V	0.8 V
CMOS	5 V	4.5 V	3.5 V	0 V	0.5 V	1.5 V
LVC MOS	3.3 V	3.0 V	2.1 V	0 V	0.3 V	0.9 V
	2.5 V	2.25 V	1.75 V	0 V	0.25 V	0.75 V
RS-232	15 V	10 V	3 V	-15 V	-10 V	-3 V

*Serial port, e.g.
on older computers*



Higher voltages can help to increase signal-to-noise ratio (SNR) and to maintain signal integrity over longer distances. Lower voltages have lower power consumption and electromagnetic interference (EMI), and can be faster.

LVDS

LVDS = Low Voltage Differential Signaling

- Uses a differential signal, i.e. two signals with opposite polarity
- The difference between the two signals is small, e.g. 350 mV
- The receiver ignores the common mode voltage, i.e. the average of the two signals
⇒ Noise that is common to both signals has no effect on the receiver
- Commonly used for high-speed communication (up to 4 Gbit/s), e.g. video stream between CPU and display or camera

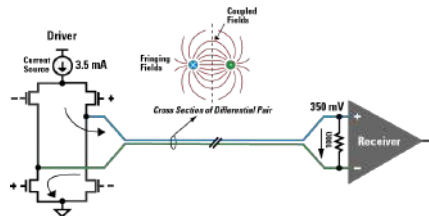


Image source: Dave at ti, CC BY-SA 3.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:Basic_LVDS_circuit_operation.png

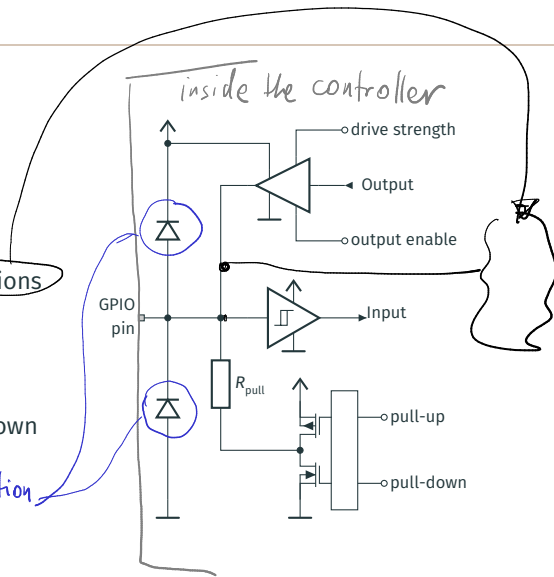
General Purpose Input/Output (GPIO)

GPIO

GPIO = General Purpose Input/Output

- We call a pin GPIO when it is used to read or write individual digital values (high or low), e.g. for controlling LEDs, reading buttons, etc.
- Usually GPIO pins can have other peripheral functions as well. The software can select the function via a memory-mapped register of the so-called IO multiplexer inside the CPU.
- GPIO pins can have configurable pull-up or pull-down resistors, drive strength, etc.

EMC protection
(static discharge protection)



ESP32 GPIO example: Arduino framework

```
// GPIO pin numbers
#define LED_PIN 6
#define BUTTON_PIN 2
① pinMode(LED_PIN, OUTPUT);
// actually set the PIN to high/low
② digitalWrite(LED_PIN, HIGH);
digitalWrite(LED_PIN, LOW);
① pinMode(BUTTON_PIN, INPUT);
② int button_state = digitalRead(BUTTON_PIN);
```

①: configure as input/output
②: read/write the actual value

The order differs between this and the next slide,
this has no practical reason.



Image source: SparkFun Electronics, CC BY 2.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:15663-SparkFun_Thing_Plus_-_ESP32_WROOM-01.jpg

ESP32 GPIO example: Memory-mapped access

```
// GPIO register addresses (defined in ESP32 headers)
#define DR_REG_GPIO_BASE 0x3ff44000
#define GPIO_OUT_REG (DR_REG_GPIO_BASE + 0x0004)
#define GPIO_OUT_W1TS_REG (DR_REG_GPIO_BASE + 0x0008) // all ones will be set, zeros ignored
#define GPIO_OUT_W1TC_REG (DR_REG_GPIO_BASE + 0x000c) // all ones will be cleared, zeros ignored
#define GPIO_ENABLE_REG (DR_REG_GPIO_BASE + 0x0020) // 1 = output, 0 = input
#define GPIO_IN_REG (DR_REG_GPIO_BASE + 0x003c)

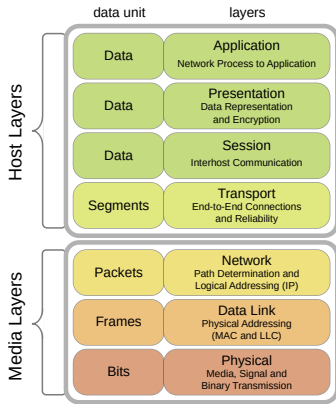
// GPIO pin numbers we want to use
#define LED_PIN 6
#define BUTTON_PIN 2

// LED_PIN as output, all others unchanged
① 1 REG_WRITE(GPIO_ENABLE_REG, GPIO_ENABLE_REG | (1 << LED_PIN));
// BUTTON_PIN as input, all others unchanged
① 1 REG_WRITE(GPIO_ENABLE_REG, GPIO_ENABLE_REG & ~(1 << BUTTON_PIN));

// actually set the PIN to high/low
② 1 REG_WRITE(GPIO_OUT_W1TS_REG, 1 << LED_PIN); // set one bit
② 1 REG_WRITE(GPIO_OUT_W1TC_REG, 1 << LED_PIN); // clear one bit
// read the button state
② 1 int button_state = REG_READ(GPIO_IN_REG) & (1 << BUTTON_PIN);
```

ISO/OSI reference model

ISO Open Systems Interconnection (OSI) Reference model



- Conceptual model for communication systems. Defines seven layers, each with a specific function
- The layer functions are implemented by hardware components (PHY) or software (drivers, protocols)
- A layer passes data to the layer above or below. Ideally, these interfaces are well defined, so implementations can be exchanged.
- The layers are not mandatory: A communication system can omit or merge layers
- Exemplary protocol stack (bottom up): Ethernet (1&2), IP (3), TCP (4), HTTP (5), JSON (6), REST API (7)

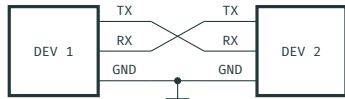
Image source: Offnft, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:OSI_Model_v1.svg

UART

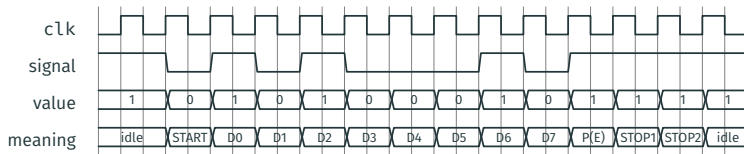
Universal Asynchronous Receiver Transmitter (UART)

- A layer-2 (Data Link) implementation for serial communication between *two* computers
⇒ Topology: Point-to-point
- "Asynchronous" means that no clock signal is part of the interface
Start of a message can occur at any time, Receiver adjusts its clock to data stream
- Two wires mandatory: TX (transmit), RX (receive).
GND as voltage reference is also required unless devices are galvanically isolated
- Layer 1 additionally specifies more control signals and voltage level
e.g. RS-232 using UART and signals like "Ready to send" (RTS), "Clear to send" (CTS), etc. on -15 V ...15 V (found e.g. on the COM port of a PC)
- Higher OSI model layers often use UART to transmit text,
but UART can transmit arbitrary binary data, allowing application-specific protocols.



UART (2)

- Sender and receiver must be configured with identical communication parameters:
 - Baud rate (bits per second) - between 50 bps and 3 Mbps
9600 bps or 115200 bps are commonly used
 - Number of data bits (5 ...9, usually 8), LSB first
 - Parity (none, even, odd) - for error detection
 - Number of stop bits (usually 1 or 2) - pause between messages, resynchronization
- **USART**: UART with optional **synchronous** clock signal.
Not commonly used today, although some MCUs do have a USART interface.



Example parameters: "115200 8E2"

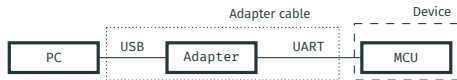
- 8 Data bits between start and stop bits, bit duration 8.68 μ s
- Parity bit: Even (set to 1 if data has an odd number of 1s)
- 2 Stop bits

UART - usage

- UART is commonly used for observing debug output of a microcontroller, sometimes also for bidirectional communication between microcontroller and PC
- Also: Embedded Linux devices can use UART for console input/output
In this context, UART is called *serial console* or *TTY* (teletype)
- UART can also be used for communication between a MCU and a peripheral IC.
 - The peripheral IC (sensor/actuator) must have a UART interface
 - The MCU must have a UART controller or a software implementation of UART
i.e. two dedicated pins per peripheral are required
⇒ not well suitable for connecting *many* peripherals to a MCU

USB UART Adapters

- Often USB to UART adapters are used to connect a UART device to a PC. On Linux, such interfaces are called `/dev/ttyUSB<N>`, in contrast to `/dev/ttyS<N>` used for serial interfaces integrated into processor.
- USB to UART adapters can be integrated into USB cables or placed as chip on a PCB
- Often the adapters are based on a chip from FTDI (Future Technology Devices International)
- By using an adapter chip, a device can be connected to a PC via USB and be usable as UART device



JTAG

Joint Test Action Group (JTAG)

- JTAG is a standard for testing integrated circuits.
- Topology: Daisy chain, signal is passed through all devices and sometimes individual components within the device
- Originally designed for testing PCBs by accessing individual pins of a chip
- For firmware developers, mostly interesting because of capability to access the internal signals of a chip
- ARM Cortex-M MCUs often offer a Serial Wire Debug (SWD) interface instead, which offers similar functionality

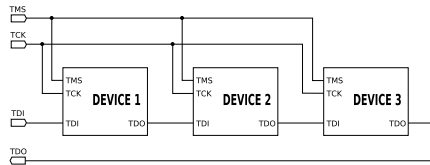


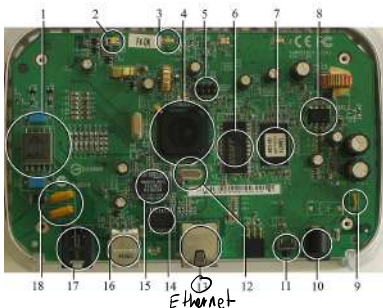
Image source: Vindicator, CC BY 2.5 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:Jtag_chain.svg

The internal state of a chip can be read and modified via JTAG. Possible use cases:

- Access an MCU's flash memory, or an FPGA's SRAM configuration memory
- Debug the program running on a microprocessor:
Read and write memory and registers, set breakpoints, step through instructions etc.
- Read and write some registers and memory blocks of an FPGA:
Allows rudimentary debugging of the FPGA configuration
- Due to daisy-chain topology, knowledge of the internal structure of the chip is required, making vendor-specific hardware (adapters) necessary for easy operation.
- Software can be vendor-specific or open source, e.g. OpenOCD

JTAG - Security considerations (1)

- JTAG is useful and often necessary for development and testing, therefore often prominently exposed on development/experimental boards.
- Sometimes also more or less accessible on products that are sold to customers, to allow analysis on devices that are returned by customers (fix a "bricked" device).



⇐ PCB of an ADSL modem/router

JTAG connector is populated:

Pin header labeled #5

Side note - Other visible components:

- 4 - CPU (TI AR7)
- 6 - RAM
- 7 - Flash ROM
- 12 - Quartz oscillator
- 8 - Power regulator
- 14 - Ethernet transformer
- 15 - Ethernet PHY

Image source: Mike1024, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:ADSL_modem_router_internals_labeled.jpg

JTAG - Security considerations (2)

- On a product that is sold to customers, JTAG can be a security risk because it allows:
 - reading out the firmware, gaining secret knowledge
 - modifying the firmware, gaining undesired control over the device, e.g disabling security features
 - It is customary to disable JTAG access to a chip in a product that is sold to customers.
 - Board-level measures: Not soldering or even not routing the JTAG pins on the PCB, or routing it but making unfunctional with e.g. superfluous connections (can be hacked by cutting PCB tracks and/or soldering wires to the chip)
 - Chip-level measures:
 - Non-permanent writing of a special register in the chip by the firmware (can be hacked by interrupting boot process)
 - Permanent "burning" a one-time writable fuse bit in the chip
- Quite secure, but irreversible.

Finished here on 14.11.2024

Inter-integrated circuits I²C

Inter-integrated circuits I²C

- Logical topology: *Star* - one *Controller* device controls the communication, multiple *target* devices can be connected
- Physical topology: *Bus* - multiple devices can be connected to the same pair of wires: SCL (clock) and SDA (data)
- The wires are pulled up to a positive voltage, device pulls the wire to ground to transmit a low bit (open drain circuit).
- The pull-up resistors and the parasitic capacitances form an RC low-pass filter, limiting the maximum speed.
- Predefined SCL frequencies:
100 kHz, 400 kHz, 1 MHz, 3.4 MHz

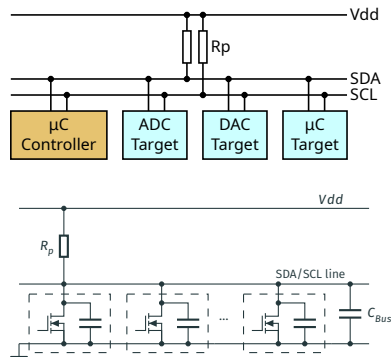


Image source: Tim Mathias, CC BY-SA 4.0 via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:I2C_controller-target.svg

Inter-integrated circuits I²C - protocol

- The devices are addressed by a 7-bit address
 - The 7-bit address is usually fixed for a product
 - Some devices allow to set one or several bits of the address via a pin

This way, two identical devices can be connected to the same bus.
- The controller initiates communication by sending a START condition (S)
High-to-low transition of SDA with SCL high
- Data transfer is done in 8-bit bytes, MSB first, with transitions of SDA during SCL low
Each byte is acknowledged by the receiver by pulling SDA low during the 9th clock pulse
- The controller ends communication by sending a STOP condition (P)
Low-to-high transition of SDA with SCL high

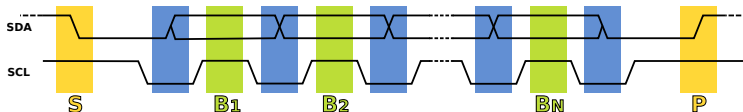


Image source: Marcin Floryan, public domain via Wikimedia Commons
https://commons.wikimedia.org/wiki/File:I2C_data_transfer.svg

I²C - Usage

- Sensors, actuators
- Small storage devices, especially EEPROMs
- System Management Bus (SMBus) is a stricter subset of I²C, used for communication with power supplies, batteries, temperature sensors, etc. Part of PCI and PCIe interface definition.
- Two-wire interface (TWI) is an incomplete implementation of I²C, used by Atmel AVR MCUs, compatible with some I²C devices.

Serial Peripheral Interface (SPI)

Serial Peripheral Interface (SPI)

- Logical topology: *Star* -
 - one *Main* device controls the communication, multiple *Subnode* devices can be connected
 - Other names for participants:
Controller/Peripheral, Master/Slave (discouraged)
- Signal names: SCLK (clock), MOSI (main out, sub in), MISO (main in, sub out), CS (chip select)
- Two physical topologies possible:
 - *Bus* on MISO/MOSI and SCLK, *Star* on CS:
 - same data signal for all devices, one CS per device
 - *Daisy chain* on MISO/MOSI, *Bus* on SCLK and CS (exotic):
 - data signals are passed through all devices, same CS for all devices

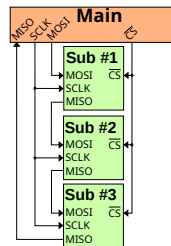
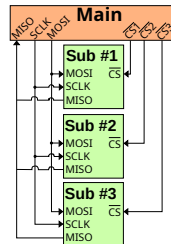


Image source: Em3rgent0rdr, public domain via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:SPI_main_sub_multidrop.svg

https://commons.wikimedia.org/wiki/File:SPI_main_sub_daisychain.svg

SPI - Protocol

- SPI is a layer-2 protocol: No specification for voltage levels and data contents
- Data transmission happens bidirectionally while a chip select (CS) line is pulled low
- The subnode selected by CS sends data to the main node on MISO, other subnodes switch their MISO outputs to high impedance (Z).
- The Main device generates the clock signal on SCLK while a subnode is selected. Polarity of SCLK and phase shift between SCLK and MISO/MOSI can be configured by setting parameters CPOL (0 or 1) and CPHA (0 or 1) in Main device.
- Number of bits per transfer is not specified.
- Parallel data transmission is possible by using multiple MOSI/MISO lines e.g. Quad SPI (QSPI) With 4 MOSI/MISO lines is commonly used by storage chips.

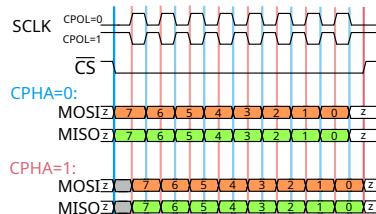


Image source: Cburnett, CC BY-SA 4.0 via Wikimedia Commons

https://commons.wikimedia.org/wiki/File:SPI_timing_diagram_CS.svg

- Sensors, actuators
- Storage devices (EEPROM, flash, etc.) - often QSPI
- Implicitely used by some other interfaces, e.g. SD card

Completed 21.11.2024

The end

Thank you for the attention!

Next lecture: Interfaces and protocols 2
Inter-device interfaces